Evaluating Caching Policies for IVF-based ANN Engines under Tiered DRAM and SSD Storage

Lucas Ding

1. Abstract

Approximate nearest neighbor search (Approximate Nearest Neighbor, ANN) is the core operator of vector retrieval systems, many applications need to find the most similar top-k vectors from massive vector collections within extremely short time. The problem is that vector indexes are often very large, the cost of putting everything into DRAM is high, while placing data on SSD will push latency to the millisecond level, and will especially significantly worsen tail latency (p95) . Classic conclusions like the Five Minute Rule tell us that the huge differences in cost and performance across storage hierarchies force systems to do tiering and cache management, and the key challenge is: which data should reside permanently in DRAM, and which can stay on SSD.

This project combines an ANN engine with a tiered DRAM + SSD model, uses an IVF structure as the index backbone, and manages the residency of IVF inverted lists (IVF lists) between DRAM and SSD with different caching policies. We compare multiple policies, including the two extreme baselines all DRAM and all SSD, the traditional recency policy LRU, the traditional frequency policies naive LFU and window LFU, and the seconds rule policy we implemented.

Through a unified experimental pipeline, we run parameter sweeps under different DRAM ratios, different nprobe values, and different SSD IOPS constraints, examining the recall–latency tradeoff, as well as I/O amplification and migration overhead. Here, nprobe indicates how many clusters / inverted lists are accessed per query in an IVF index, a larger nprobe means more candidates and higher accuracy, but also higher compute and I/O cost; recall measures what fraction of the “true correct answers” that should be returned are actually retrieved by ANN; I/O amplification describes how many extra times the system is forced to read from SSD in order to obtain a small number of useful results.

Our goal is to maintain acceptable recall as much as possible under limited DRAM capacity, while significantly reducing tail latency caused by SSD reads, and keeping migration overhead within a reasonable range, thus providing policy-selection recommendations for practical system design. The experimental results show that system latency is highly controlled by the number of SSD pages read, and p95 latency is mainly determined by I/O amplification. Increasing nprobe can steadily improve recall, but it also linearly amplifies the size of inverted lists that must be scanned, further amplifying SSD I/O. When comparing different policies, window LFU and naive LFU can often trade relatively low migration overhead for some degree of I/O reduction across multiple

workloads; while in scenarios where hotspots change faster, LRU and seconds rule are more able to track hotspots in time, thereby further reducing SSD reads, but the migration cost will also increase significantly.

In addition, under the page-level latency model used in this work, if the SLA (Service Level Agreement, service-level agreement, that is, the latency threshold the system promises externally) is set to 200 microseconds or 500 microseconds, apart from all DRAM, the other policies are difficult to meet the target within the given parameter range. This indicates that merely placing part of the IVF lists into DRAM is not enough, and stronger I/O-avoidance methods are needed next, such as more aggressive compression and tiered candidate filtering mechanisms.

2. Introduction

2.1 Problem Motivation

Vector retrieval is increasingly common in real systems: image search encodes images into vectors, semantic search and large-model retrieval augmentation encode text or dialogue history into vectors, recommendation systems represent user and item features as vectors, log analysis compresses user behavior sequences into vector embeddings. All these applications after going online will face a common problem: the scale of the vector database will quickly expand to millions, tens of millions, or even hundreds of millions or more, and the dimension of each vector is often between 100 ~ 1000. Doing exact nearest neighbor search directly requires comparing each query vector with every vector in the database one by one, with complexity roughly $O(Nd)$, which is not only computationally huge, but also causes a lot random memory accesses, in high-concurrency scenarios it cannot sustain latency and is also hard to meet throughput requirements. Therefore engineering practice usually adopts Approximate Nearest Neighbor (Approximate Nearest Neighbor, ANN) techniques, ANN is a class of vector retrieval methods that allow results to be only “approximately nearest” rather than absolutely nearest neighbors, trading a small loss of accuracy for significantly lower latency and higher throughput.

The real engineering difficulty is: the index structure and the vector data itself often cannot be fully placed in DRAM. Take a conservative example, 10^9 128-dimensional float32 vectors is already close to 512GB, and the accompanying index structure will additionally occupy a large amount of space. In real systems it is hard to equip a single service with so much DRAM, and DRAM cost and power consumption are very high. SSD has much larger capacity and is much cheaper, which looks like a more natural medium, but SSD random access latency is several orders of magnitude higher than DRAM: DRAM access is usually tens of nanoseconds, while a single random SSD read is often tens to hundreds of microseconds. For a single query, even if the average latency is barely acceptable, this gap will very obviously push up tail latency (such as p95 / p99) , directly affecting online SLA and user experience——especially in multi-tenant, high-concurrency, and long pipelines (for example “retrieval + reranking + model inference”) scenarios, millisecond-level jitter in retrieval is easily amplified.

This naturally turns the problem into a tiered storage and cache management problem: under the premise that “DRAM is fast but expensive, SSD is slow but cheap”, we must explicitly decide which data should reside in

DRAM, which data can stay only on SSD, and how the system should dynamically adjust the boundary between these two parts according to access patterns at runtime. The core idea given by classic conclusions like the Five Minute Rule is: whether data should be placed on a faster medium cannot be decided only by capacity constraints, but must also consider access frequency, data size, and the price and access cost per unit capacity of different media; for sufficiently frequently accessed objects, placing them on a more expensive but faster medium (such as DRAM) is worthwhile, while sparsely accessed cold data should remain on a cheaper medium (such as SSD) . In vector retrieval scenarios, query distributions across different applications, different stages, and even different time periods can be completely different: some workloads have long-term stable and concentrated hotspots, some workloads have rapidly migrating hotspots, and some are close to uniform. These differences directly change the optimal decision of “which data is worth keeping in DRAM”, and therefore also require us to systematically compare the performance of different caching policies under these workloads, rather than hoping that one fixed policy “performs well in all scenarios”.

2.2 Research Question

This project focuses on several more specific and actionable questions, rather than vaguely saying “vector retrieval is slow on SSD”:

- **What does tiered management look like when using IVF lists as the caching unit?**
  In ANN engines with inverted-list structures like IVF, each cluster corresponds to an inverted list (IVF list) , and each query will access only a small number of clusters. We treat each list as a “block” that can be placed in DRAM or SSD, that is, we do caching and migration at list granularity: if a list is hot we try to move it to DRAM, if it becomes cold we put it back on SSD. A key question is, under this setting, what the decision space of tiered management looks like——for example, when to trigger migration, how much to migrate each time, and what signals to use to judge whether a list is worth residing in DRAM.
- **When DRAM is tight, how much tail latency can policies really save?**
  In reality DRAM can only hold a small fraction of all lists, and many queries inevitably have to access SSD. We want to systematically evaluate: under a fixed DRAM fraction (for example only 5% or 20% of lists can fit) , how different caching policies change the overall performance curve of queries——specifically, on one hand whether recall is affected by “skewed hot/cold residency decisions”, and on the other hand how much average latency and tail latency like p95 can be reduced, and whether there exists some policy combination that, under a given DRAM cost, can bring p95 down to an engineering-acceptable level.
- **Under what workloads does a policy like seconds rule that “emphasizes reuse interval” have an advantage?**
  Traditional LRU mainly looks at “time of most recent access”, LFU mainly looks at “cumulative access count”, while seconds rule cares more about the “reuse interval”, that is, how many queries elapse between two accesses. What we want to ask is: under different access distributions such as default, fast-changing hotspots, slow-changing hotspots, uniform, and Zipf, does this reuse-interval-based signal fit data access patterns better than pure recency or pure frequency? For example, when the hotspot window moves very fast, can seconds rule more accurately distinguish “short-interval high-value reuse” from “cold

data that is touched only occasionally", thus achieving a higher DRAM hit rate and lower SSD access volume under the same DRAM capacity.

- **Will the benefits brought by policies be eaten up by migration cost?**
  Moving lists in and out of DRAM itself requires I/O and CPU, which is not free in real systems: migration consumes SSD bandwidth, affects concurrent queries, and may also increase code complexity and maintenance cost. Therefore we not only compare policy benefits in recall and latency, but also explicitly count total migrated bytes, the average migration overhead amortized per query, and how many clusters need to be migrated per rebalance. The concrete questions we want to answer are: under different workloads and DRAM configurations, whether there exists a policy that shows "great tail latency on paper, but the amount of data migrated per second has already reached an unacceptable level"; and under an acceptable migration budget, which types of policy combinations are more cost-effective.

2.3 Contributions

The main work of this project can be summarized as follows:

- **Built a tiered model for IVF-style ANN**
  Around the IVF index, we abstract the system as a combination of "ANN computation + tiered storage": explicitly distinguishing DRAM and SSD as two tiers, and using a configurable dram fraction parameter to control what fraction of IVF lists DRAM can hold. On this basis, we introduce a page-level I/O latency model, decomposing SSD reads into "number of pages × cost per page", so we can use a unified formula to estimate the I/O latency of each query, and distinguish it from ANN computation time. This model allows us to directly derive the source of p95 latency changes from statistics such as `avg ssd pages` and `avg ssd bytes`, providing quantitative support for the subsequent analysis that "latency is mainly determined by I/O amplification".
- **Implemented and compared multiple caching policies, covering extreme baselines, classic policies, and seconds rule**
  On top of the tiered model above, we implemented multiple list-level caching policies: including the two extreme baselines "all lists in DRAM" all DRAM and "all lists in SSD" all SSD, to provide an ideal lower bound and a worst upper bound; including LRU based on recency signals, and naive LFU and window LFU based on frequency signals, to represent classic policies commonly used in practice; and the seconds rule policy we implemented based on the reuse-interval idea, to explore whether reuse interval can provide a better decision signal when hotspots change quickly. These policies share a common interface, so they can be compared directly within the same experimental pipeline.
- **Systematically evaluated policy behavior under multiple workloads and parameter combinations**
  To avoid drawing one-sided conclusions from a narrow scenario of "single distribution + single parameter", we designed multiple access distributions: including a default distribution close to real traces, two hotspot distributions with fast-changing and slow-changing hotspots, a fully uniform distribution, and a Zipf distribution with long-tail characteristics (s=1.2) . Under each workload, we systematically swept key parameters such as DRAM ratio (for example 5%, 20%) , nprobe (1, 4, 16) , and SSD max IOPS (1M, 5M) , and uniformly recorded metrics such as recall, average latency, p95/p99, I/O amplification, SSD pages

read, and migrated bytes, so as to observe how policies behave under different “space budgets” and “hardware constraints”, rather than looking at only a single point result.

- **Provided unified result export and analysis assets, forming a reusable experiment and report framework**
  Aggregate raw per-query statistics into `ann_policy_agg.csv` , including mean, standard deviation, confidence intervals, and so on, to facilitate subsequent statistics or plotting;
  Automatically compute the maximum reachable recall for each policy under different SLAs (for example 200µs, 500µs) , output `sla_reachable_recall.csv` , making it easy to compare policies from the perspective of “performance promises”;
  Generate several key tables (for example the key-point summary table, the SLA table) and multiple visualization figures (p95 vs DRAM, I/O amplification vs DRAM, migration overhead vs DRAM, recall–latency frontier, and so on) , and collect them uniformly in the `report_assets` directory;
  Finally stitch everything together into a master report with a one-click script.
  This asset set makes it so that if new policies are added later, a new dataset is used, or hardware parameters are changed, one only needs to reuse the same pipeline to obtain an analysis report with consistent style and complete structure.

3. Background & Related Work

3.1 Approximate Nearest Neighbor and IVF

The goal of approximate nearest neighbor (Approximate Nearest Neighbor, ANN) is: given a query vector q, find the k vectors with the smallest distance in the database vector set $X = x_i$. Here, the “distance” is usually Euclidean distance or inner-product distance. The most straightforward exact approach is to compute the distance between q and all x_i for every query, with time complexity roughly $O(Nd)$ (where N is the number of vectors, and d is the dimension) , when N reaches the million or hundred-million scale, this linear scan is unacceptable in both latency and throughput, so engineering systems widely adopt ANN techniques, trading a controlled loss in accuracy for lower latency.

The inverted file index (Inverted File Index, IVF) is a classic ANN structure. Its core idea is to make an explicit two-stage “coarse-to-fine” search:

- In the offline stage (index building) , first use a coarse quantizer to partition the vector space into $nlist$ clusters, which can also be understood as training a set of centroid vectors $c_j{}_{j=1}^{nlist}$. Each database vector x_i is assigned to its nearest centroid c_j, and is appended to the inverted list corresponding to that cluster, and this list is an IVF list.
- In the online query stage, first compute the distance between q and c_j over all cluster centroids, select the nearest few centroids, and then only scan the inverted lists corresponding to these clusters, compute the exact distance to q for the candidate vectors in the lists, and then select top-k from the merged candidate set.

In the query stage, the number of scanned clusters is usually denoted as $nprobe$. Intuitively, $nprobe$ is a key accuracy–performance knob:

- The smaller $nprobe$ is, the fewer lists each query accesses, the fewer distance computations and storage reads are needed, so latency is lower, but it is easy to miss true nearest neighbors, and recall will drop;
- The larger $nprobe$ is, the larger the candidate set and the wider the covered cluster range, so it is less likely to miss the true neighbors, recall usually increases monotonically, but compute cost and I/O cost also roughly scale proportionally.

In engineering practice, open-source libraries such as Faiss have made IVF a fairly general component, and support combining it with compression techniques such as Product Quantization PQ, for example IVF-PQ、IVF-OPQ etc. Under fixed hardware resources, systems can tune parameters such as $nlist$, $nprobe$, the number of PQ subspaces, and codebook size, to find a tradeoff among speed, memory footprint, and recall that fits a specific workload.

This project chooses IVF as the research object mainly for two reasons:

- The candidate set is naturally “grouped by cluster”: IVF partitions the database into inverted lists, each list corresponds to a cluster, so we can very naturally treat “one list” as the basic unit of caching and migration, which corresponds in implementation to “whether the IVF list of a cluster resides permanently in DRAM, or is stored only on SSD and read on demand”. This is easier to manage and model than caching at the granularity of a “single vector”.
- The cost decomposition is clear, making it easy to combine with an I/O model and caching policies: the main cost of an IVF query can be cleanly split into two parts: the cluster-selection stage computes distances to all centroids in memory, this part almost does not involve external storage access; the list-scanning stage accesses several lists, which includes both the scan/compute overhead for lists in DRAM, and the page-level read overhead for lists on SSD.

This structure is very suitable for layering on the DRAM + SSD tiered model and the page-level I/O latency model we construct, so we can clearly see the direct impact of strategies like “which lists to cache in DRAM” on SSD I/O volume, latency distribution (especially p95) , and overall system performance.

3.2 Tiered Memory and Caching Policies

The basic setting of tiered storage is: DRAM is fast but expensive, SSD is slow but has large capacity and is cheap. In vector retrieval scenarios, the index structure and vector data are usually much larger than DRAM capacity, so it is impossible to keep everything resident in memory, and the system must decide which data goes into DRAM and which can only stay on SSD. The core view of the Five Minute Rule is: which tier an object should be placed on depends not only on capacity constraints, but also on its access frequency and the unit cost of different media, we cannot assume access is uniform, and we also cannot look only at “how much fits”.

When designing caching policies in practice, the two most commonly used signals are recency and frequency. recency believes that “data accessed recently is more likely to be accessed again soon”, with a typical policy being LRU (Least Recently Used) . LRU always prioritizes keeping the most recently accessed data, and evicts those that have not been accessed for the longest time, its advantage is that it is sensitive to hotspot switches and can quickly follow new hotspots, its disadvantage is that it is easily polluted by one-time scans: a large batch of data accessed only once will be temporarily treated as “hot”, squeezing truly long-term valuable objects out of DRAM.

frequency believes that “data accessed many times in the past will continue to be accessed in the future”, with a typical policy being LFU (Least Frequently Used) . LFU maintains an access counter for each object, and prioritizes keeping objects with high cumulative access counts, it is suitable for scenarios where hotspots are relatively stable and is not easily disturbed by a single scan. But the problem of naive LFU is also obvious: the counter accumulates without bound, early hotspots may occupy the cache for a long time, and new hotspots need a long time to “build up” sufficiently high counts, so a window or decay mechanism is needed to gradually forget old history.

To leverage both recency and frequency, engineering practice often uses window LFU or LFU with decay. window LFU can be understood as “only counting accesses within a recent window”, history outside the window is either discarded directly or given very low weight, thus preventing early hotspots from permanently occupying the cache; compared with naive LFU, it is more sensitive to hotspot changes, but is also less likely than pure LRU to be polluted by one-off traffic.

seconds rule represents another line of thinking, it does not directly look at “most recent access” or “cumulative access count”, but instead focuses on the reuse interval between two accesses to the same object. If an object is repeatedly accessed within a very short time, then it is a typical “short-interval reuse” object and is more worth promoting to DRAM; if the interval is long each time, even if the total access count is not low, it may not be worth occupying precious memory for a long time. Classic seconds rule sets a threshold to distinguish short-interval reuse from long-interval reuse, and only gives sufficiently high caching priority to the former.

In this project, the “object” in the above policies is an IVF cluster / list. We use LRU、naive LFU、window LFU and seconds rule to decide which lists should reside in DRAM and which should remain on SSD, so that under different access distributions and hotspot change speeds, we can systematically compare the tradeoffs among latency (especially p95) , I/O amplification, and migration overhead for different policies.

3.3 SSD based and Hybrid ANN Systems

In large-scale vector retrieval, if we simply place IVF lists entirely on SSD and, for each query, read all corresponding lists back according to nprobe, the system will quickly be bottlenecked by I/O throughput and random-access latency. Therefore many works are not simply “moving the parts that do not fit in memory onto SSD”, but instead start from the index structure itself, designing more “SSD-friendly” ANN solutions so that the amount of external storage access truly triggered per query is as small as possible.

DiskANN is a representative system-level work, aiming to host tens of billions of vectors on SSD on a single machine, and achieve high recall under strict latency constraints. It adopts a hierarchical graph structure: the upper-layer graph is relatively sparse and is used to quickly locate candidate regions; the lower-layer graph is denser and is used for fine-grained search within a local region. During querying, it does not linearly pull up an entire list as IVF does, but instead walks a bounded-length path along graph adjacency edges, accessing only a small number of nodes related to the current search boundary (that is, a small number of vector storage locations) , thereby keeping the number of random I/Os at a very small constant level. DiskANN also combines DRAM caching, caching the most frequently accessed nodes or graph layers in memory, further reducing SSD pressure.

SPANN represents another design route centered on “partitioning + bounded I/O”. It partitions the vector space into many partitions, and then builds a local index structure within each partition. During querying, it first uses a lightweight module to select a small number of candidate partitions, and then performs a heavier search inside these partitions. The whole process is designed so that “the number of accessed partitions has a hard upper bound”, which corresponds on SSD to “how many data blocks/files each query can read at most”, thus guaranteeing controllable overall latency. Compared with pure graph structures, SPANN emphasizes modeling “partition boundaries” and “cross-partition costs”, which is important in SSD scenarios because crossing many partitions implies a large amount of random I/O.

Product Quantization (PQ) and its variants are a more fundamental but very critical family of techniques. PQ uses codebooks over subspaces to approximate the original vectors, often reducing storage and bandwidth from float32 down to tens or even single-digit bits per dimension. A typical approach is to store only PQ codes and a small amount of auxiliary information on SSD, during querying first use PQ codes to compute approximate distances, quickly filter a relatively small set of candidates from a massive number of vectors, and then only for these candidates read the original vectors from DRAM or SSD for exact computation. PQ is often combined with IVF into IVF-PQ: IVF handles coarse partitioning and shrinking the candidate range, PQ handles compression and fast scoring, and this combination is already the default baseline for many industrial ANN systems.

learned indexes treat the index itself as an approximate function that can be learned: use a model to predict “where a key (here it can be some vector encoding or hash) should fall in position or partition”, thereby reducing lookup cost. In vector retrieval scenarios, the idea of learned indexes usually does not completely replace IVF or graph structures, but serves as an auxiliary module to improve partition selection, reduce the number of misselected partitions, or help route queries better to machines / shards that are “likely to hit”. Their common goal is still to reduce unnecessary accesses, rather than simply improving CPU computation efficiency.

The shared conclusion of these system-level and algorithm-level works is: in SSD scenarios, what truly determines latency is not “whether cache replacement is done”, but “how many bytes must be read from SSD per query, and how many random I/Os are performed”. Caching policies are of course important, but if the underlying structure requires pulling up lists that are themselves very large or very many, then merely doing simple list-level caching between DRAM and SSD makes it hard to push p95 below strict SLA requirements.

This is also why this project deliberately chooses, within a fixed IVF framework, to change only the “residency policy of lists in DRAM/SSD”, and to isolate and observe “the limit of what caching alone can achieve”, providing a clear baseline for later layering on PQ, graph structures, or more complex routing.

3.4 Positioning of this Work

The positioning of this project is closer to a course project and a system design exercise. It does not try to directly surpass complete industrial systems like DiskANN and SPANN in absolute performance, but instead intentionally “drops one dimension”: it fixes the search algorithm as IVF, sets aside more complex engineering optimizations (such as cross-machine distribution, asynchronous I/O pipelines, operating-system tuning, and so on) , and focuses on the more controllable and more interpretable question of “under a DRAM + SSD tiered model, what can list-level caching policies actually look like”. In other words, we treat IVF as a stable backbone, and do systematic exploration only along the single dimension of “which tier the lists live on”, so as to avoid being drowned by too many engineering details. The value of this positioning is mainly reflected in the following aspects

Decompose a complex system into three parts: ANN search, I/O model, and caching policy, so each part can be explained and verified independently
We intentionally keep the ANN part as “clean” as possible: using the standard IVF search workflow, explicitly exposing parameters such as nprobe and nlist, and using all DRAM as a reference lower bound of “pure compute latency + highest recall”; the I/O part uses a simple but transparent page-level latency model, caring only about “how many pages were read, and how expensive each page is”; the caching policy part is only responsible for deciding which IVF lists go into DRAM and which remain on SSD. This decomposition allows readers to clearly distinguish: whether a phenomenon is due to the ANN algorithm itself (for example nprobe is too large) , page latency is too high, or the caching policy failed to capture the true hotspots, instead of blurring all problems into “SSD is slow” or “the index is bad”.

Through multi-workload comparisons, show that policy pros and cons are not fixed, but are determined by the access distribution and hotspot change speed
We design multiple workloads such as default, fast-changing hotspots, slow-changing hotspots, uniform, and Zipf, and under each workload evaluate all policies using the same set of metrics (p95 latency, I/O amplification, migration overhead, SLA reachability, and so on) . Such comparisons clearly show that: LRU / seconds rule have more advantages when hotspots move rapidly, window LFU is more cost-effective when hotspots are relatively stable but still evolve, while in nearly uniform-access scenarios, any “smart” policy will not be much better than simple baselines. Through these results, readers can directly see that “no policy is always optimal”, and policies must be selected according to concrete access patterns, rather than blindly applying some “industry common solution”.

Through explicit migration statistics, compare policies not only by latency and recall, but also by implementation cost, avoiding focusing only on superficial gains
In most discussions about caching policies, migration cost is often ignored: as long as the hit rate goes up and latency goes down, it is considered a “good policy”. This project explicitly tracks migrated bytes, migration

count, and migration time for each policy in the experimental framework, and normalizes these quantities to the scale of “average per query”, thereby exposing extreme behaviors such as “p95 looks good but several GB were migrated”. This design reminds us: no policy is a free lunch, if it is to be put into a real system, one must consider query latency, SSD bandwidth consumption, and the impact of background migration on system stability at the same time. By quantifying these costs, system designers can more rationally evaluate “whether a small latency gain is worth so much migration”, rather than looking only at the superficial p95 number.

4. System Model & Problem Formulation

4.1 ANN Engine and IVF Lists

We use IVF as the main structure of ANN, and we can understand the whole query process as a two-stage pipeline with two layers of “coarse-to-fine”.

The first layer is the coarse search stage (coarse search) . The system maintains a set of cluster centroids (that is, the coarse quantizer) , with a count of $nlist$. Given a query vector q, we first compute the distances from q to all cluster centroids, then sort them from near to far by distance, and select the nearest $nprobe$ clusters. The output of this step is not the final result, but a set of routing decisions about “which regions should be focused on”, which can be formalized as

$$\mathrm{Top}(q, nprobe) \subseteq 1, 2, \ldots, nlist,$$

where $\mathrm{Top}(q, nprobe)$ is the set of the top $nprobe$ cluster IDs for q.

The second layer is the fine search stage (list scan) . Each cluster c has an inverted list L_c, which stores the IDs of database vectors assigned to this cluster and the corresponding vectors or encodings. During querying, we only scan the small subset of clusters in $\mathrm{Top}(q, nprobe)$, and score entries in their inverted lists one by one. Denote the inverted list of the c-th cluster as L_c, then the candidate set of a query is

$$C(q) = \bigcup_{c \in \mathrm{Top}(q, nprobe)} L_c.$$

Within these candidates, we then compute the (exact or approximate) distance between the query vector q and each candidate vector, and select the k vectors with the smallest distances from $C(q)$ as the final returned results. Intuitively, coarse search decides “which buckets to scan”, and list scan decides “which specific items to pick from the buckets”.

In this project, this IVF structure has an additional role: it naturally provides the granularity for caching and migration. Each inverted list L_c is treated as an independent “object”, either entirely placed in DRAM or entirely placed on SSD. For a query q, if a hit cluster c happens to reside in DRAM, then scanning L_c requires no SSD I/O; if L_c is on SSD, we need to read its corresponding data pages from SSD into memory first. Therefore, for

a query, “how many SSD accesses a query needs to trigger” is almost equivalent to “among the clusters in $\mathrm{Top}(q, nprobe)$, how many clusters have lists that are not in DRAM”.

From the parameter perspective, nprobe is a core knob controlling the performance–accuracy tradeoff:

- The larger nprobe is, the more clusters $\mathrm{Top}(q, nprobe)$ contains, the larger the candidate set $C(q)$ becomes, the more likely it is to cover the true nearest neighbors, and recall will improve;
- But at the same time, more lists need to be scanned, CPU computation increases, and the amount of DRAM / SSD data accessed is also linearly amplified, so I/O pressure and latency rise accordingly.

In this project, we set nprobe to $1, 2, 4, 8, 16, 32$ for systematic comparison: from the all DRAM results, we can clearly see that as nprobe increases from 1 to 16, recall increases from a medium level and gradually approaches 1, but the average latency and p95 latency also rise significantly, which is typical behavior of IVF in engineering practice. And once SSD is introduced into the model, nprobe not only affects CPU computation time, but more directly determines “how many lists a query touches, and how many SSD reads it triggers”, thereby scaling up into overall tail latency through SSD page counts and I/O amplification.

4.2 Tiered DRAM + SSD Model

We explicitly model two storage tiers:

- DRAM: access latency is very small, and can be approximated as only computation time plus a tiny amount of memory-access overhead
- SSD: capacity is large but random access is slow, and access cost is given by the page-level read model

In this model, the basic unit of caching and migration is an IVF list, that is, we decide at cluster granularity whether a list currently resides in DRAM or SSD. For each cluster c, we maintain a residency flag $tier(c)$:

- $tier(c) = DRAM$: the list of cluster c resides permanently in DRAM, scanning this list produces no SSD I/O
- $tier(c) = SSD$: the list of cluster c is stored on SSD, scanning this list requires first reading the corresponding data pages from SSD into memory

The DRAM capacity constraint is represented by a dimensionless $dram\ fraction$. In the experiments, $dram\ fraction$ takes 0.05 and 0.2, which can be understood as:

- $dram\ fraction = 0.05$: DRAM can hold at most about 5 of all lists
- $dram\ fraction = 0.2$: DRAM can hold at most about 20 of all lists

Here we constrain capacity by “number of lists”, and the actual occupied bytes will vary with the length distribution of lists, which is also one reason why migration overhead differs greatly across policies later.

To characterize SSD access cost, we adopt a page-level I/O model. All accesses that fall on SSD are decomposed into random reads of fixed-size data pages. By fitting experimental results, we can infer the average cost per page read:

$$t_{\text{page}} = 20 + \frac{10^6}{\text{max\_iops}}$$

where max_iops is the theoretical upper limit of SSD random read capability. In the experiments we choose two levels:

- $\text{max\_iops} = 10^6$, $t_{\text{page}} \approx 21\ \mu s$
- $\text{max\_iops} = 5 \times 10^6$, $t_{\text{page}} \approx 20.2\ \mu s$

This formula can be understood as: around $20\ \mu s$ is a "base access overhead", and $\frac{10^6}{\text{max\_iops}}$ approximately characterizes the queuing or scheduling overhead caused by IOPS capability differences; when max_iops increases from 10^6 to 5×10^6, this term drops from $1\ \mu s$ to $0.2\ \mu s$, so the per-page benefit from higher IOPS is actually quite limited.

For a given query q, let the total number of SSD pages actually accessed during the search be $N_{\text{pages}}(q)$, then the I/O latency of that query is approximated as:

$$L_{\text{io}}(q) = N_{\text{pages}}(q) \cdot t_{\text{page}}$$

Here, $N_{\text{pages}}(q)$ comes from the combined effect of three factors: the set of clusters hit by the query, whether these clusters are currently in DRAM, and the actual sizes of the corresponding lists. In the implementation, we first determine which lists need to be scanned for this query based on $nprobe$ and the index structure, then for the lists that reside on SSD we count pages aligned to 4KB pages, and sum them to obtain $N_{\text{pages}}(q)$. The fields `avg_ssd_pages_mean` and `avg_ssd_bytes_mean` in the result files are the averages of $N_{\text{pages}}(q)$ and its corresponding bytes over all queries.

To separate ANN computation itself from I/O, we decompose total query latency into three parts:

$$L(q) = L_{\text{ann}}(q) + L_{\text{io}}(q) + L_{\text{fixed}}$$

where:

- $L_{\text{ann}}(q)$: the computation time of IVF search itself, including coarse search (distance computation to centroids) and list scan (distance computation to candidate vectors) , recorded as `avg_ann_us` in the results
- $L_{\text{io}}(q)$: the SSD page-level I/O time defined above, which is the main bottleneck we focus on
- L_{fixed}: a small fixed overhead independent of the specific policy (scheduling, framework overhead, etc) . From the difference between `avg_latency_us` and `avg_ann_us` under all DRAM, we can see that L_{fixed} is about $0.2\ \mu s$, which is negligible compared with millisecond-level SSD I/O

The intuitive meaning of this model is: once a query falls onto SSD, overall latency is almost entirely determined by "how many pages were read". Under our configuration, even if we increase max_iops from

10^6 to 5×10^6, the per-page cost only changes from $21\ \mu s$ to $20.2\ \mu s$, which is far less effective than "halving the number of pages that need to be read". Therefore, for different policies and parameter combinations, comparing $p95$ latency is almost equivalent to comparing their average and tail SSD page counts, which is also why in subsequent analysis we heavily use I/O amplification and `avg_ssd_pages_mean` as intermediate indicators.

4.3 Objective and Metrics

Our optimization goal is not a single metric, but a multi-objective tradeoff problem:

- Primary goals: under given hardware and DRAM constraints, achieve as much as possible high recall, low average latency, and low tail latency (especially p95)
- Secondary goals: while ensuring the primary metrics, reduce I/O amplification as much as possible, and control the additional costs introduced by cache migration

Therefore we do not compress everything into a single score, but explicitly analyze several core metrics. Below we explain the key metrics that appear in the result outputs one by one.

4.3.1 Recall at k

recall at k is denoted as recall@k. For a single query q, it is defined as:

$$\text{recall@}k(q) = \frac{\#\{\text{vectors that are in the ANN returned top } k \text{ and also belong to the true top } k\}}{k}$$

Then taking the average over all queries gives the overall recall@k in the report. Its range is 0 to 1, and a higher value means "more of what should be retrieved is indeed retrieved".

In the experiment, the true top k is an "approximate ground truth" obtained by doing exact search over a smaller candidate set or by offline precomputation, the result files do not directly print k, but from the values of I/O amplification we can infer $k \approx 10$ (each vector is 128-dimensional, float32, so $10 \times 128 \times 4 = 5120$ bytes) , which also matches common ANN evaluation practice

We use recall rather than precision because in ANN scenarios the top-k returned results are fixed to k items, precision and recall are highly related in form, and recall more directly reflects "whether the truly nearest neighbors were missed".

4.3.2 Latency metrics

Latency-related metrics mainly include three categories:

- `avg_latency_us_mean` : average query latency, in microseconds (μs)
- `p50_latency_us_mean` 、 `p95_latency_us_mean` 、 `p99_latency_us_mean` : latency at different percentiles
- `avg_ann_us_mean` : average time of the ANN computation part (coarse search + list scan)

The average latency reflects overall throughput, p50 can be roughly understood as the latency of a “typical” query, p95、p99 characterize tail latency, which is the metric system design cares more about

The definition of p95 is: sort the latencies of all queries from small to large, and take the value at the 95 position, that is, 95 of queries have latency no greater than this value. Online services commonly use p95 (or even p99) as an SLA constraint, because real users often encounter these “slightly slower requests”, rather than only seeing the average.

`avg_ann_us_mean` only counts the computation overhead that still exists even under all DRAM. In our model, total latency can be roughly decomposed as:

$$L(q) = L_{\text{ann}}(q) + L_{\text{io}}(q) + L_{\text{fixed}}$$

where L_{ann} corresponds to `avg_ann_us_mean` , L_{io} comes from SSD page-level I/O, and L_{fixed} is a tiny constant of about $0.2\ \mu s$. By comparing all DRAM and SSD-involved policies, we can directly see that I/O pulls latency from microseconds up to milliseconds.

4.3.3 SSD read volume and pages

SSD read volume related metrics mainly include:

- `avg_ssd_pages_mean` : average number of pages read from SSD per query
- `avg_ssd_bytes_mean` : average bytes read from SSD per query

The relationship between the two depends on page size. In the experiment we use 4KB pages, so we have:

$$\text{bytes} = \text{pages} \cdot 4096$$

`avg_ssd_pages_mean` is a very key intermediate quantity, because under our page-level latency model, I/O latency is almost a linear function of page count:

$$L_{\text{io}}(q) = N_{\text{pages}}(q) \cdot t_{\text{page}}$$

That is, as long as a policy causes queries to trigger more SSD page reads, p95 will almost linearly worsen; conversely, as long as it can reduce the average and tail page counts, it can significantly improve tail latency. `avg_ssd_bytes_mean` helps us intuitively feel from a byte perspective “how much data each query is scanning on SSD”.

4.3.4 I/O amplification

`avg_io_amplification_mean` denotes I/O amplification (IOAmp) , used to measure how much extra “useless data that was scanned along the way” the system actually reads from SSD in order to obtain the small top k set of truly useful data.

Formally it can be written as:

$$\text{IOAmp} = \frac{\text{SSD bytes read per query}}{\text{useful bytes per query}}$$

where the denominator $\text{useful bytes per query}$ can be approximated as:

$$\text{useful bytes per query} \approx k \times d \times \text{sizeof(float)}$$

In this project's configuration, $d \approx 128$、$\text{sizeof(float)} = 4$, and $k \approx 10$, so the useful data is about 5120 bytes.

Take the default workload with all SSD and $nprobe = 1$ as an example, the results give:

- `avg_ssd_bytes_mean ≈ 179703.808` bytes
- `avg_io_amplification_mean ≈ 35.0984`

Thus we can infer:

$$\text{useful bytes} = \frac{179703.808}{35.0984} \approx 5120$$

This matches $10 \times 128 \times 4$ exactly. Therefore IOAmp can be explained as obtain about 5KB of truly useful results, each query on average must read about 170KB of data from SSD, amplifying by about 35 times

The larger IOAmp is, the more useless data is "incidentally scanned" on SSD, and in scenarios where random I/O cost is very high, this almost directly translates into higher latency, especially p95 and p99. Compared with looking only at `avg_ssd_bytes_mean` , IOAmp normalizes results to the dimension of "how much physical I/O must be paid per byte of useful data", making it easier to compare across datasets and across k.

4.3.5 Migration metrics

Migration-related fields characterize how much data different policies actually moved during execution in order to chase hotspots and adjust the residency set, and how much extra time was spent on this. They mainly include:

- `total_migration_bytes_mean` : total migrated bytes over the entire experiment
- `migration_bytes_per_query_mean` : total migrated bytes divided by the number of queries, normalized to "how many extra bytes are moved on average per query for maintaining the policy"
- `total_migration_time_us_mean` : total time spent on migration operations (in microseconds)
- `migration_time_us_per_query_mean` : average migration time per query
- `avg_migrated_clusters_per_rebalance_mean` : average number of clusters (lists) migrated per rebalance

It should be emphasized that migration itself does not directly show up in the latency of a single query: in many real systems, migration can be done slowly in the background, or performed in batches during low load, so looking only at query p95 can easily ignore this maintenance cost. But in engineering practice, migration will:

- consume SSD bandwidth, competing with online queries for I/O capability
- consume CPU for data copying and metadata updates

- trigger extra cache writes and invalidations

Therefore we explicitly track migrated bytes and migration time in the results, with the purpose of presenting the question of “whether a policy is worth it” in a complete way: even if a policy can reduce p95 by a few hundred microseconds on the query dimension, if the cost is moving several GB of data and continuously occupying SSD bandwidth, it may not be a better choice in a real system. By comparing `migration_bytes_per_query_mean` with latency improvements, we can more clearly see the differences among policies in “benefit / cost”.

5. Policy Design

In this project, all policies are in fact revolving around the same core question: under a given upper bound of DRAM capacity, which clusters’ IVF lists should reside permanently in DRAM, and which should remain on SSD. The ANN algorithm itself (the IVF structure, distance computation method, nprobe values, etc) is completely identical across different policies, what changes is only the decision logic of “which tier the list resides in”, and how that decision affects recall, latency, I/O amplification, and migration cost.

In other words, we did not “invent a new ANN”, but under a fixed ANN engine, we study how different caching policies shape the performance of the same engine.

5.1 Baseline Policies

To provide a “coordinate system” for later comparisons, we first define several baselines. The two extremes all DRAM and all SSD provide an “ideal upper bound” and a “worst lower bound” respectively, while LRU, naive LFU, and window LFU represent several classic caching ideas.

5.1.1 all DRAM

all DRAM assumes that all IVF lists fully reside in DRAM. For each query, after nprobe selects several clusters, the list scans of these clusters all happen in memory, without any SSD reads. In this case, the system’s query latency is almost entirely determined by the ANN search itself, namely the computation time of the coarse search stage (coarse search) and the fine search stage (list scan) , plus a very small fixed overhead. In the experimental results, we can see that under this configuration avg_latency_us and avg_ann_us are very close, and their difference is only on the order of about 0.2 microseconds.

Given nprobe, all DRAM also provides the highest achievable recall. Because there is no I/O constraint and no missed scans caused by cache mistakes, ANN only “loses to approximation”, and will not additionally “lose to I/O”. Therefore all DRAM can be regarded as “under the same IVF configuration + the same nprobe, the truly reachable upper bound of performance and accuracy”, and all DRAM+SSD based policies later are moving toward this point, but cannot surpass it.

5.1.2 all SSD

all SSD is the other extreme: all IVF lists are stored on SSD, and DRAM caches no list. For each query, as long as nprobe selects a cluster, the corresponding list must be read from SSD and then participates in computation. Here there is no notion of hit or miss, and there is no migration decision either, DRAM is only viewed as a “mandatory buffer during computation”.

Under this configuration, we can clearly see several things. First, under the current data scale, list sizes, and nprobe settings, how many pages and how many bytes a query needs to read from SSD on average, which is directly reflected in avg_ssd_pages and avg_ssd_bytes. Second, to obtain only that small amount of “useful data” of top-k, how much extra data the system actually reads from SSD, which is reflected by I/O amplification (avg_io_amplification) . Third, once SSD I/O is involved, to what magnitude query latency is pulled, which can be directly seen from millisecond-level p95.

In our experiments, the p95 of all DRAM is usually a few microseconds to a dozen microseconds, while the p95 of all SSD is a few milliseconds to over ten milliseconds, differing by two orders of magnitude. This also indicates from another angle: as long as part of the lists are placed on SSD, SSD reads are the main determinant of tail latency. All subsequent caching policies can be understood as searching for a tradeoff point between the two endpoints all DRAM and all SSD: use limited DRAM to reduce SSD reads as much as possible, while keeping migration cost under control.

5.1.3 LRU

In this project, LRU (Least Recently Used) maintains recency information at the “cluster” granularity. We can view the set of clusters currently residing in DRAM as an ordered linked list or queue: the tail represents “clusters accessed most recently”, and the head represents “clusters not accessed for the longest time”.

Each time a query hits a cluster c, if c is already in DRAM, we move it to the tail, indicating it is the most recently used object; if c is not in DRAM, then we decide according to the policy whether to migrate c’s list from SSD into DRAM. If migration is needed and DRAM is already full, we evict one cluster from the head that has been unused for the longest time, and treat its list as residing only on SSD. This “move to tail” and “evict from head” forms the typical LRU behavior.

The biggest advantage of this policy is that it is very sensitive to hotspot changes. If some clusters suddenly become very hot in a recent period, they will immediately move to the tail and stably remain in DRAM; as long as a cluster has not been accessed recently, even if it was historically very hot it will gradually slide to the head and eventually be evicted. The disadvantages are also very typical: once there is a one-off “full-database sweep”, a large number of clusters are accessed once within a short time, and the true hot clusters may be squeezed out of DRAM by these one-time accesses, this is the so-called cache pollution. In addition, under workloads where hotspots switch frequently, LRU may be very “restless”: a new hotspot just enters, an old hotspot that was just evicted may become hot again soon, so it has to be migrated back, leading to very considerable migration counts and migrated bytes.

From the experimental results, we can see that under workloads where hotspots change very fast, LRU often achieves relatively good reductions in I/O amplification and p95 latency, but the corresponding migration_bytes is often the largest among all policies, reflecting the characteristic of “diligently chasing hotspots, but also moving house very frequently”..

5.1.4 naive LFU

naive LFU (Least Frequently Used) uses the dimension of “access count” to judge whether a cluster is worth residing in DRAM. For each cluster c, we maintain an access counter $\mathrm{cnt}(c)$. Each time a query hits cluster c, we increment $\mathrm{cnt}(c)$ by one; when we need to free space in DRAM, we preferentially evict clusters with the smallest counters, leaving their lists on SSD.

Intuitively, clusters with higher access counts are more likely to be long-term hotspots, and should be prioritized to stay in DRAM; clusters that appear only once or twice are likely just local noise or tail data touched by some random scan, and are not worth occupying memory long-term. This idea is often very effective in systems where hotspot patterns are relatively stable.

But naive LFU has a fatal engineering flaw: the counts “accumulate without bound”. Clusters that were extremely popular early on but have already “cooled down” later may still occupy DRAM because their historical access counts are too high; while newly emerged hotspot clusters, even if they are very active recently, still find it hard in the short term to catch up to the old hotspots’ counts. As a result, the system behaves as “vengeful but not forgetful”: historical hotspots consume most of DRAM capacity, and new hotspots are repeatedly read from SSD but still cannot squeeze in for a long time.

We can also see this in our experiments: in scenarios where hotspots move very slowly, or where the Zipf-style distribution is very extreme, naive LFU performs reasonably well, because the small portion of truly hot clusters remain hot over a long period, and the counts do accurately reflect their importance. But under workloads where hotspots change quickly, naive LFU’s ability to chase hotspots is far worse than LRU or seconds rule, the reduction in I/O amplification is very limited, and p95 cannot be driven down.

5.1.5 window LFU

window LFU can be seen as a version of naive LFU “with a forgetting mechanism”. The core idea is: only count frequency within the most recent “time window”, and ignore all history outside the window. In this way, the system will not be held hostage by “hotspots from a long time ago”, but instead pays more attention to clusters that are truly active in the current period.

There can be different implementation choices. One is a strict sliding window: for example, only count how many times each cluster is accessed in the most recent W queries, and as the window slides forward, old statistics are discarded. Another implementation is more engineering-friendly, using exponential decay to approximate a sliding window: for each cluster c maintain a score $\mathrm{score}(c)$, and on each access perform

$$\mathrm{score}(c) \leftarrow \alpha \cdot \mathrm{score}(c) + 1,$$

where $0 < \alpha < 1$.Thus, if a cluster is not accessed for a long time, its $\mathrm{score}(c)$ will automatically decay to a very low level, and even if it was once very hot, it will gradually lose priority.

In the experimental results of this project, window LFU is almost always more stable and more engineering-friendly than naive LFU. On the one hand, it can better adapt to hotspot changes, reducing the influence of "historical baggage", and under multiple workloads it can reduce I/O amplification and p95 a bit further than naive LFU; on the other hand, its migration overhead usually stays at a medium level, and will not show hundreds of MB or several GB of migration volume like LRU or seconds rule. Therefore, if a real system can only choose one frequency-based policy as the default, window LFU is usually more suitable than naive LFU.

5.2 Seconds Rule Policy

The perspective of seconds rule is clearly different from the previous policies: it does not focus on "whether it was accessed recently" or "how many times it was accessed", but on the "reuse interval"——that is, the time difference or query-step difference between two accesses.

For each cluster c, we record the timestamp or query index of its last access $t_{\mathrm{last}}(c)$. The current query occurs at "time" t (in the actual implementation, t can be approximated by "which query number") , then the reuse interval of this access is defined as

$$\Delta t(c) = t - t_{\mathrm{last}}(c).$$

seconds rule uses a threshold θ to distinguish "short reuse" and "long reuse". If $\Delta t(c) \le \theta$, it indicates this cluster is repeatedly accessed within a relatively short time, so it closely resembles a part of the current hotspot and is worth being placed in DRAM; if $\Delta t(c) > \theta$, it indicates that a long time has passed between two accesses, and even if this cluster has appeared multiple times before, it is more like cold data and can remain on SSD. Through such a threshold, the policy distinguishes "short-interval reuse" (short-term locality) from "long-interval reuse" (historical residue) .

In practical implementation, a simplified workflow can be as follows: during initialization, we can first select a batch of clusters to place in DRAM by some simple method, or simply start from empty and learn gradually. Afterwards, each time a query hits cluster c, we first compute the current $\Delta t(c)$, if the reuse interval is less than or equal to the threshold, we boost c's priority; if c is not in DRAM, we try to migrate it into DRAM. If DRAM is full at this time, we evict one cluster with the "lowest value" according to some priority rule. Meanwhile, we update $t_{\mathrm{last}}(c)$ to record the new time for the next access.

Behaviorally, seconds rule lies between LRU and LFU. Compared with LFU, it is more sensitive to "short-term repeated access", and is not overly constrained by distant history; compared with LRU, it does not only look at "whether it came recently", but precisely distinguishes "came many times recently" from "came once after a long interval". In scenarios where hotspots change very fast, this sensitivity to short reuse often helps the policy quickly catch up with current hotspots, switching most active clusters to DRAM, thereby significantly reducing I/O amplification and p95.

However, the cost is also clear: seconds rule is often more “aggressive” than window LFU and naive LFU, and will migrate frequently when cluster hotspots switch back and forth, once the list is large, such migration imposes very real pressure on SSD bandwidth and CPU resources. In the experimental results, we can see that under the hotspot fast-fast workload, seconds rule’s `migration_bytes` is usually on the order of tens of MB or even hundreds of MB, far higher than frequency-based policies; under some parameter combinations, although it further reduces I/O amplification, the migration cost has already become so high that it is difficult to directly transplant into a real system.

5.3 Complexity and Implementation Notes

From an engineering implementation perspective, this project intentionally designs the cache granularity as “cluster-level IVF lists”, rather than finer-grained “blocks within a list” or “per-vector”. A direct benefit of this is that the metadata scale is controllable: assuming the number of clusters is $nlist$, regardless of which policy is used, we only need to maintain one piece of state for each of these $nlist$ clusters.

For LRU, we need to maintain for each cluster a position in a doubly linked list or an index in a queue, the overall metadata size is $O(nlist)$, and each access to a cluster only requires $O(1)$ linked-list operations. For LFU, we need to maintain access counts (and possibly a heap structure or a layered counting structure) , the metadata is also $O(nlist)$; window LFU additionally maintains window information or a decay factor on top of that, and the complexity remains $O(nlist)$. seconds rule needs to store for each cluster the last access time $t_{\text{last}}(c)$ and a priority score, used to evict clusters with “very long reuse intervals” when DRAM is full, and it is also maintained at the $O(nlist)$ scale.

What can easily “run out of control” is in fact migration cost. List sizes across different clusters often differ greatly: some clusters’ lists contain only a few hundred vectors, while some clusters may gather over a thousand or even more vectors. Migrating a small list can be negligible, while migrating a huge list may consume a large amount SSD bandwidth and memory copy time in a single operation. If a policy happens to be particularly sensitive to such “giant lists”, and frequently pulls them from SSD into DRAM and then throws them out across different phases, `migration_bytes` and `migration_time` will soar.

The huge differences in migrated bytes across different workloads in the experimental results are exactly the combined effect of “list size distribution” and “policy migration behavior”. This also reminds us that when discussing the pros and cons of policies we cannot only stare at “hit rate” or “whether p95 becomes smaller”, but must jointly examine three dimensions: whether I/O amplification is significantly reduced, whether migration cost is acceptable, and whether the entire scheme truly falls within the feasible region under the target SLA. In real engineering systems, it is very likely that migration rate or budget limits still need to be set, for example “at most how many MB can be migrated per second”, or compute cost-effectiveness indicators such as “SSD bytes saved per byte migrated” for each migration action, so as to avoid policies like seconds rule and LRU under extreme workloads “chasing hotspots until they migrate themselves to collapse”.

6. Methodology

6.1 Datasets

This project uses SIFT vector data as the experimental object. Based on the back-inference from I/O amplification results, we can determine that the vector dimension is 128, and `float32` storage is used, which is consistent with the classic SIFT configuration. We denote the database vectors as

$$X \in \mathbb{R}^{N \times 128},$$

and call it the base set; denote the query vectors as

$$Q \in \mathbb{R}^{M \times 128},$$

and call it the query set. All index construction, search, and statistics in the experiments are performed on this pair (X, Q), the ANN engine only changes the index structure and tiering policy, and does not change the underlying vectors themselves, thus ensuring that different policies are compared under exactly the same data conditions.

6.2 Workloads

To cover different access patterns, we construct five workloads, each corresponding to an independent run directory. The directory for the default workload is
`results/sr_sift_default_fast_20251215_135028` ,
the fast-changing hotspot workload uses
`results/sr_sift_exp_hotspot_fast_fast_20251215_135028` ,
the slow-changing hotspot workload uses
`results/sr_sift_exp_hotspot_slow_fast_20251215_135028` ,
the uniform-distribution workload uses
`results/sr_sift_exp_uniform_fast_20251215_135028` ,
and the Zipf distribution (parameter $s = 1.2$) workload uses
`results/sr_sift_exp_zipf_s1.2_fast_20251215_135028` .

The differences among these workloads are mainly reflected in the different “probability distributions of queries landing on each cluster”: the uniform workload approximates that each cluster has a similar access probability; the Zipf workload makes a small number of clusters bear most accesses, forming a long tail; the two hotspot workloads explicitly construct a “hot cluster set”, so that queries concentrate on this set for a period of time, and as time progresses continuously switch the hotspot set, where fast / slow controls the speed of hotspot movement. The default workload can be understood as a “mild” distribution between real traffic and an ideal model, used as a control group for the other workloads.

6.3 Parameter Sweep

Under each workload, we conduct a systematic parameter sweep over key system parameters, so as to observe performance tradeoffs under different combinations. DRAM capacity ratio (denoted as dram fraction) takes

$$0.05, 0.1, 0.2$$

representing that DRAM can hold at most 5 or 20 of all IVF lists. The number of probed clusters $nprobe$ takes

$$1, 2, 4, 8, 16, 32$$

covering different intensities from “very few candidates, low I/O, high miss recall” to “sufficient candidates, obvious I/O amplification”. The SSD maximum IOPS is denoted as $\max\_iops$, the default workload tests both

$$10^6, \quad 5 \times 10^6,$$

while the other four workloads, to control experimental scale, only test

$$5 \times 10^6.$$

For each $(\text{workload}, \text{dram fraction}, nprobe, \max\_iops)$ combination, we uniformly run a set of policies, including `all_dram`、 `all_ssd`、 `lru`、 `naive_lfu`、 `window_lfu` and `seconds_rule`. All policies share the same query sequence, the same underlying IVF structure, and the same SSD latency model, thus ensuring that the comparison among different policies is a clean control of “only changing the policy, changing nothing else”.

6.4 Experimental Pipeline

From the directory structure under `results/`, we can infer that the experimental pipeline consists of several stable steps. First, for each run (that is, a specific workload and parameter combination) , the online search process continuously records raw statistics and writes them into `raw/ann_policy_raw.csv`, this level of data retains the finest-grained metrics, such as per-query latency, SSD pages, and migration behavior.

Second, we aggregate the raw data to generate `agg/ann_policy_agg.csv`. In this step, we summarize various means, standard deviations, and confidence-interval fields by “policy × configuration”; at the same time, based on the relationship between latency and recall we additionally export `agg/sla_reachable_recall.csv`, used to analyze the maximum recall each policy can achieve under a given SLA. In the current experiment `num_seeds = 1`, so the standard deviation and confidence-interval fields are 0, and these aggregated values can be understood as “the observation point under this one random seed”, if more seeds are added in the future, the same pipeline can directly produce more rigorous statistical results.

The third step automatically generates a series of visualization charts based on the aggregated `agg/` data, and saves them in the `figures/` directory. They mainly include: curves of p95 latency varying with DRAM ratio and IOPS, the relationship between I/O amplification and DRAM ratio, migration overhead varying with policy and configuration, and recall–latency frontier curves, and so on. These charts are the main basis for the subsequent result analysis writing (Section 7) .

Finally, the script generates a batch of tables and Markdown snippets under the `report_assets/` directory, such as a key-point summary table and an SLA reachability table, to facilitate stitching into a more complete master report. In the current version of this work, we mainly "manually read `agg/` and the charts, then write the analysis", but the entire pipeline already supports one-click generation of report assets in a unified style from raw statistics, laying a foundation for future experiment expansion or reproduction of experimental results.

7. Results

This section presents results and analysis based on the experimental outputs (COMPACT summary + figures under each run directory) . We focus on four categories of metrics:

- **p95 latency (µs)**: tail query latency (the smaller the better) .
- **recall@k**: recall (the larger the better) .
- **avg_io_amplification (IOAmp)**: the degree of SSD I/O amplification triggered by each query (the smaller the better) . It is highly correlated with "how many SSD pages/how much list data each query needs to read".
- **total_migration_bytes**: the cumulative number of bytes of list data migrated between SSD↔DRAM by a policy during execution (the smaller, the more "economical in movement/more stable") .

Experimental variables include:

- `nprobe` $\in$ `[1, 32]` (number of IVF lists scanned; larger values give higher recall but typically larger I/O and latency)
- `dram_fraction` $\in$ `{0.05, 0.1, 0.2}`
- `max_iops` $\in$ `{1M, 5M}`
- policy: `all_dram, all_ssd, naive_lfu, window_lfu, seconds_rule, lru`

7.1 Overall observations

7.1.1 The essential role of nprobe: trading I/O for recall

In all workloads, increasing `nprobe` will drive:

- recall to increase monotonically (more lists are scanned, and the probability of hitting true neighbors becomes larger)
- p95 latency to increase (more lists scanned → more pages read → larger I/O amplification)

Therefore the system exhibits a very typical recall–latency frontier: a curve from low-latency/low-recall (small nprobe) to high-latency/high-recall (large nprobe) .

7.1.2 In SSD scenarios, tail latency is almost determined by “how many pages are read”

When lists need to be read from SSD, p95 latency quickly rises to the millisecond level, and shows a strongly positive relationship with IOAmp / avg_ssd_pages:

- IOAmp decreases → fewer SSD pages read per query → lower p95
- IOAmp increases (for example when nprobe becomes larger) → more pages → higher p95

7.1.3 The essence of “policy benefit” is a tradeoff between two things

When comparing different caching policies, the core is to look at two chains:

- Hit-benefit chain: can the policy place “lists that will be accessed” into DRAM → do SSD pages decrease → does p95 decrease
- Cost chain: does the policy need frequent migration in order to chase hotspots → does `total_migration_bytes` explode (which introduces engineering costs, write amplification, resource usage, etc)

7.2 Default workload

Run directory: `results/sr_sift_default_fast_20251215_135028`

7.2.1 Key numbers from COMPACT output (max_iops=5M, nprobe=1)

The table below directly picks the COMPACT numbers from the aggregated result files (units: p95 = μs; migration = bytes converted to MB for readability).

dram_fraction = 0.05

policy	p95 (μs)	recall	IOAmp	total_migration_bytes
all_dram	2.42	0.5293	0.00	0
all_ssd	1416.22	0.5293	35.10	0
naive_lfu	1274.82	0.5293	30.97	~8.60 MB
window_lfu	1274.82	0.5293	31.13	~24.42 MB
seconds_rule	1335.42	0.5293	31.94	~78.61 MB
lru	1335.42	0.5293	32.60	~104.57 MB

**Observations and explanations: **

- `all_dram` puts all data into DRAM, so p95 is only ~2.4 μs (pure compute/memory-access path) .
- `all_ssd` has IOAmp ~35, meaning each query triggers a considerable number of SSD page reads, so p95 jumps to ~1.4 ms.
- `naive_lfu/window_lfu` significantly reduce IOAmp (~31) , and p95 drops to ~1.27 ms: this shows they can keep part of the “commonly used lists” in DRAM, thus reducing SSD pages.
- `seconds_rule/lru` do not show clearly better IOAmp and p95 than the LFU family, but have larger migration volume: this indicates that under this workload, the extra movement spent on chasing hotspots does not translate into proportional hit benefits.
- After DRAM increases from 0.05 → 0.2, LFU-family IOAmp drops from ~31 to ~21.6, and p95 from ~1.27 ms to ~1.09 ms.
- Increasing DRAM indeed significantly reduces SSD page reads, but still cannot bring tail latency back to the microsecond level (because a substantial portion of list accesses still fall on SSD) .
- Under DRAM=0.2, `lru/seconds_rule` see clearly higher migration volume (hundreds of MB) but IOAmp is not better than LFU: this indicates they pay more movement to chase “temporal locality”, but the hit benefits are not proportional.

7.2.2 Behavior as nprobe increases from 1 to 32

In the default workload, increasing `nprobe` brings two synchronous changes:

- **recall increases**: points on the frontier plot move along the curve toward “higher recall”
- **p95 increases**: under the same policy, points move to the right (higher latency)

`nprobe` determines the number of lists scanned; scanning more lists linearly increases the total number of list entries that must be accessed, thereby pushing up SSD page count and IOAmp.
Thus in policies that involve SSD paths, the cost of `nprobe` is amplified more noticeably than in all_dram.

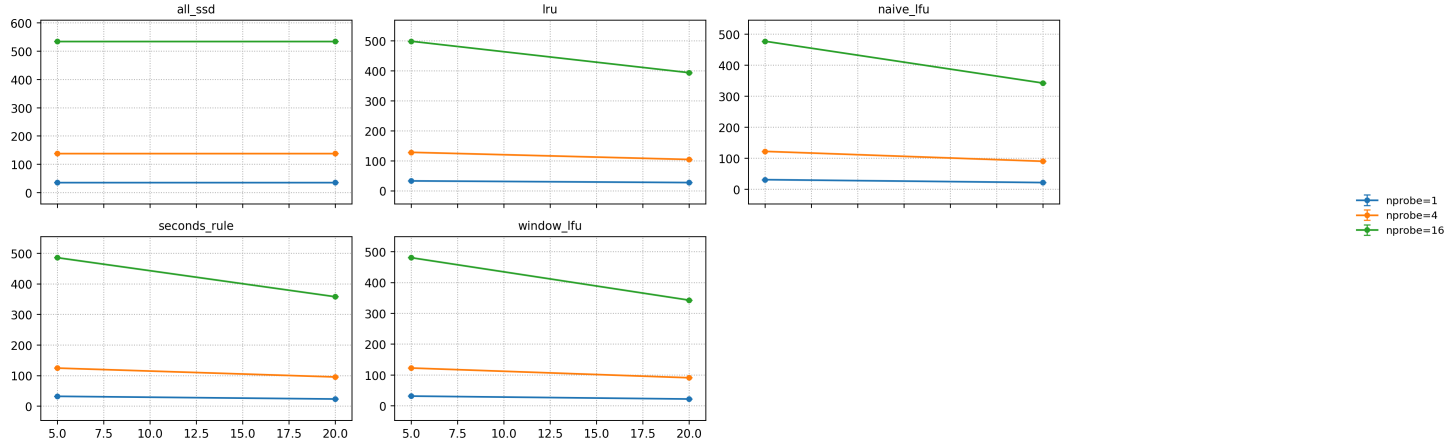
7.2.3 SLA reachable recall (200 μs / 500 μs)

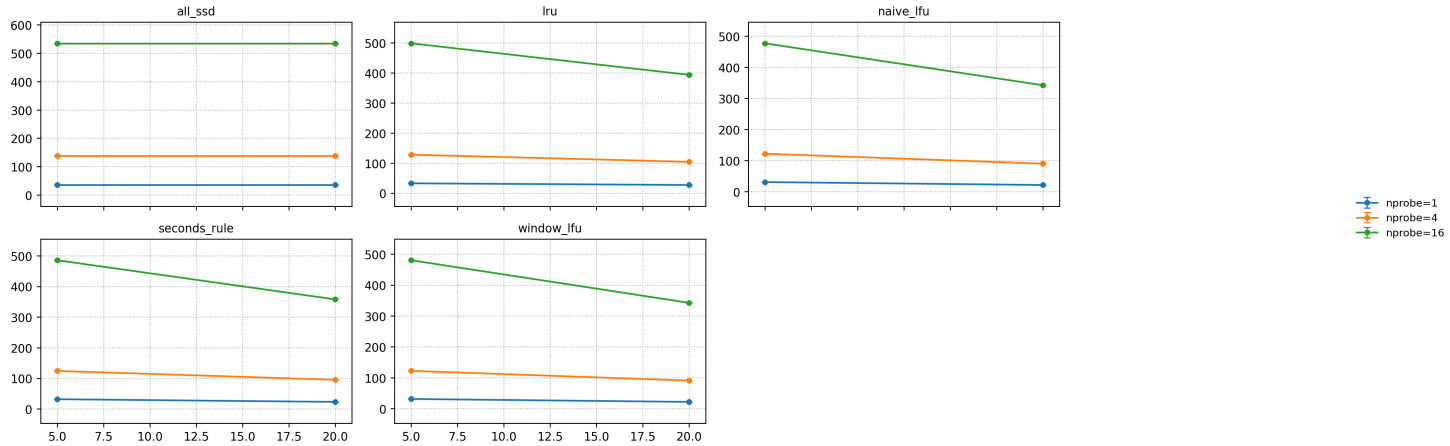
According to the SLA result plots/tables for this run: under the given SLA thresholds, only the pure DRAM path can choose a valid `best_nprobe` and achieve non-zero reachable recall; policies involving SSD are unreachable under these SLAs (appearing as `best_nprobe = -1` , reachable recall = 0) .

As long as the tail of queries still reads SSD pages, p95 will fall in the millisecond range, and it is very hard to squeeze it within a few hundred microseconds.

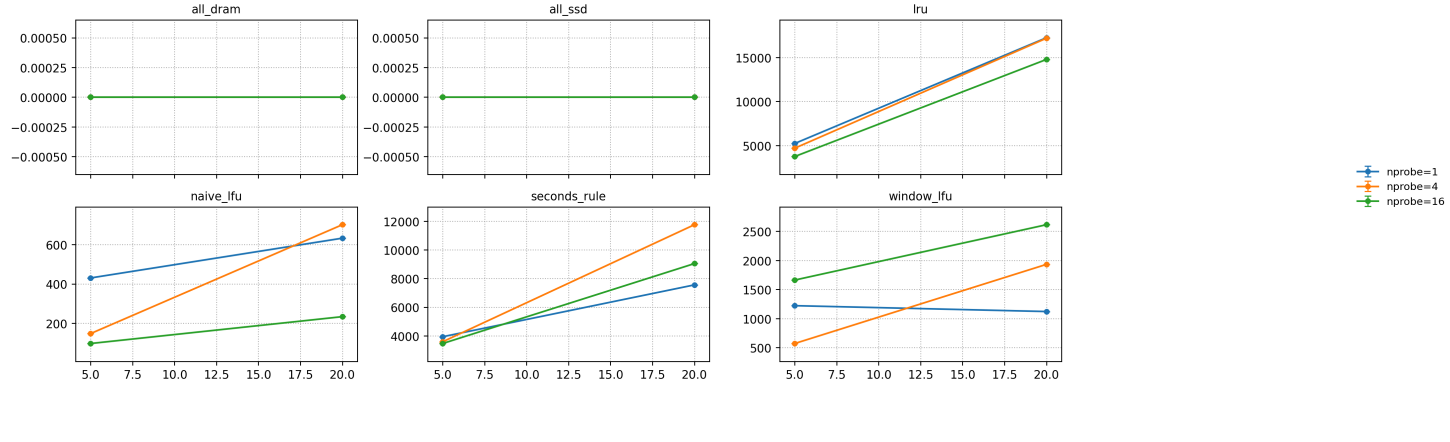
7.2.4 Figures

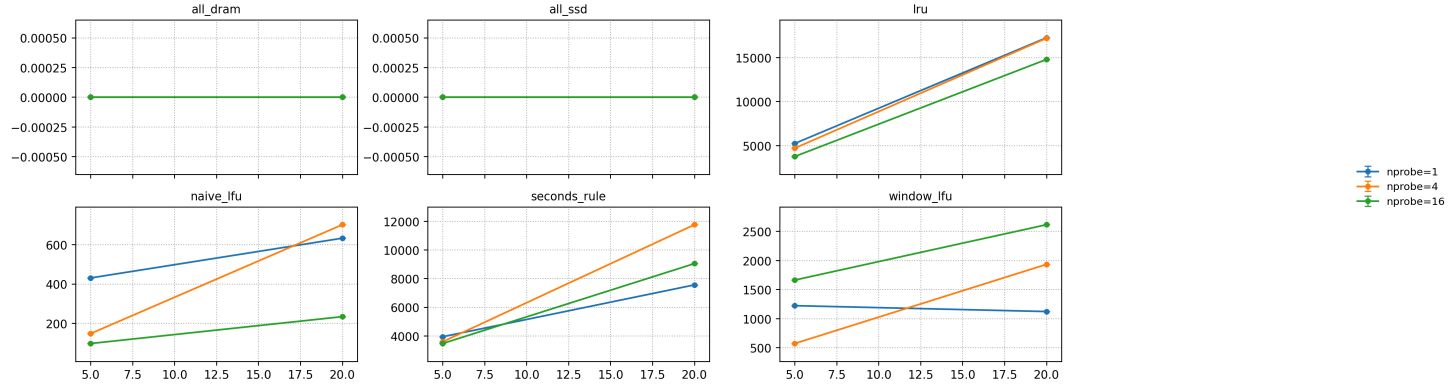
IO amplification



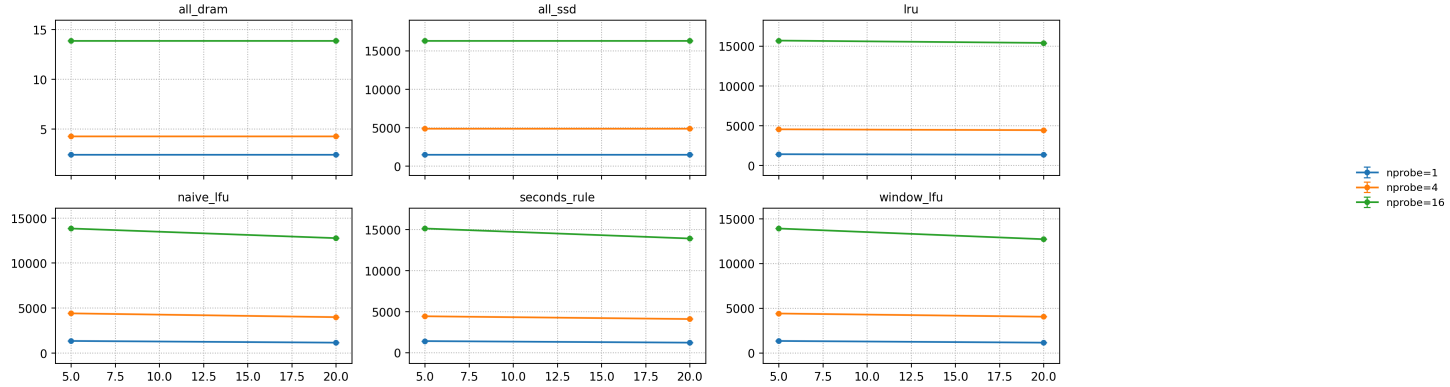


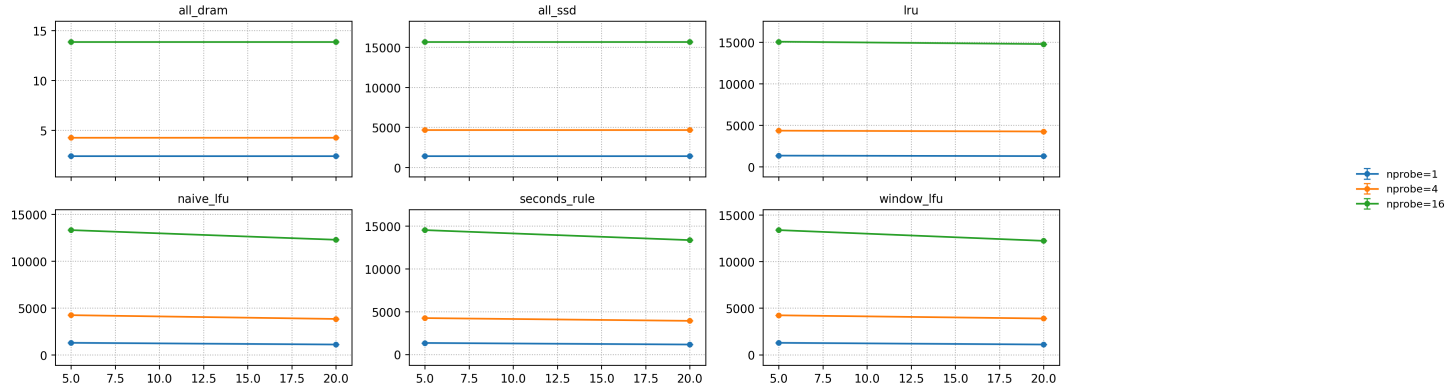
Migration



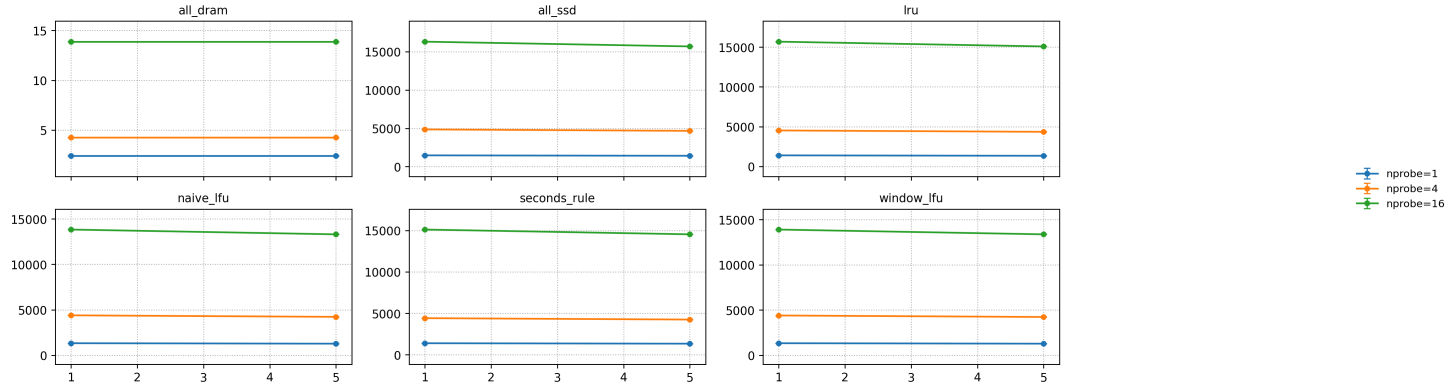


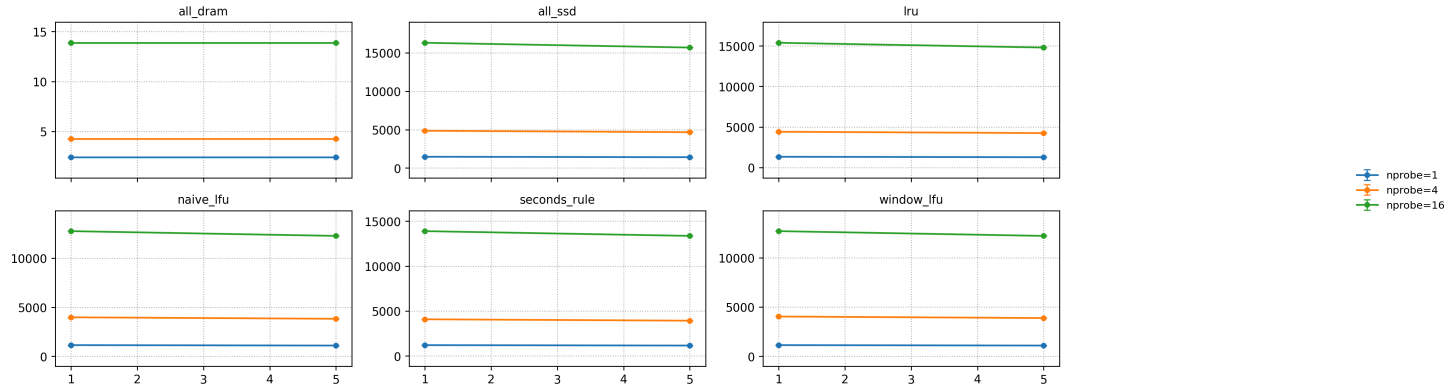
p95 vs DRAM



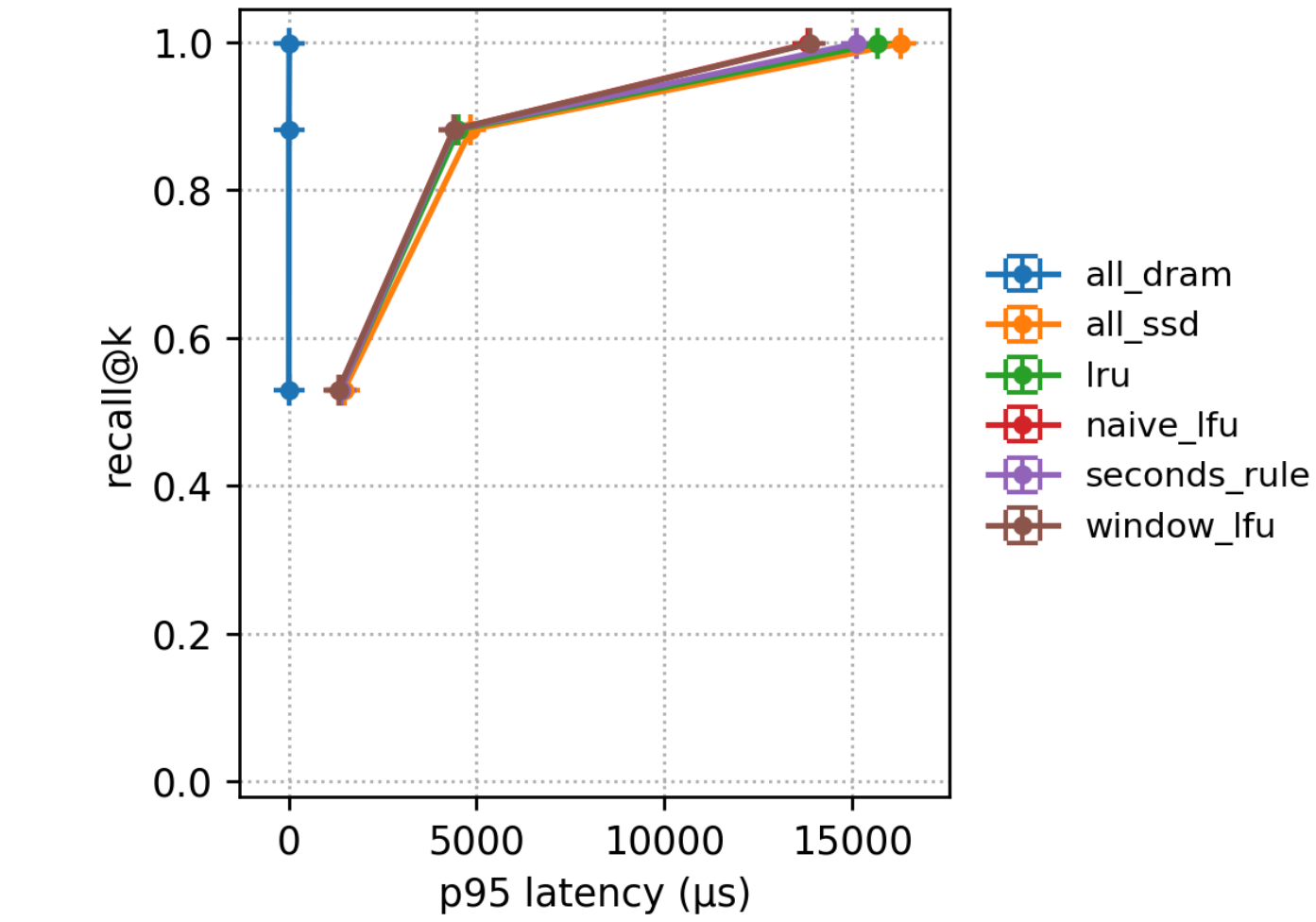


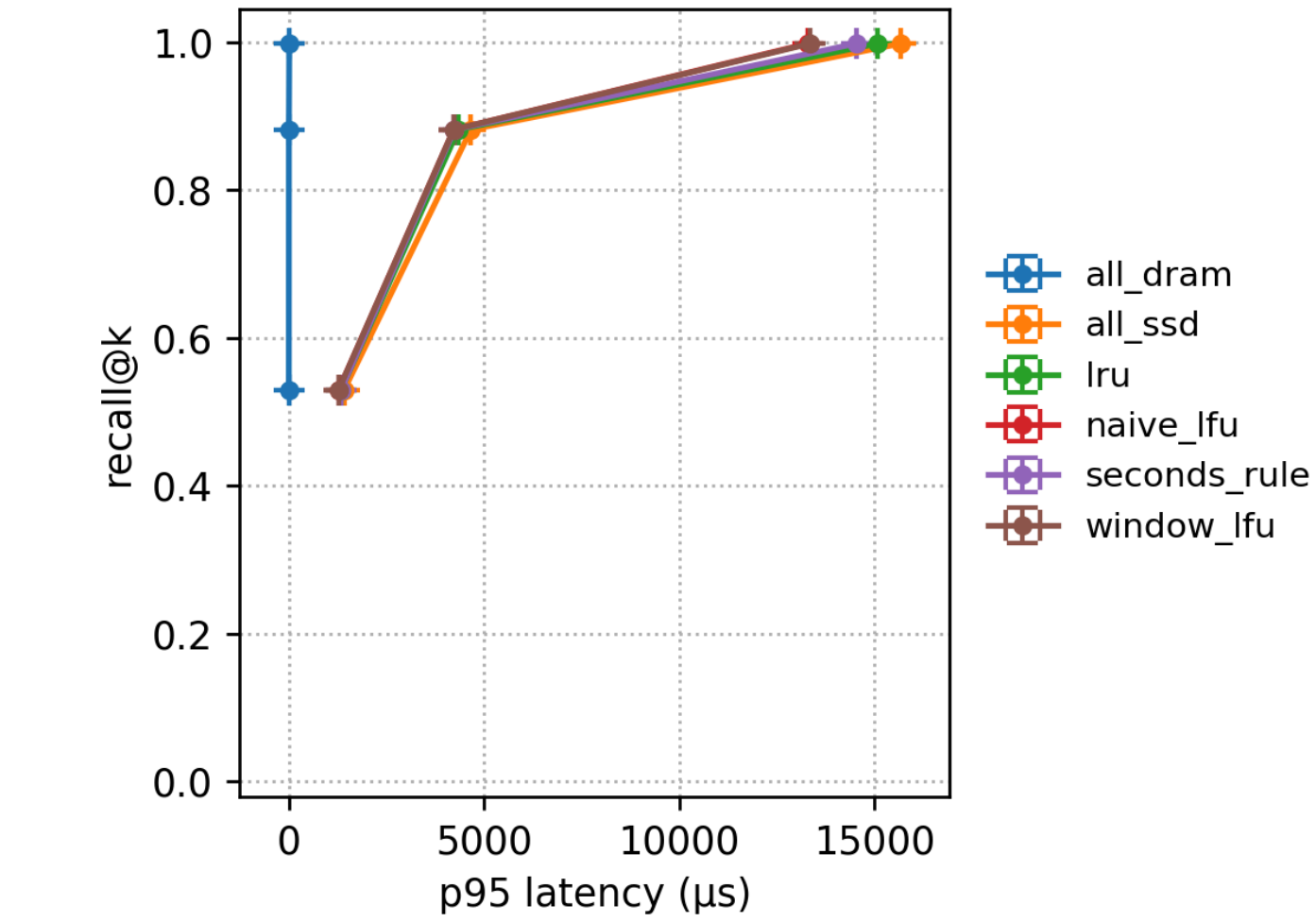
p95 vs IOPS

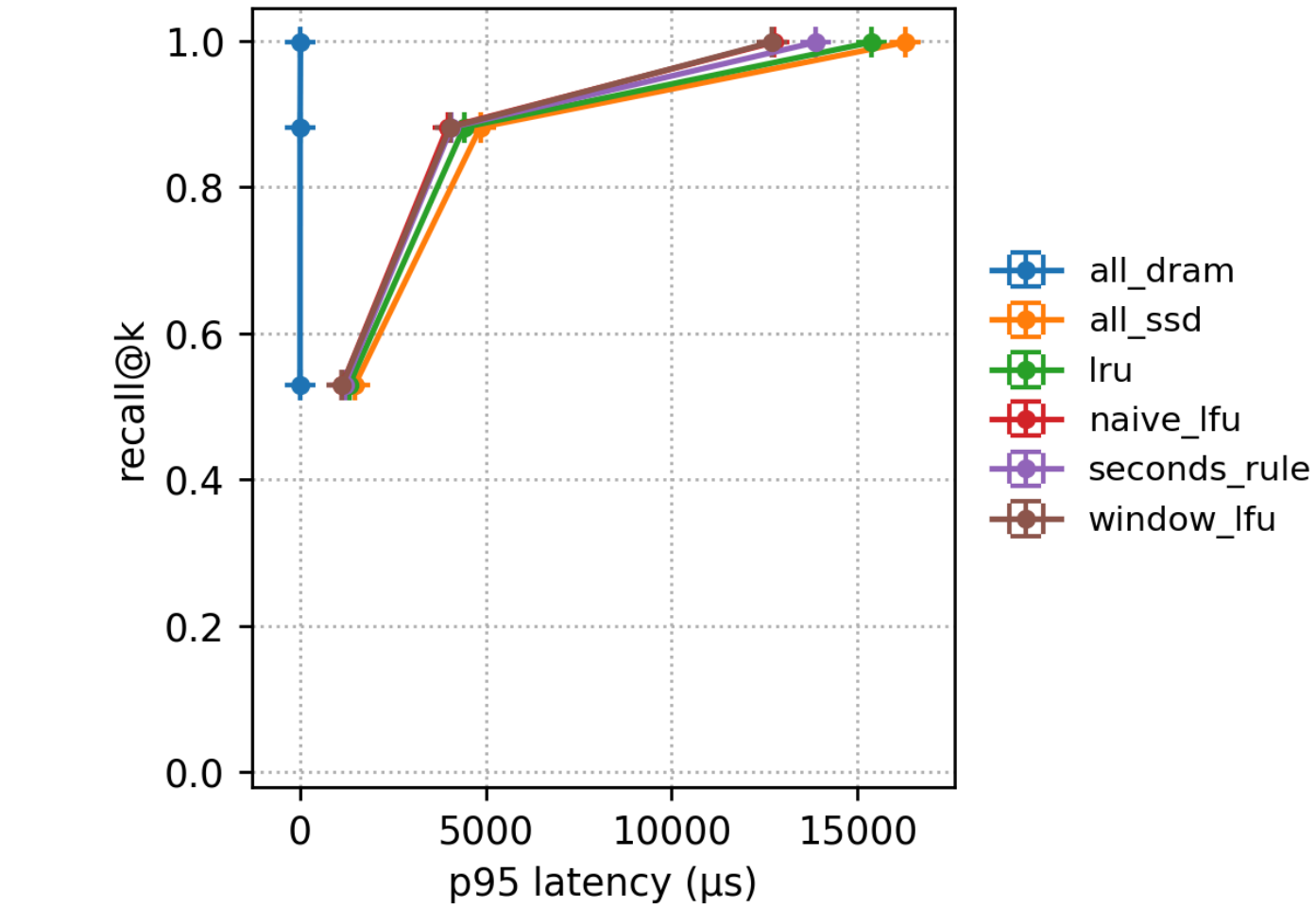


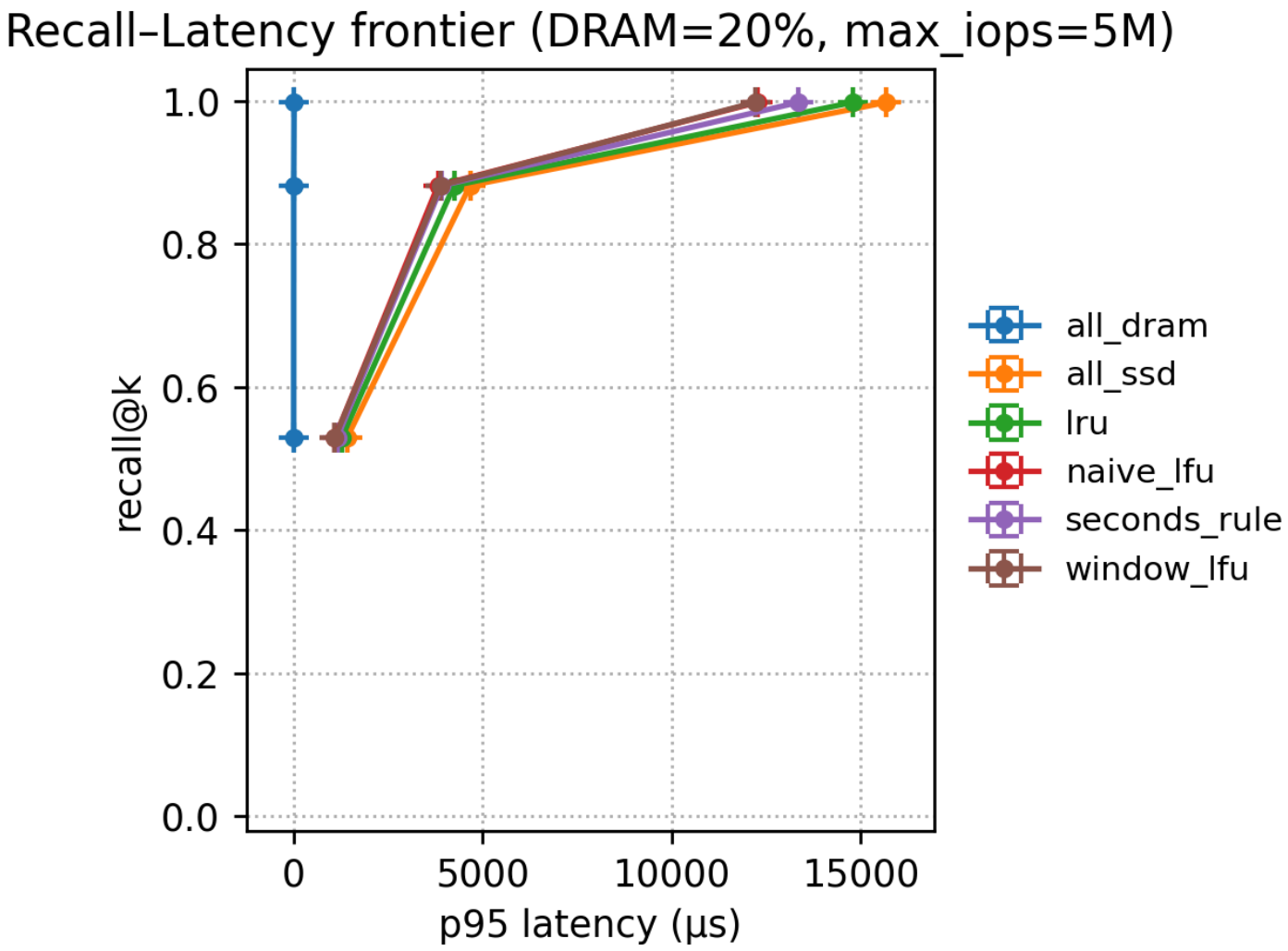


Recall–latency frontier

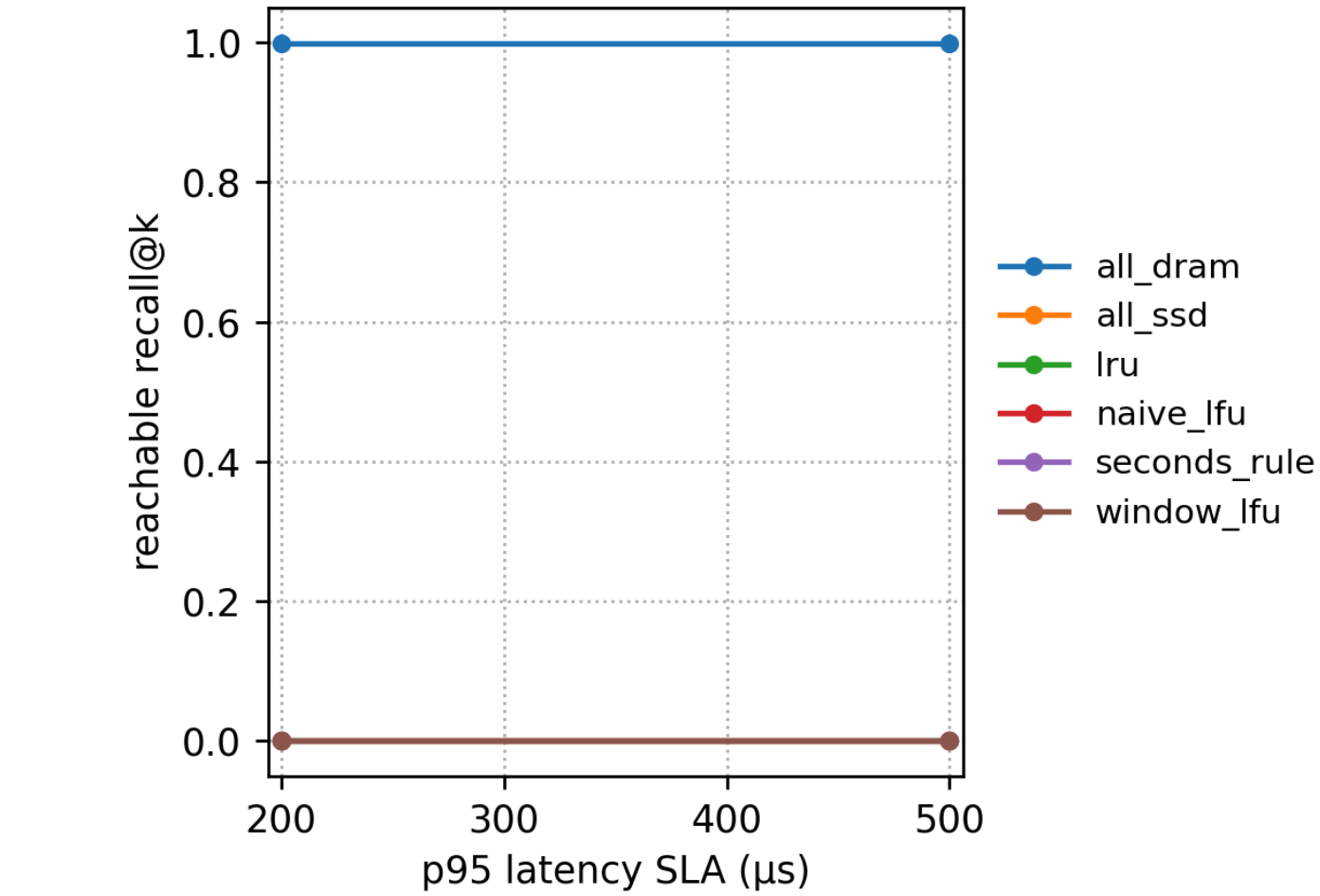

Recall–Latency frontier (DRAM=5%, max_iops=1M)
recall@k
1.0
0.8
0.6
0.4
0.2
0.0
0
5000
10000
15000
p95 latency (µs)
all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu

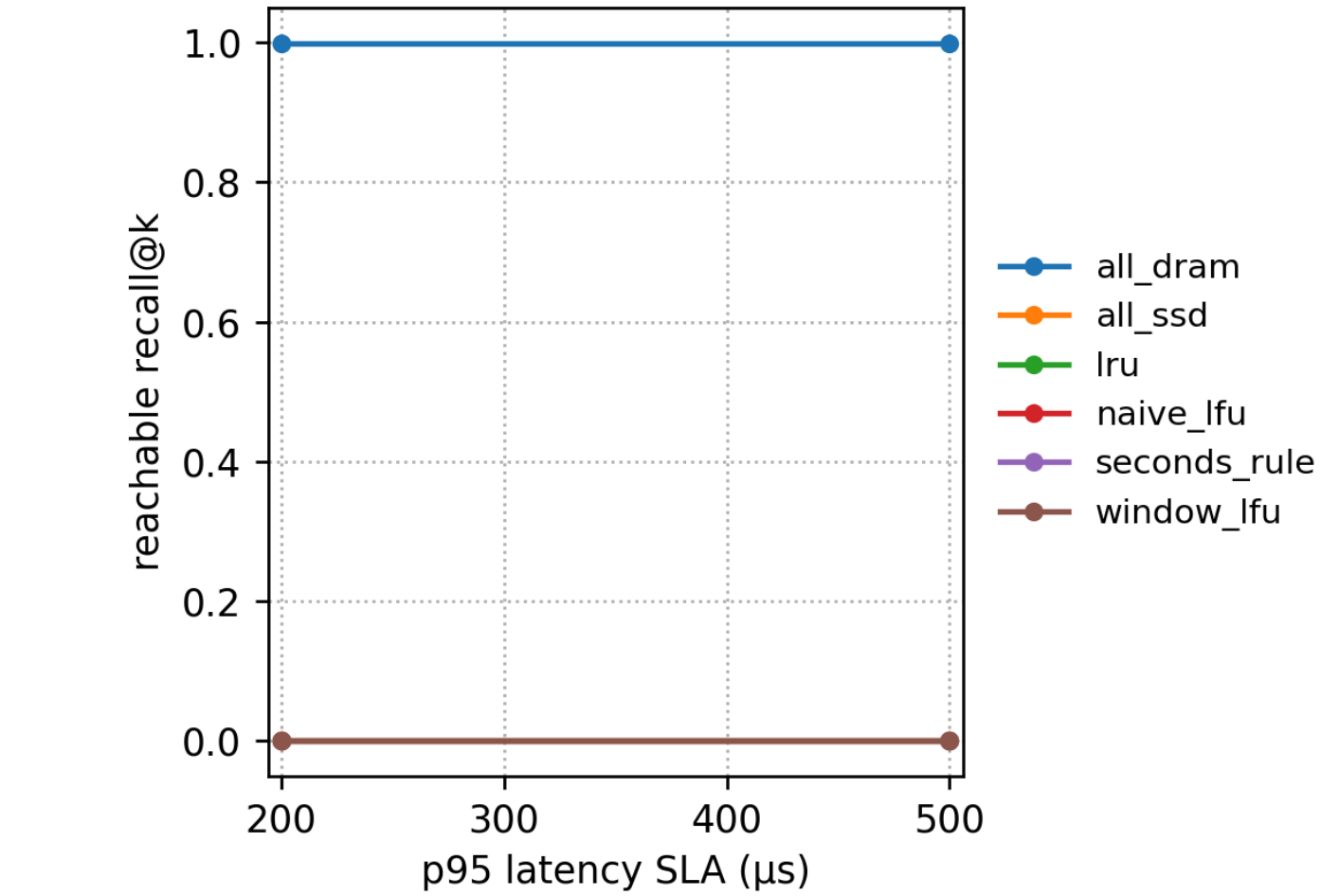

Recall–Latency frontier (DRAM=5%, max_iops=5M)
recall@k
1.0
0.8
0.6
0.4
0.2
0.0
0
5000
10000
15000
p95 latency (µs)
all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu

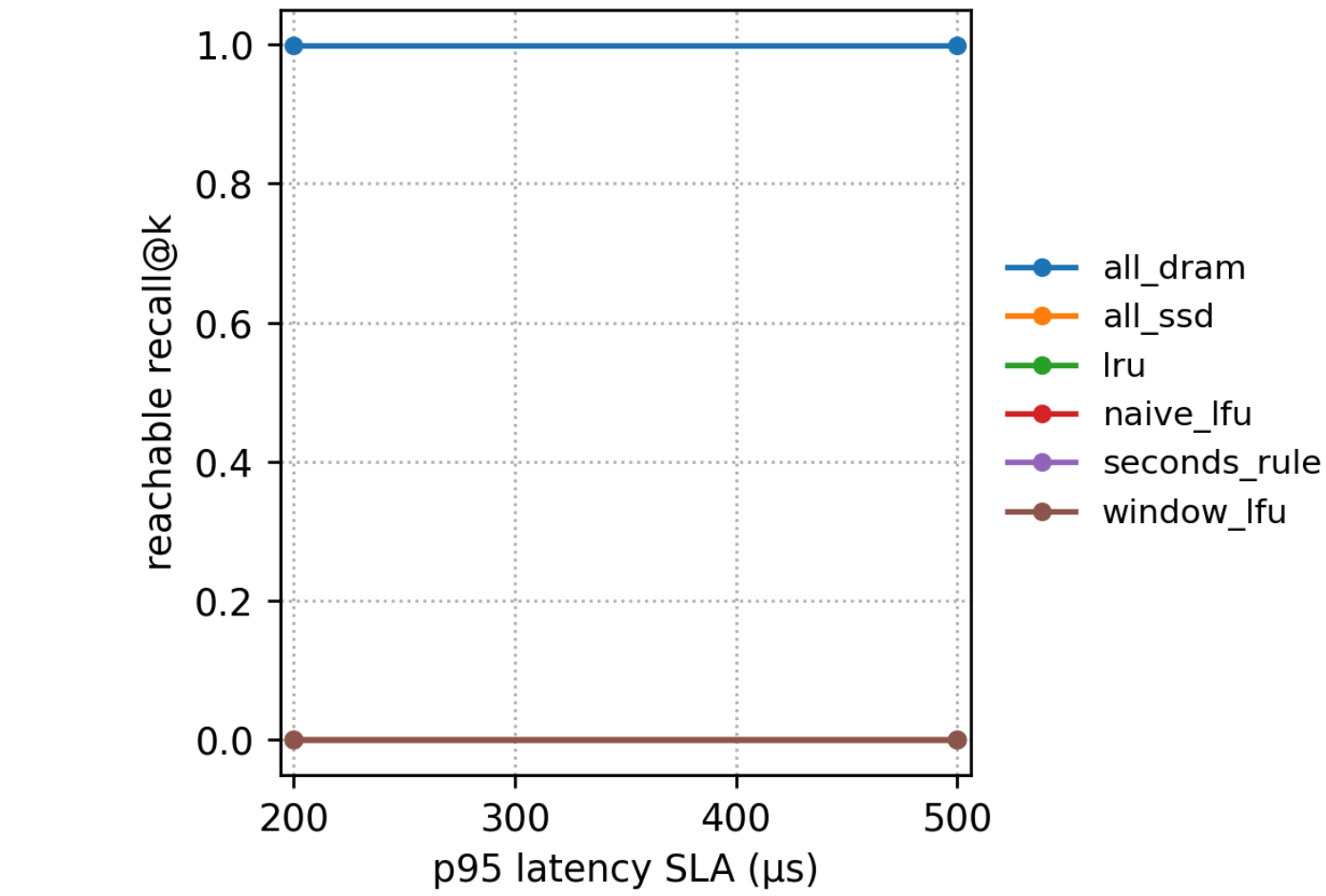

Recall–Latency frontier (DRAM=20%, max_iops=1M)
recall@k
1.0
0.8
0.6
0.4
0.2
0.0
0
5000
10000
15000
p95 latency (µs)
all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu

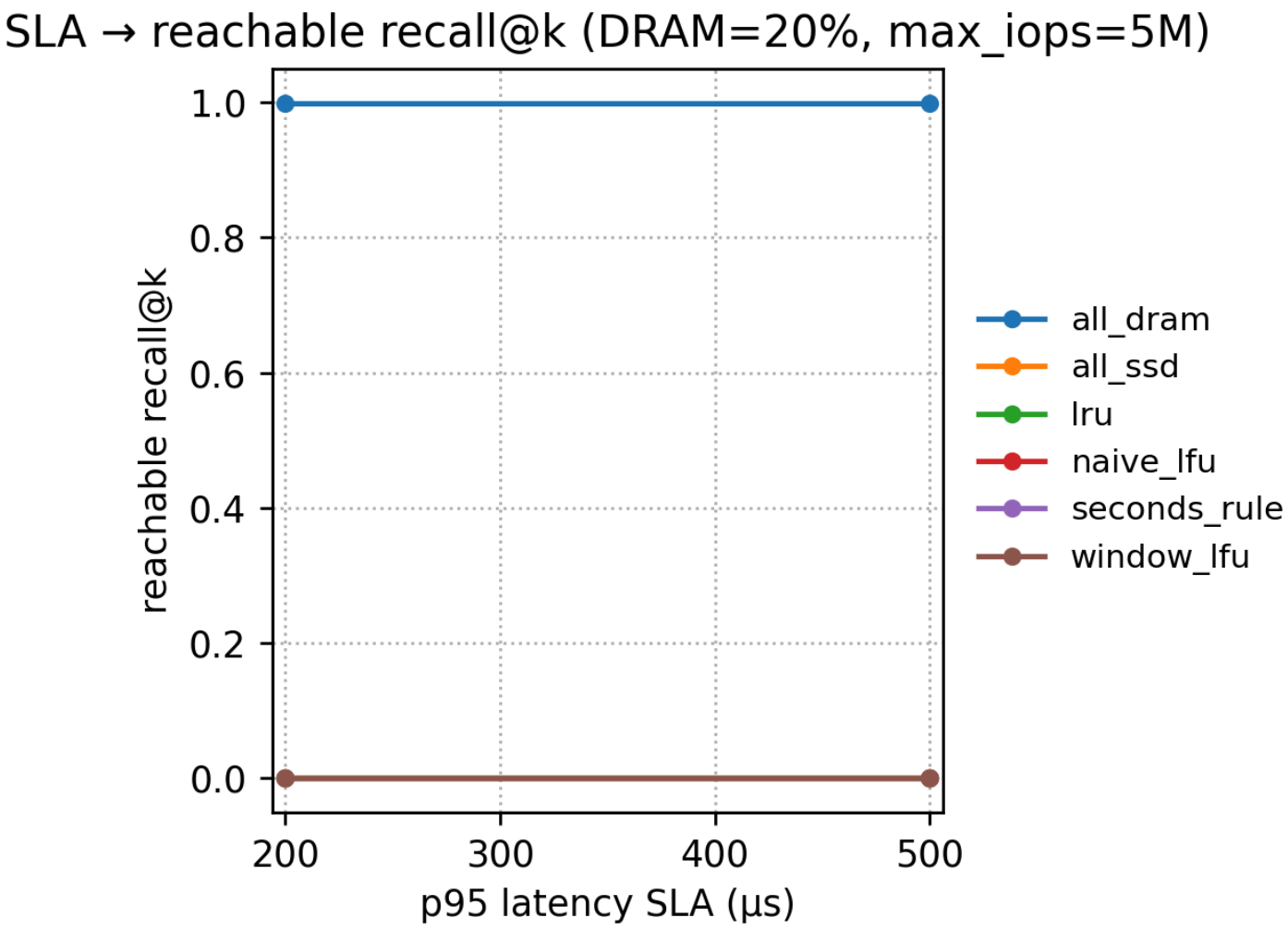


SLA reachable recall


SLA → reachable recall@k (DRAM=5%, max_iops=1M)
reachable recall@k
1.0
0.8
0.6
0.4
0.2
0.0
200
300
400
500
p95 latency SLA (µs)
all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu


SLA → reachable recall@k (DRAM=5%, max_iops=5M)
reachable recall@k
1.0
0.8
0.6
0.4
0.2
0.0
200
300
400
500
p95 latency SLA (µs)
all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu


SLA → reachable recall@k (DRAM=20%, max_iops=1M)
reachable recall@k
1.0
0.8
0.6
0.4
0.2
0.0
200
300
400
500
p95 latency SLA (µs)
all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu



7.3 Hotspot workload (fast fast)

Run directory: `results/sr_sift_exp_hotspot_fast_fast_20251215_135028`

The core characteristic of this workload is: hotspots change quickly, and the lists that are accessed tend to be long, so SSD page reads under all_ssd are very high, and p95 latency naturally stays in the millisecond range.

7.3.1 Key numbers (max_iops=5M, nprobe=1)

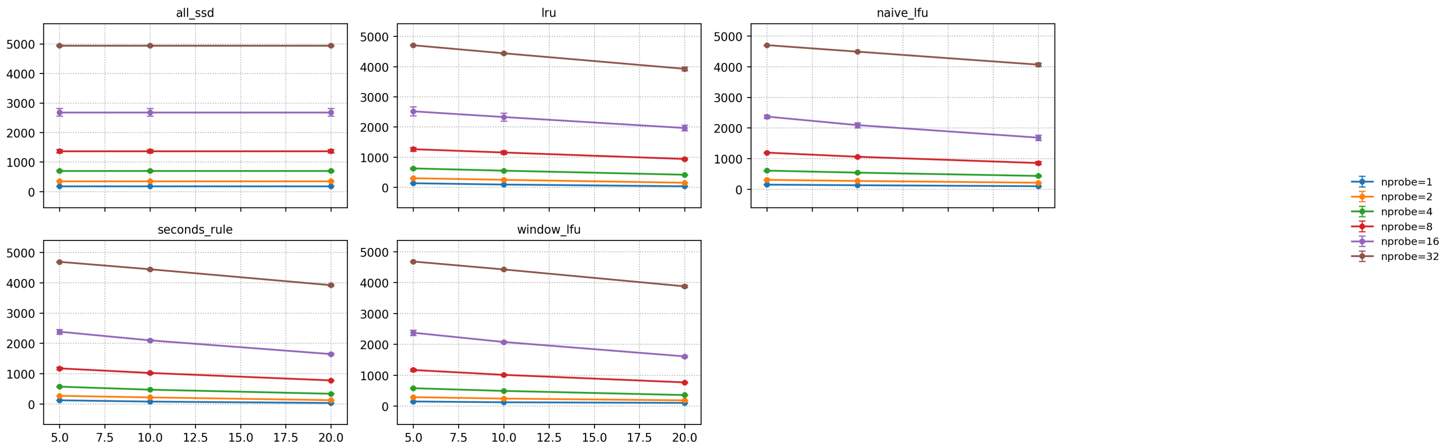
dram_fraction	policy	p95 (µs)	recall	IOAmp	total_migration_bytes
0.05	all_ssd	~8100	0.5293	~179.99	0
0.05	naive_lfu	~6560	0.5293	~150.10	~50 MB
0.05	window_lfu	~6960	0.5293	~148.18	~121 MB
0.05	seconds_rule	~6680	0.5293	~119.26	~671 MB
0.05	lru	~6800	0.5293	~131.46	~1.04 GB
0.10	all_ssd	~8100	0.5293	~179.99	0

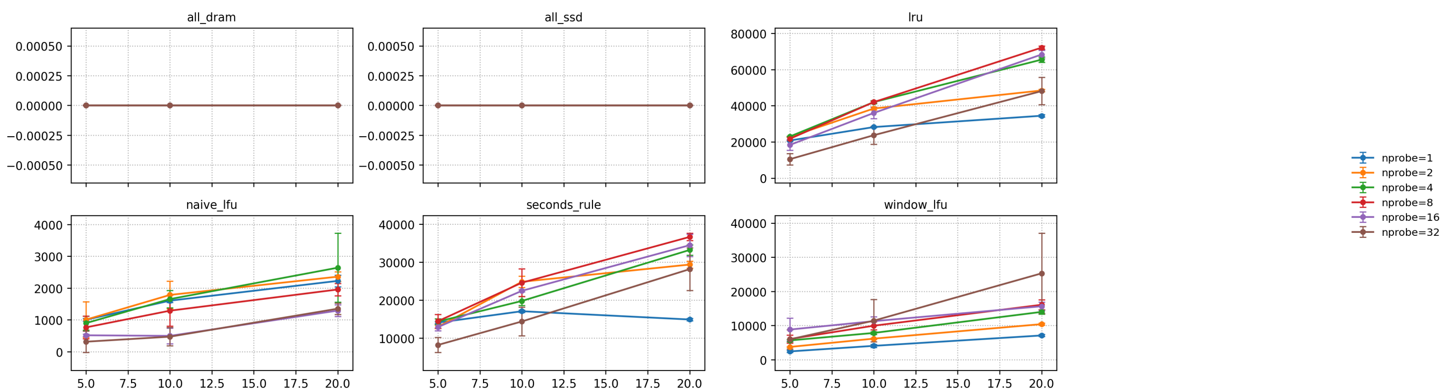
dram_fraction	policy	p95 (μs)	recall	IOAmp	total_migration_bytes
0.10	naive_lfu	~6210	0.5293	~130.87	~80 MB
0.10	window_lfu	~6680	0.5293	~122.45	~203 MB
0.10	seconds_rule	~6090	0.5293	~76.23	~854 MB
0.10	lru	~6680	0.5293	~86.80	~1.41 GB
0.20	all_ssd	~8100	0.5293	~179.99	0
0.20	naive_lfu	~5720	0.5293	~99.11	~111 MB
0.20	window_lfu	~6170	0.5293	~102.39	~355 MB
0.20	seconds_rule	~5090	0.5293	~31.57	~745 MB
0.20	lru	~5090	0.5293	~30.63	~1.72 GB

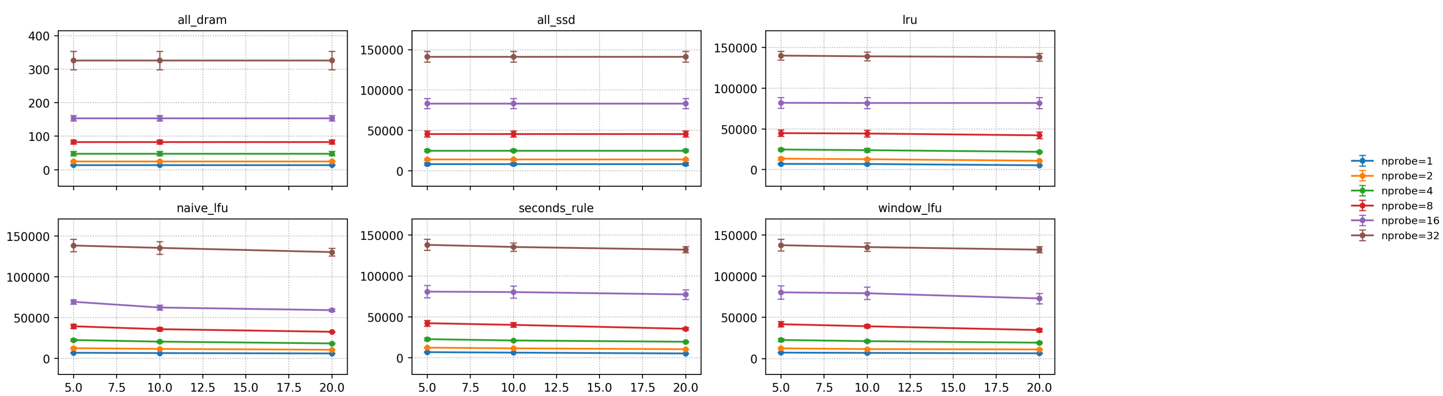
Observations and explanations

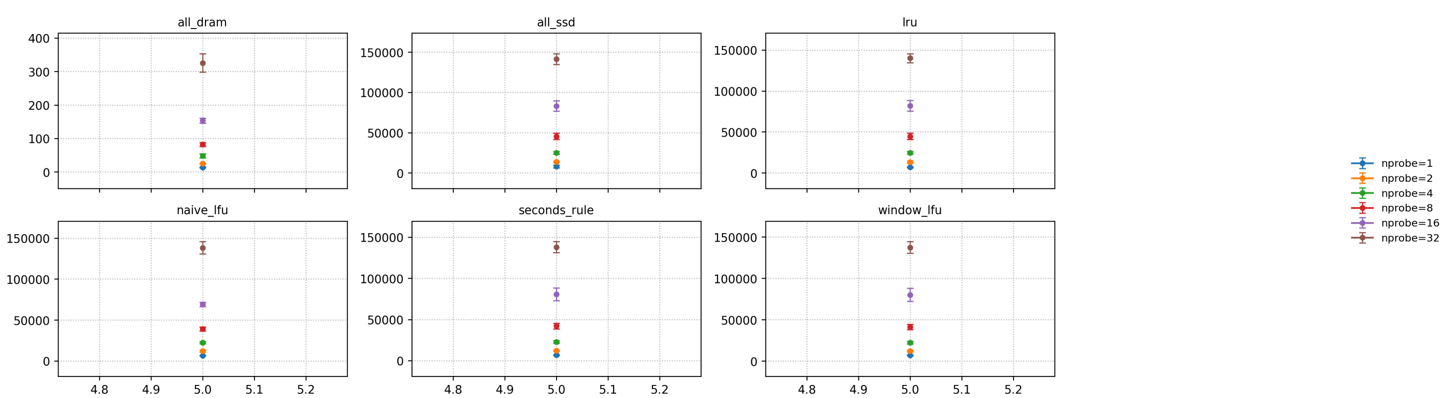
- all_ssd has IOAmp ~180, meaning that a query reads a very large number of SSD pages, so p95 ~8 ms.
- As dram fraction increases, IOAmp drops across all policies, and p95 drops accordingly: this is the direct outcome of “more lists residing in DRAM → fewer SSD pages read”.
- Under dram=0.2, `seconds_rule/lru` can push IOAmp down to ~31, with p95 ~5.09 ms: this shows they are very good at catching the true hotspots when “hotspots drift quickly”, thus moving hotspot lists into DRAM as much as possible.
- But the cost is very clear: `total_migration_bytes` reaches the GB level. This means the policy’s benefits come from “aggressive movement”. From an engineering perspective, one must consider whether this movement will squeeze bandwidth, increase write amplification, or affect stability.
- `naive_lfu/window_lfu` have significantly smaller migration volumes (tens to hundreds of MB) , but their IOAmp is clearly higher than seconds_rule/lru: this indicates that in fast-changing-hotspot workloads, pure frequency statistics suffer from “slow reaction/lag”, and when hotspots just shift, they have not yet moved the new hotspots into DRAM in time.

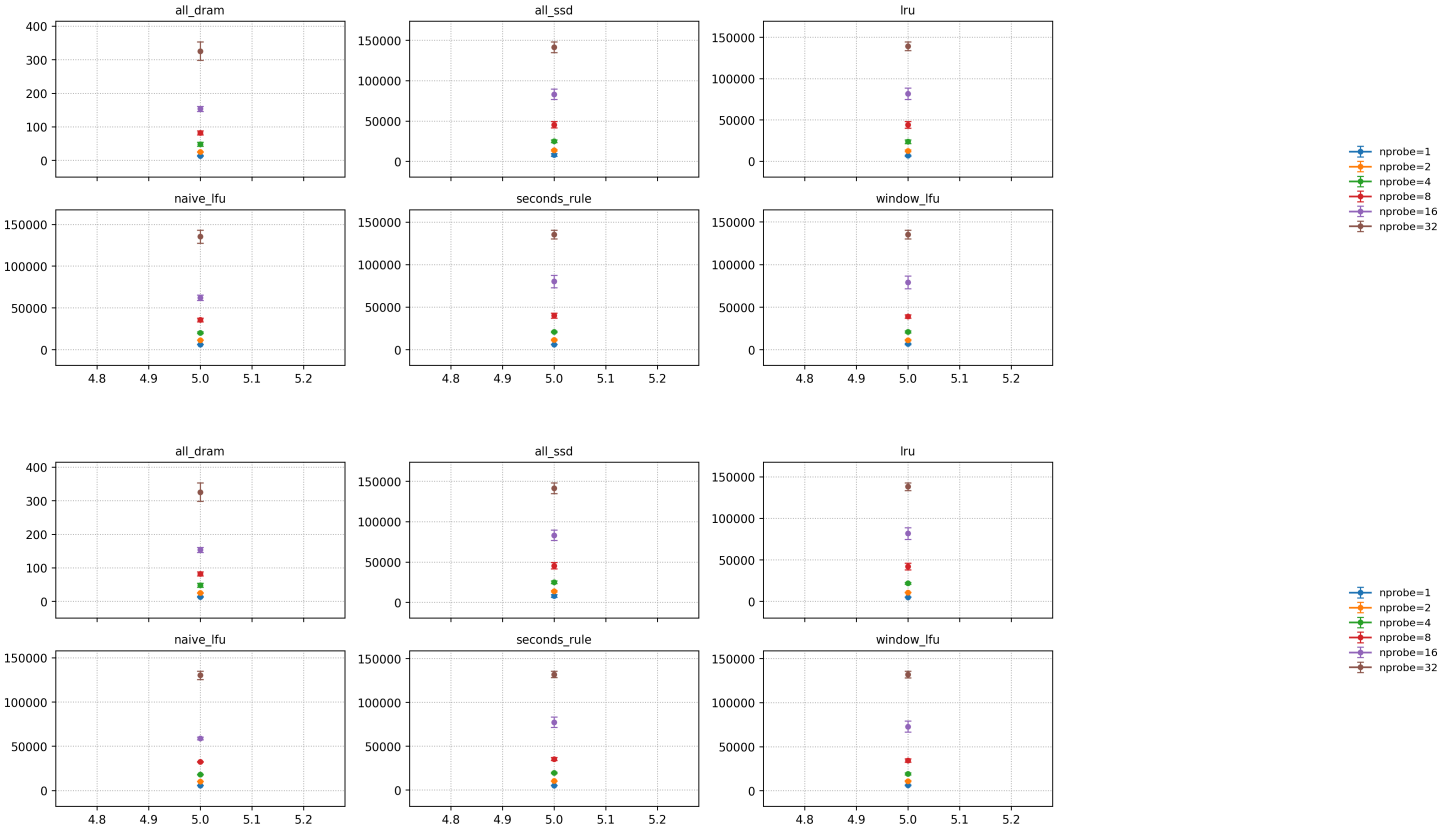
7.3.2 Figures


all_ssd
lru
naive_lfu
seconds_rule
window_lfu
nprobe=1
nprobe=2
nprobe=4
nprobe=8
nprobe=16
nprobe=32

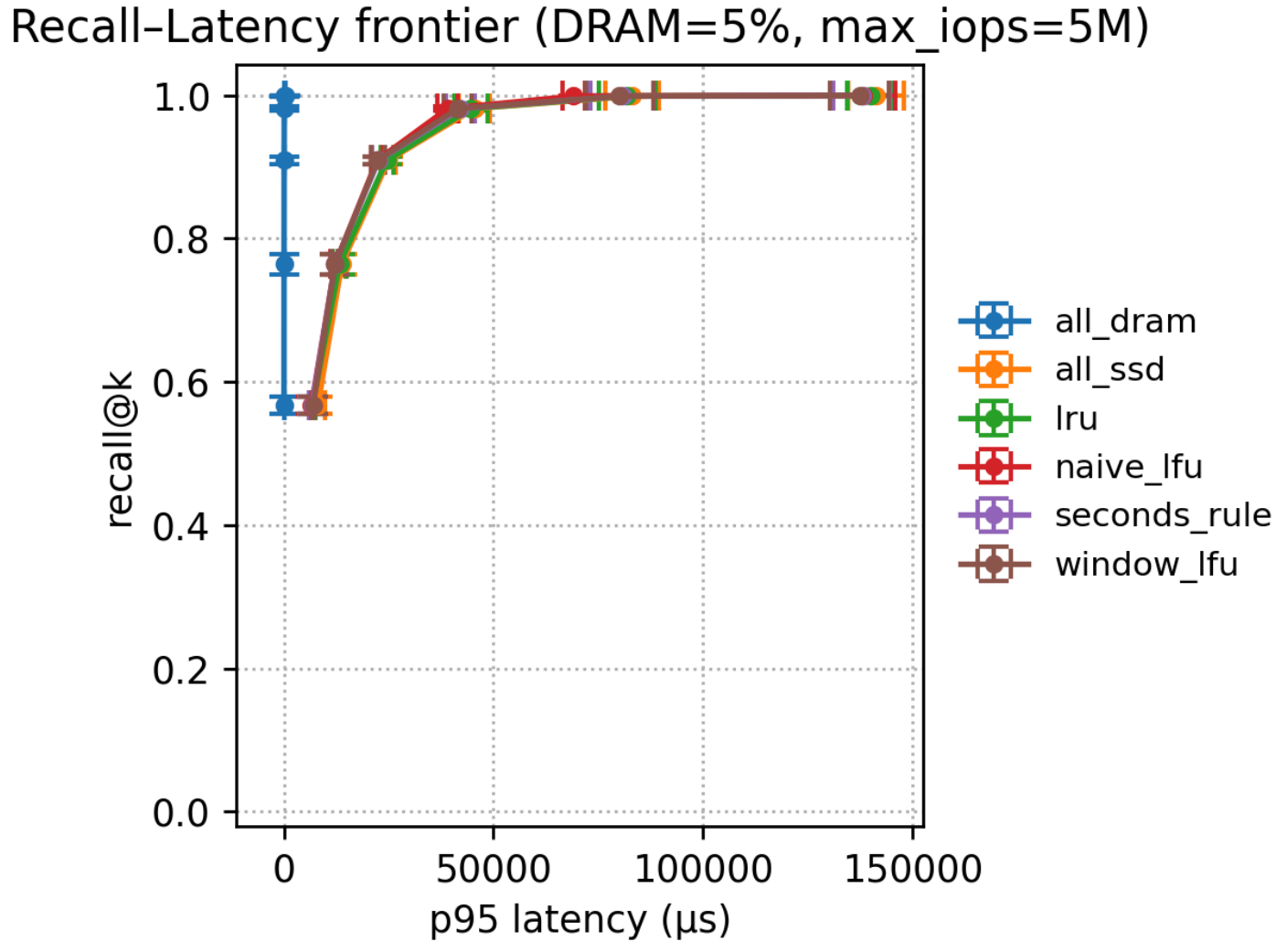

all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu
nprobe=1
nprobe=2
nprobe=4
nprobe=8
nprobe=16
nprobe=32

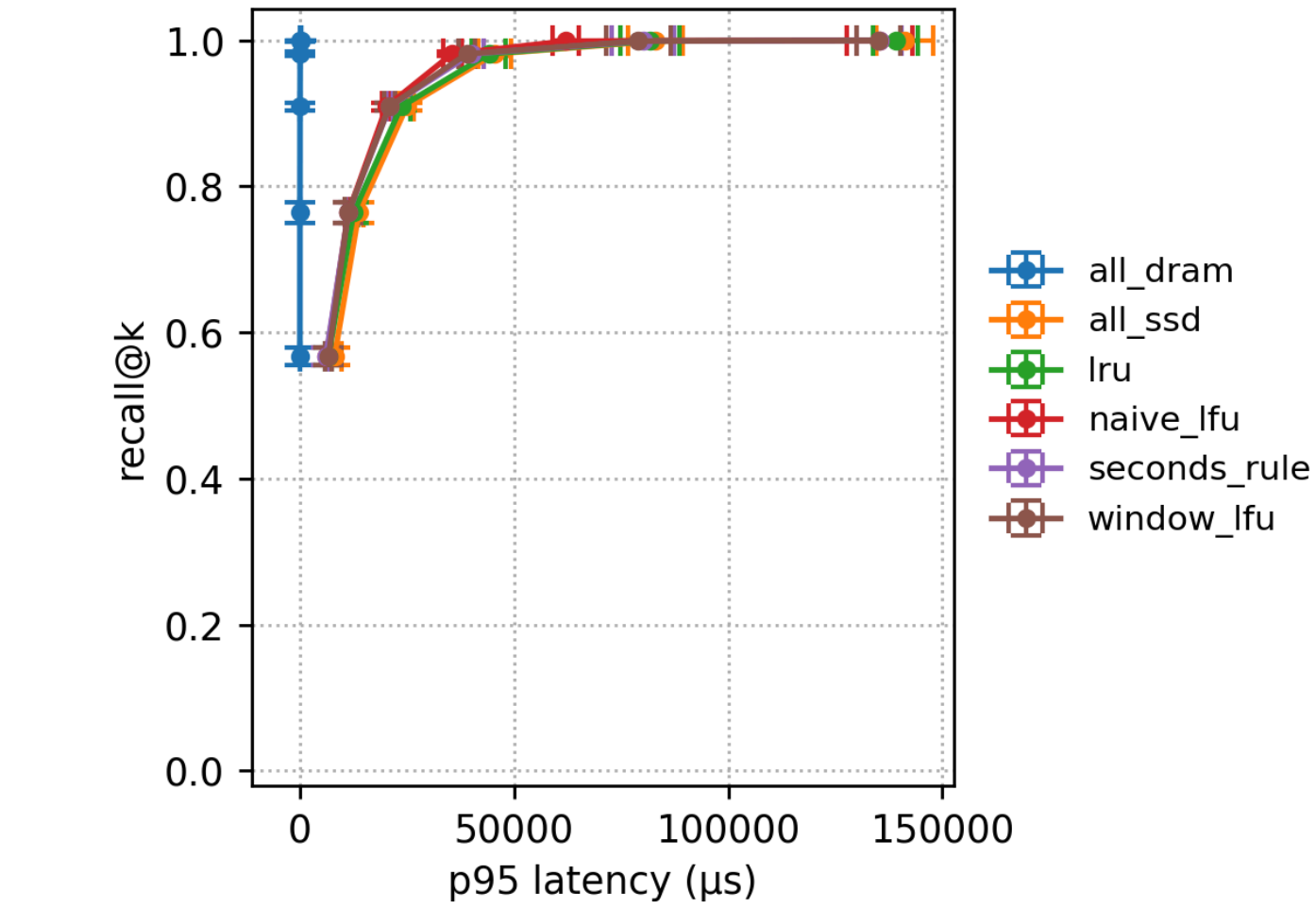

all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu
nprobe=1
nprobe=2
nprobe=4
nprobe=8
nprobe=16
nprobe=32

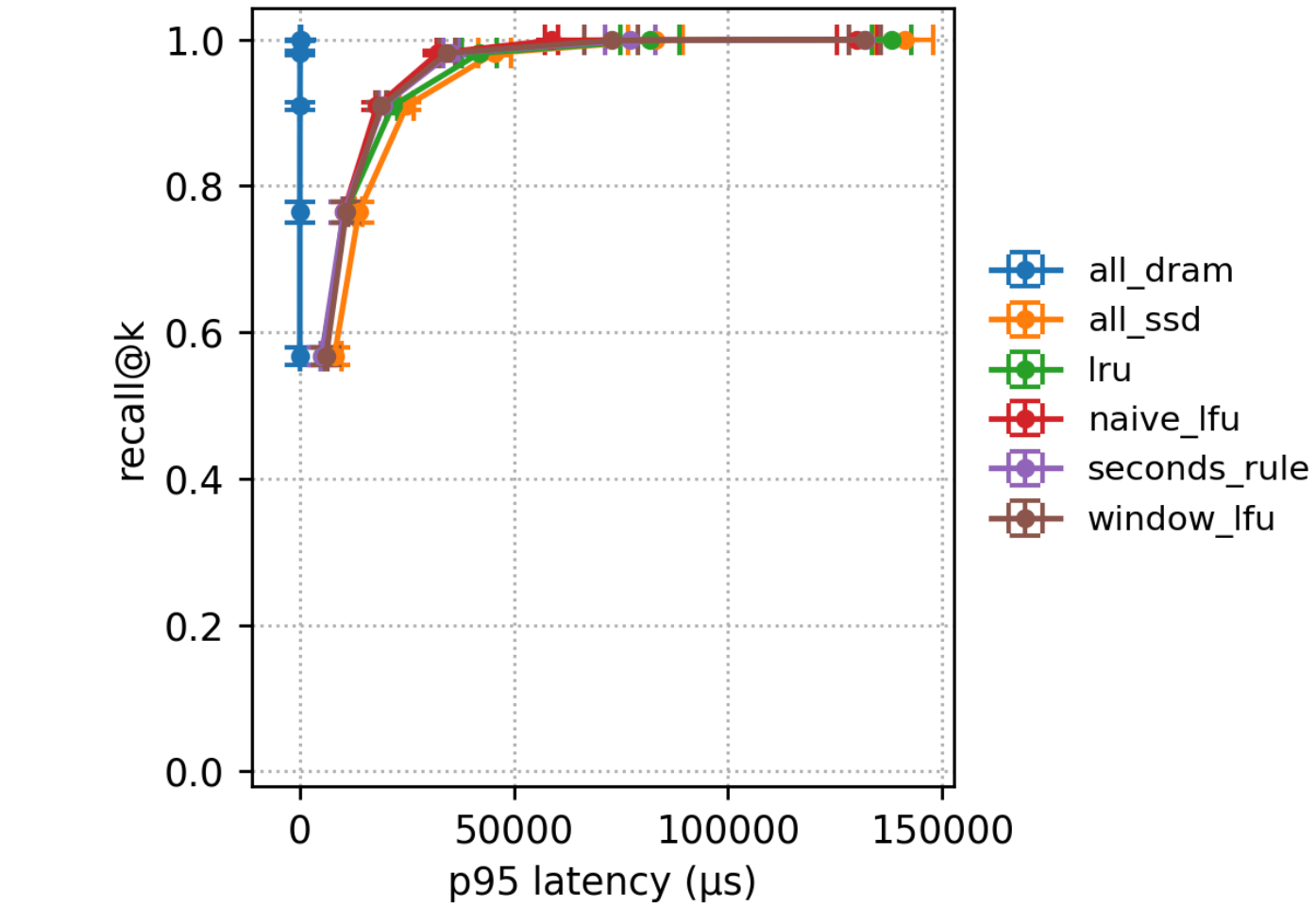

all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu
nprobe=1
nprobe=2
nprobe=4
nprobe=8
nprobe=16
nprobe=32

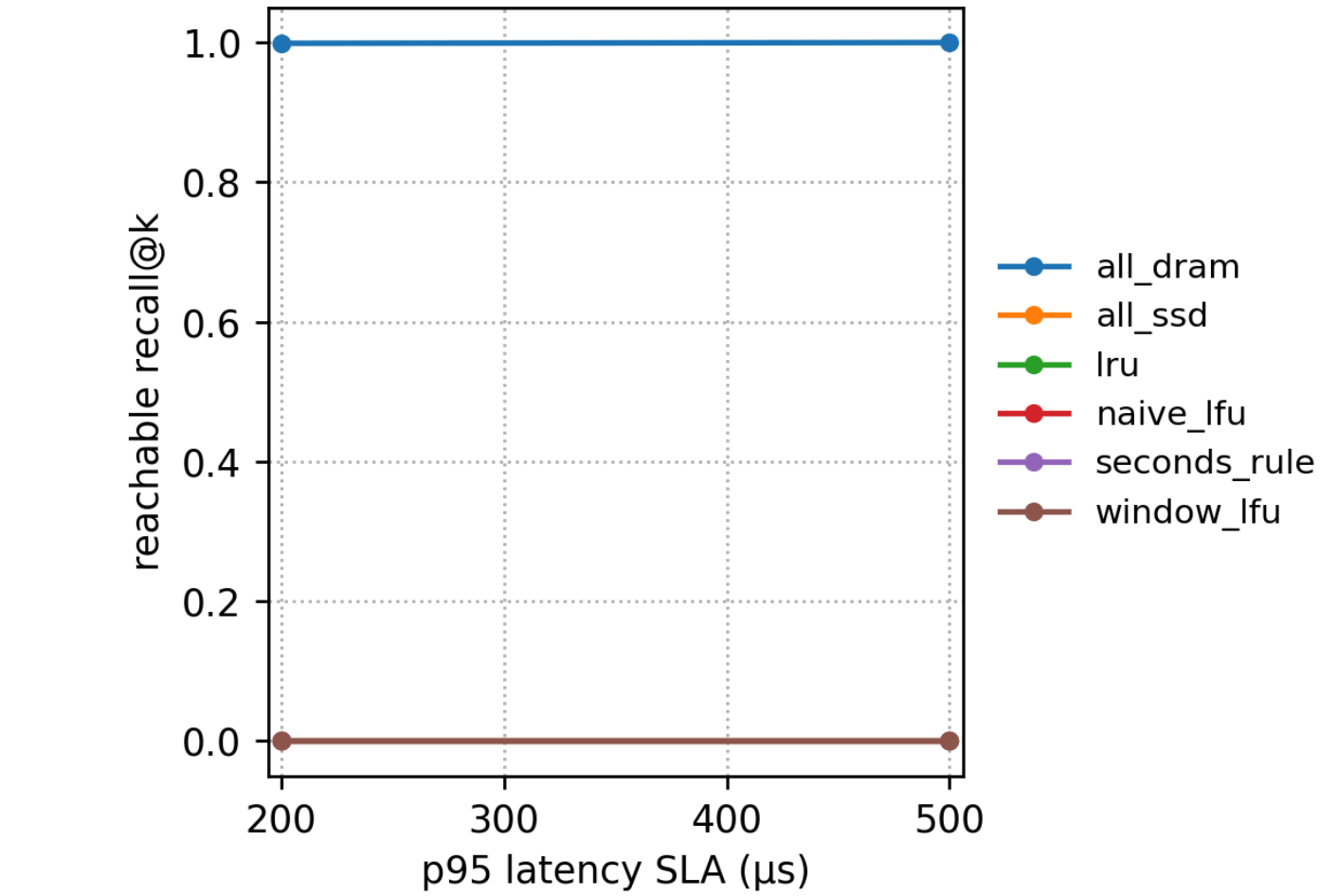

all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu
nprobe=1
nprobe=2
nprobe=4
nprobe=8
nprobe=16
nprobe=32

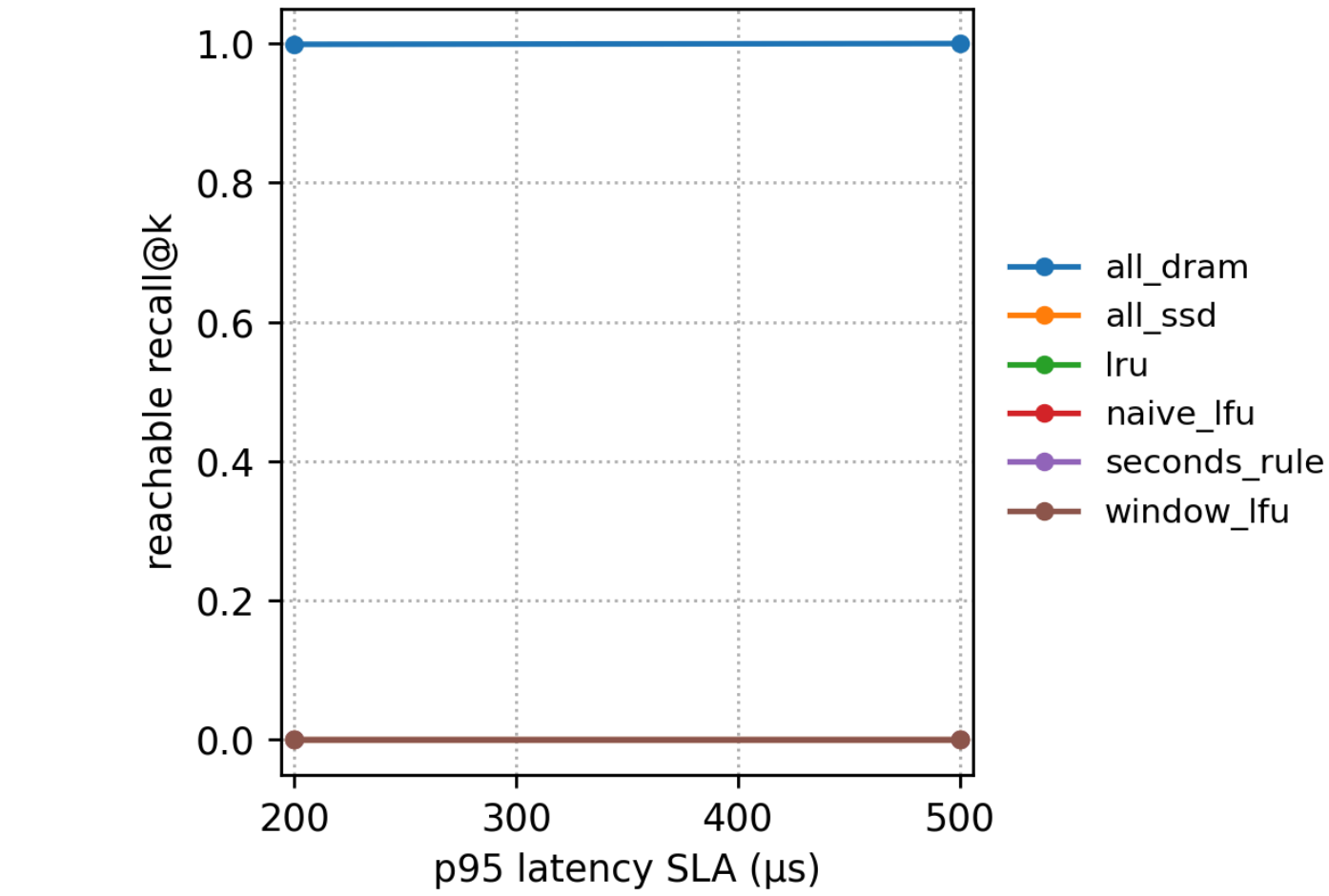
all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu
nprobe=1
nprobe=2
nprobe=4
nprobe=8
nprobe=16
nprobe=32

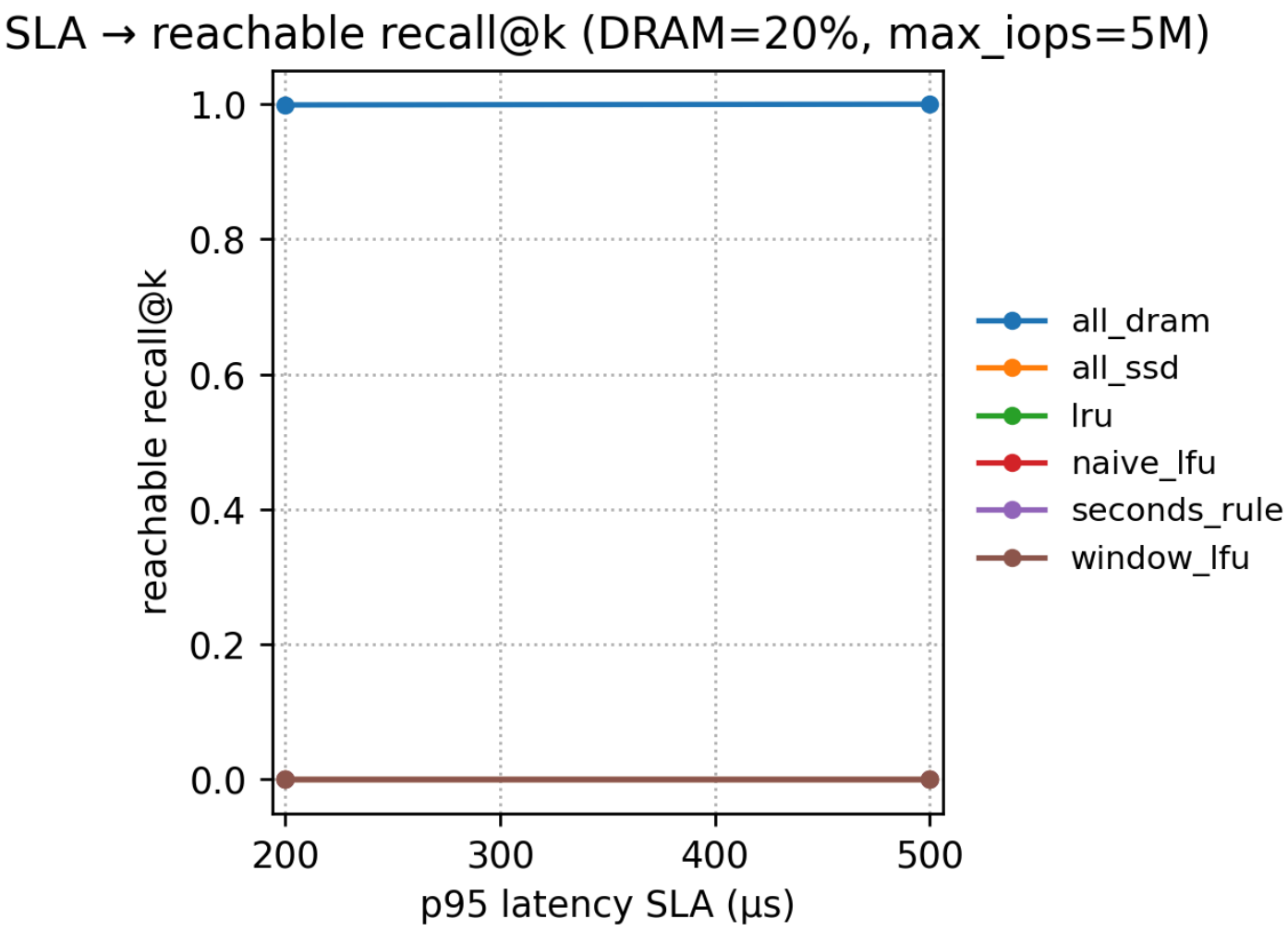

Recall–Latency frontier (DRAM=5%, max_iops=5M)
recall@k
p95 latency (µs)
all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu


Recall–Latency frontier (DRAM=10%, max_iops=5M)
recall@k
1.0
0.8
0.6
0.4
0.2
0.0
0
50000
100000
150000
p95 latency (µs)
all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu


Recall–Latency frontier (DRAM=20%, max_iops=5M)
recall@k
1.0
0.8
0.6
0.4
0.2
0.0
0
50000
100000
150000
p95 latency (µs)
all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu


SLA → reachable recall@k (DRAM=5%, max_iops=5M)
reachable recall@k
1.0
0.8
0.6
0.4
0.2
0.0
200
300
400
500
p95 latency SLA (µs)
all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu


SLA → reachable recall@k (DRAM=10%, max_iops=5M)
reachable recall@k
1.0
0.8
0.6
0.4
0.2
0.0
200
300
400
500
p95 latency SLA (µs)
all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu



7.4 Hotspot workload (slow fast)

Run directory: `results/sr_sift_exp_hotspot_slow_fast_20251215_135028`

This workload still has hotspots and long lists, but hotspots change more slowly. Intuitively, this makes “frequency-based policies” more likely to stably capture hotspots, thus achieving decent IOAmp reductions with lower migration overhead.

7.4.1 Key numbers (max_iops=5M, nprobe=1)

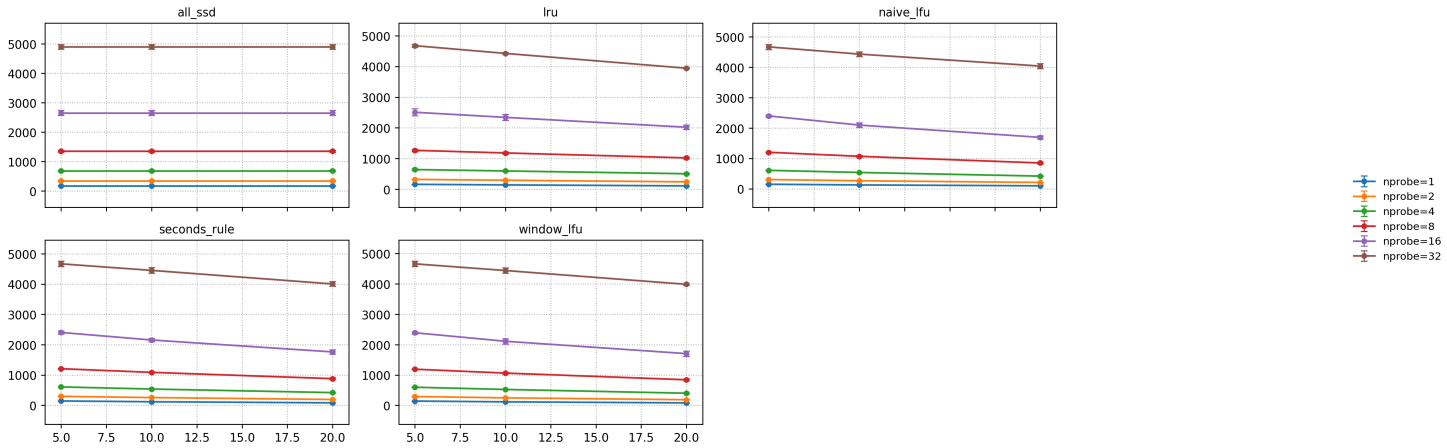
dram_fraction	policy	p95 (µs)	recall	IOAmp	total_migration_bytes
0.05	all_ssd	~7110	0.5293	~178.60	0
0.05	window_lfu	~6680	0.5293	~140.11	~84 MB
0.05	naive_lfu	~6840	0.5293	~147.64	~54 MB
0.05	seconds_rule	~6680	0.5293	~141.50	~651 MB
0.05	lru	~7110	0.5293	~159.71	~1.24 GB

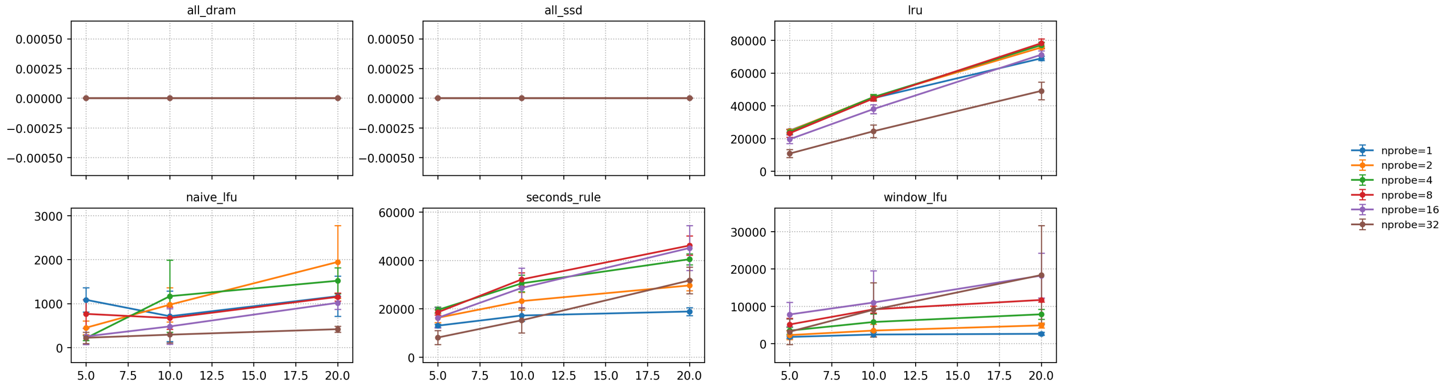
dram_fraction	policy	p95 (μs)	recall	IOAmp	total_migration_bytes
0.10	window_lfu	~5870	0.5293	~114.20	~121 MB
0.10	naive_lfu	~6290	0.5293	~125.51	~35.5 MB
0.10	seconds_rule	~6280	0.5293	~69.73	~299 MB
0.10	lru	~6840	0.5293	~141.72	~2.23 GB
0.20	window_lfu	~5610	0.5293	~80.13	~130 MB
0.20	naive_lfu	~6030	0.5293	~96.84	~58 MB
0.20	seconds_rule	~5570	0.5293	~80.11	~944 MB
0.20	lru	~6280	0.5293	~109.89	~3.45 GB

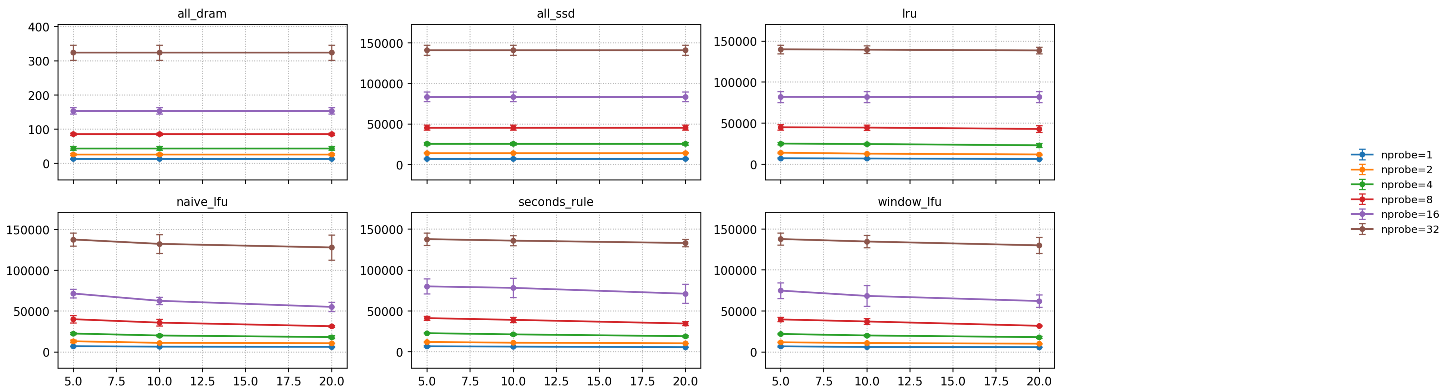
Observations and explanations

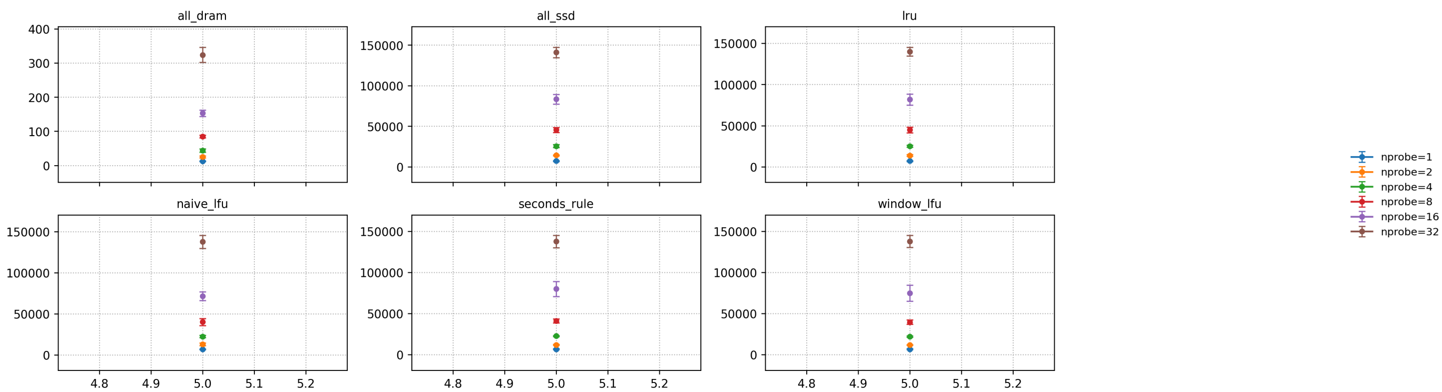
- Under dram=0.2, `window_lfu` has IOAmp ~80.13, almost the same as `seconds_rule` at ~80.11, but its migration volume is only ~130 MB, compared to seconds_rule's ~944 MB, showing that when hotspots change more slowly, "sliding-window frequency" is enough to stably lock onto hotspots.
- `lru` is extreme in migration (GB level) but IOAmp is not better than window_lfu, exhibiting the characteristic of "excessive movement with limited benefit".
- Overall, slow fast is more suitable for choosing "frequency/sliding-window frequency" policies: they obtain close or even better IOAmp and p95 with much lower migration cost.

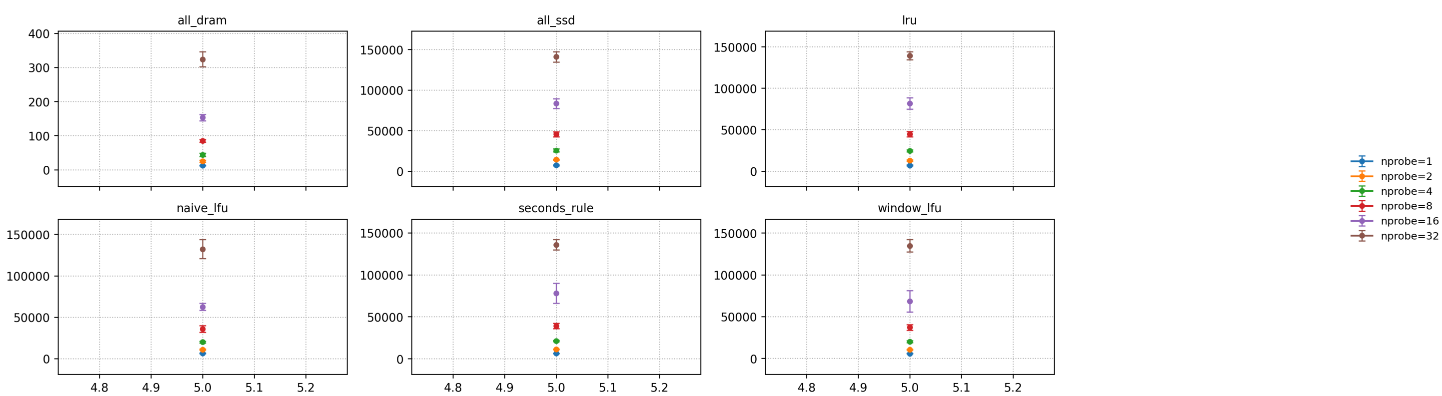
7.4.2 Figures

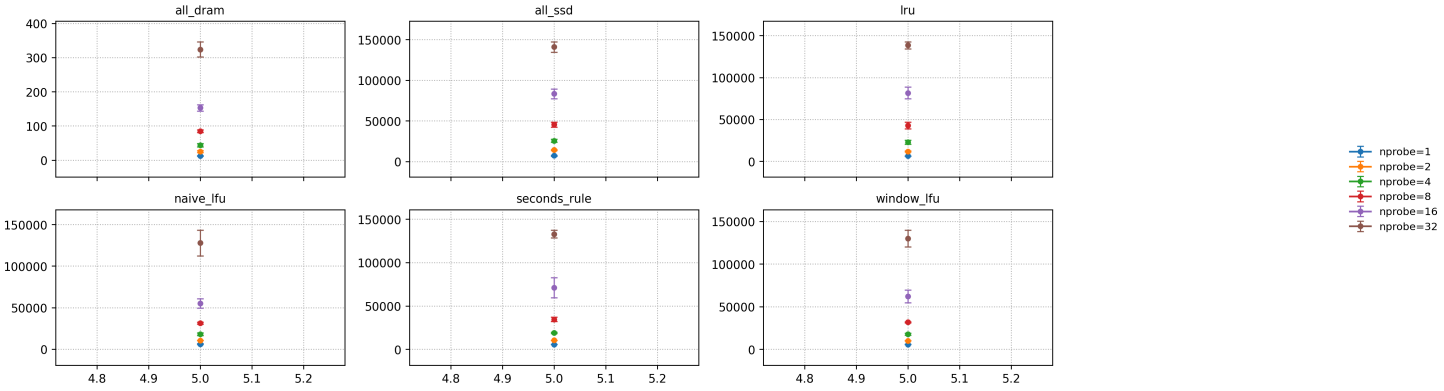


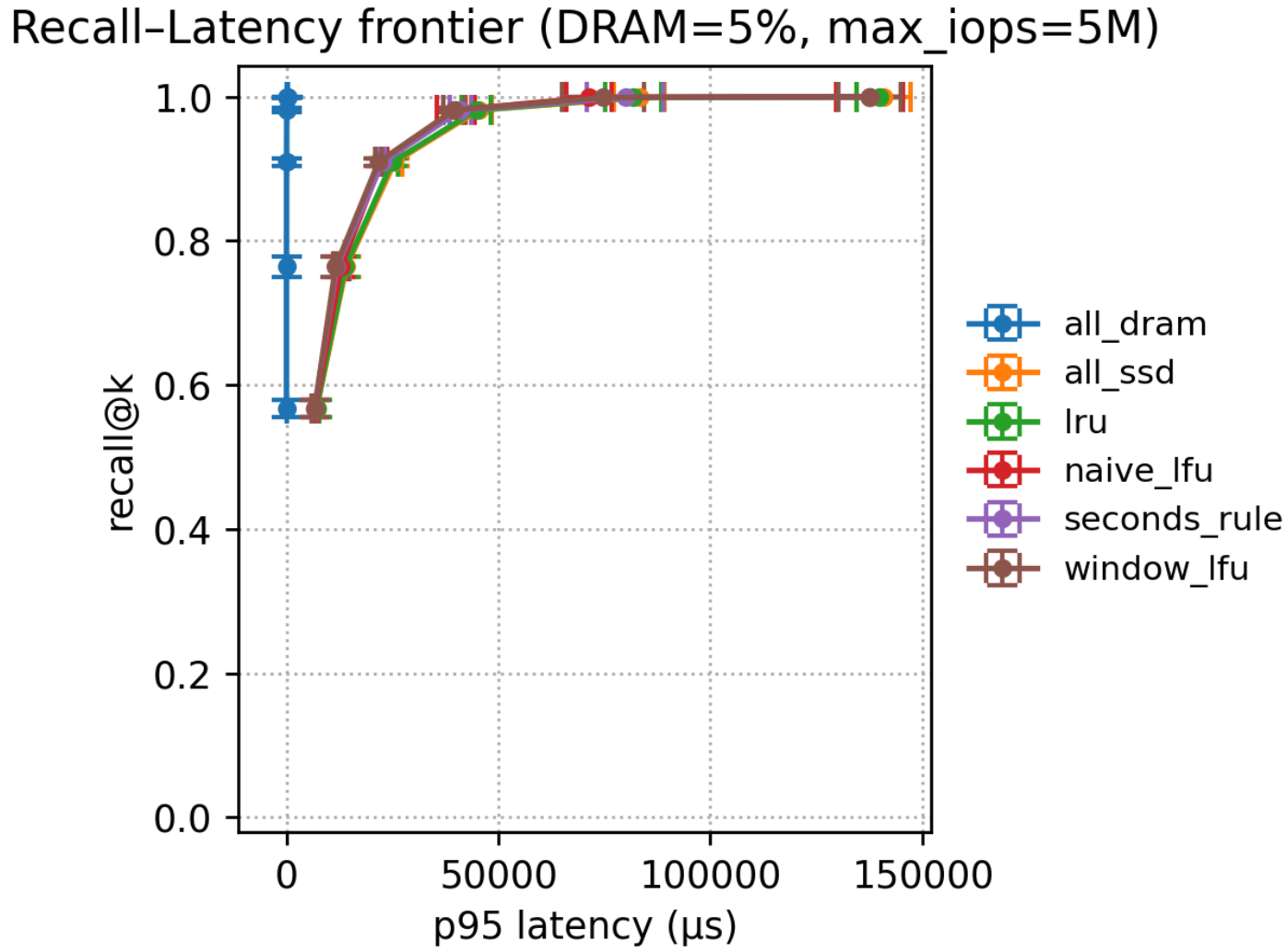

all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu
nprobe=1
nprobe=2
nprobe=4
nprobe=8
nprobe=16
nprobe=32

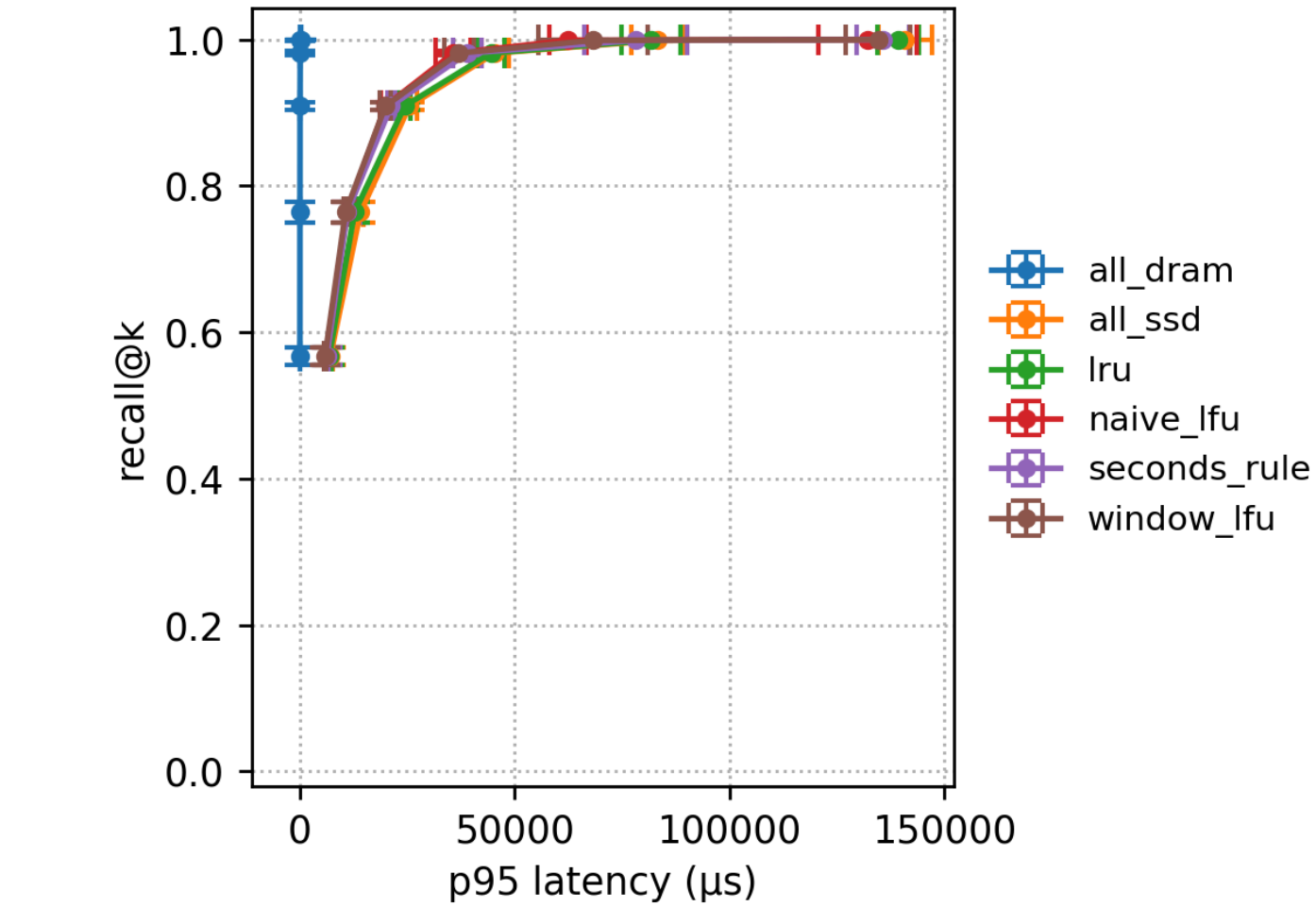

all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu
nprobe=1
nprobe=2
nprobe=4
nprobe=8
nprobe=16
nprobe=32

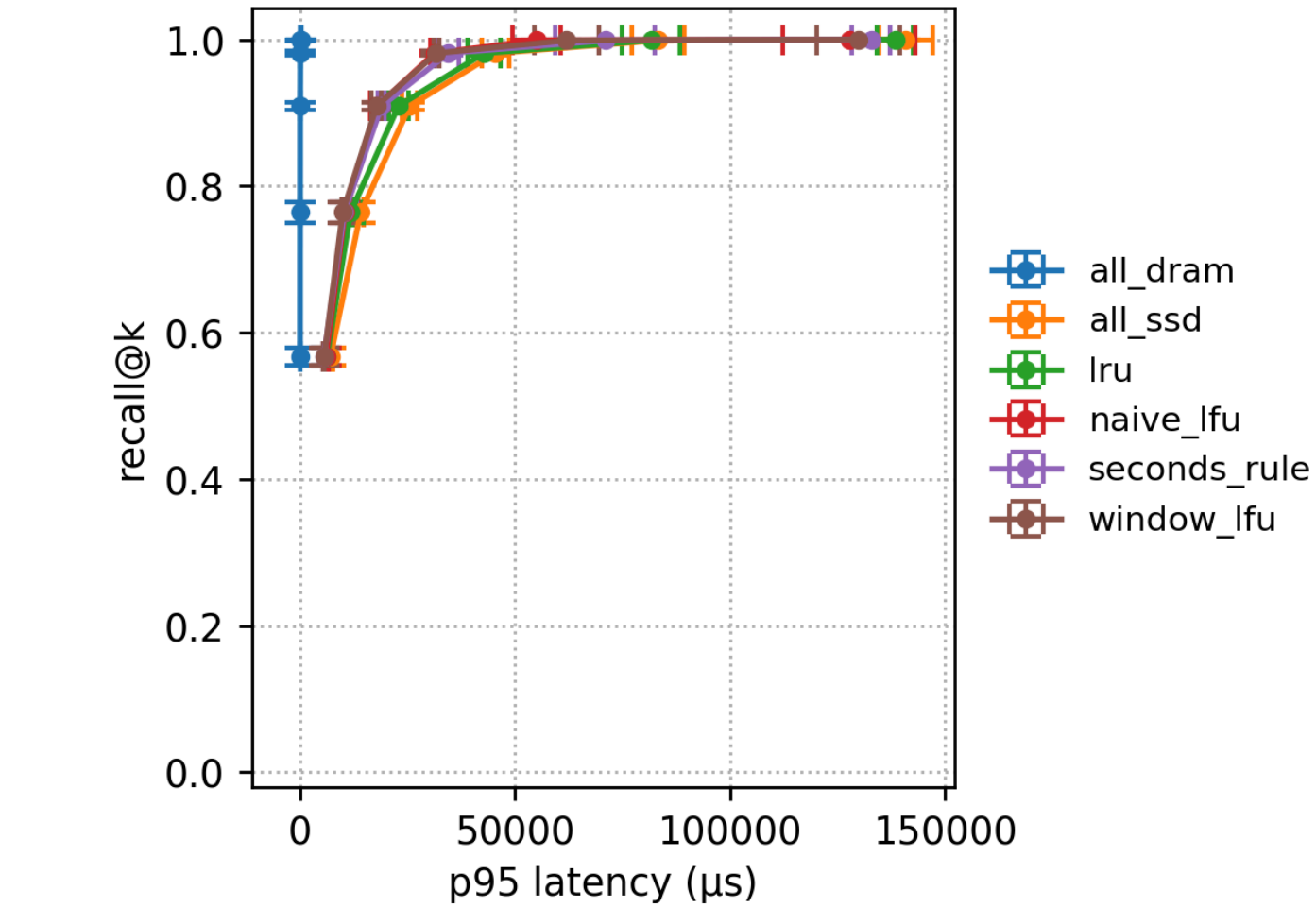

all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu
nprobe=1
nprobe=2
nprobe=4
nprobe=8
nprobe=16
nprobe=32

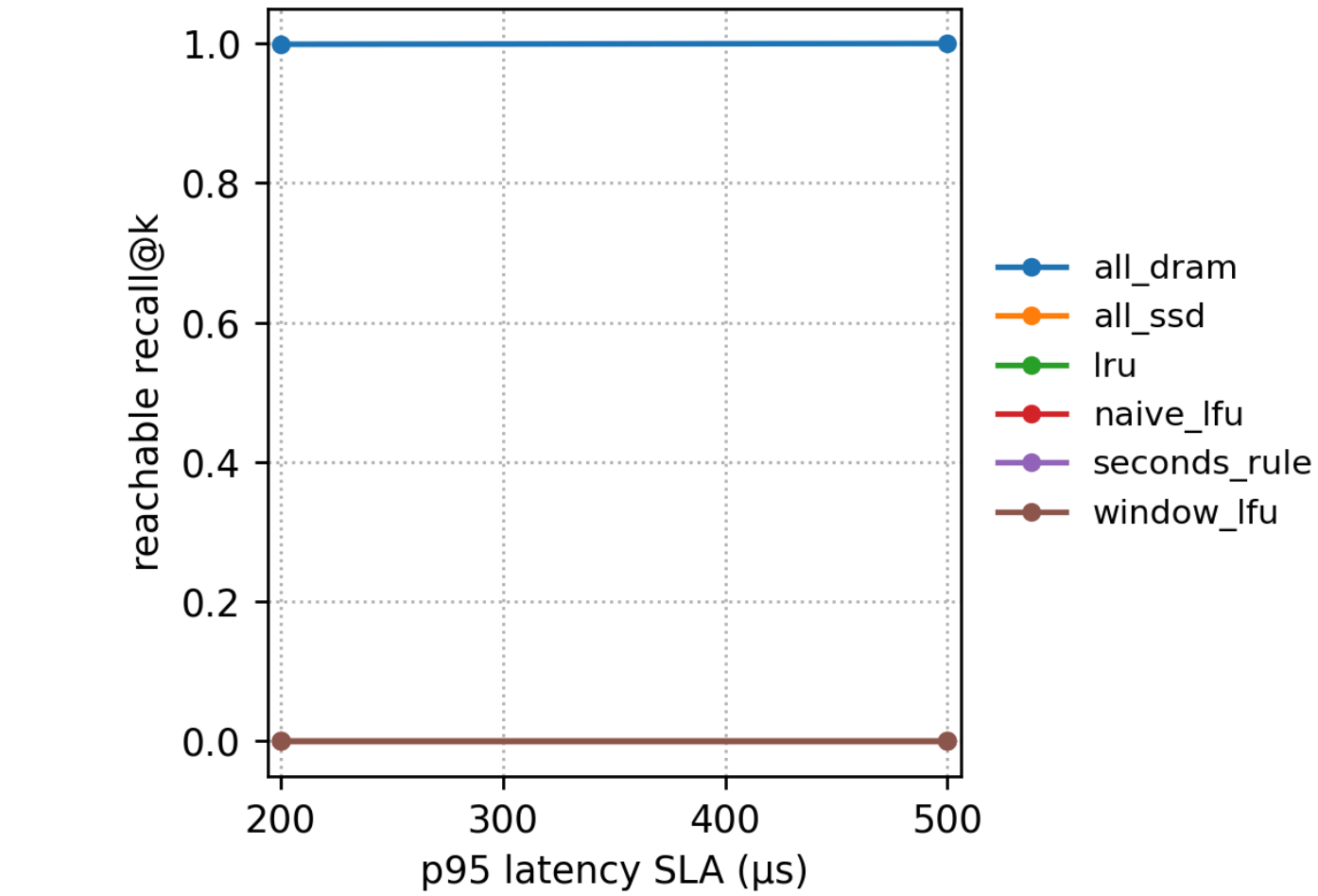

all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu
nprobe=1
nprobe=2
nprobe=4
nprobe=8
nprobe=16
nprobe=32

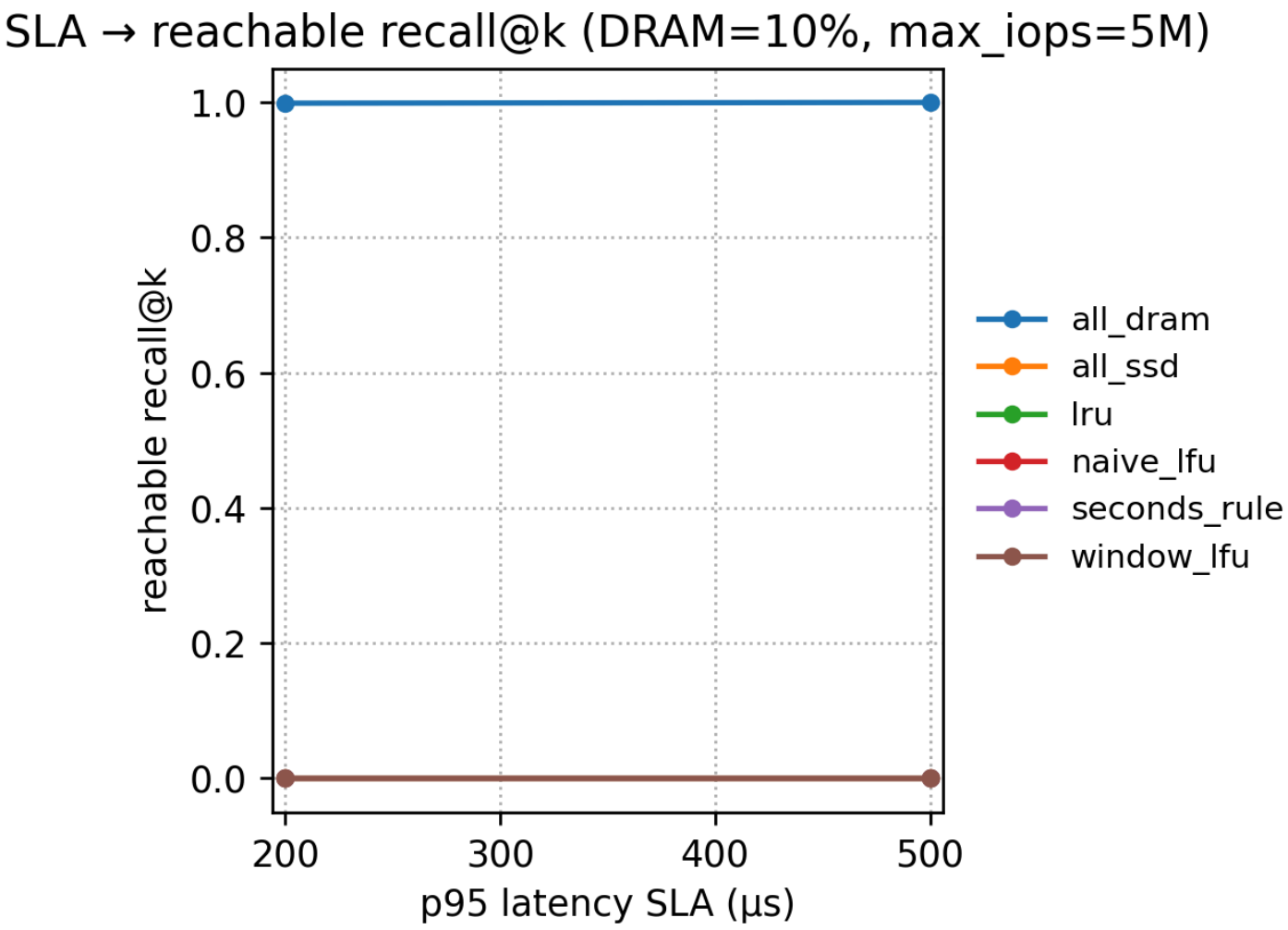

all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu
nprobe=1
nprobe=2
nprobe=4
nprobe=8
nprobe=16
nprobe=32

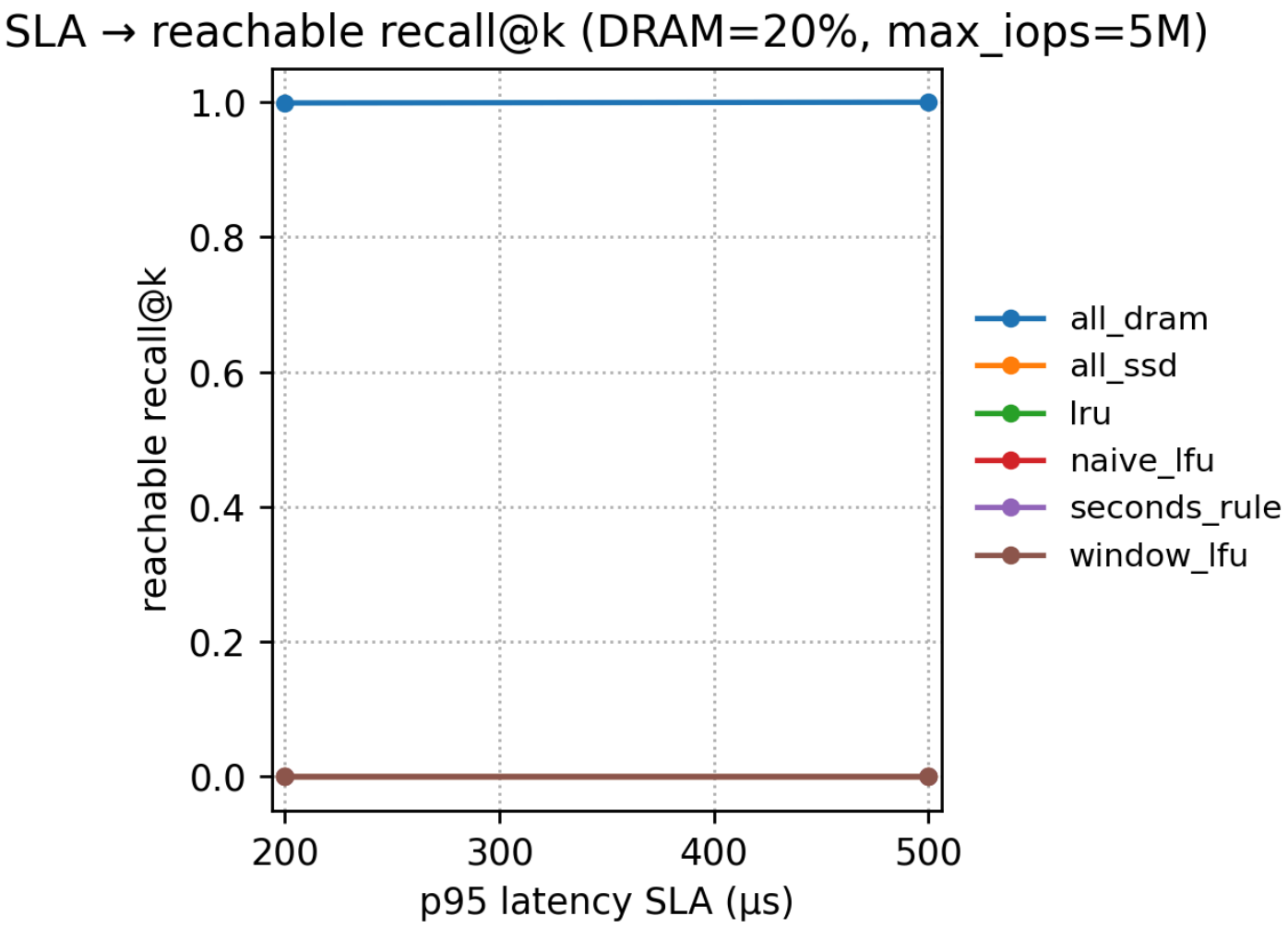

Recall–Latency frontier (DRAM=5%, max_iops=5M)
recall@k
p95 latency (µs)
all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu


Recall–Latency frontier (DRAM=10%, max_iops=5M)
recall@k
1.0
0.8
0.6
0.4
0.2
0.0
0
50000
100000
150000
p95 latency (µs)
all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu


Recall–Latency frontier (DRAM=20%, max_iops=5M)
recall@k
1.0
0.8
0.6
0.4
0.2
0.0
0
50000
100000
150000
p95 latency (µs)
all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu


SLA → reachable recall@k (DRAM=5%, max_iops=5M)
reachable recall@k
1.0
0.8
0.6
0.4
0.2
0.0
200
300
400
500
p95 latency SLA (µs)
all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu


SLA → reachable recall@k (DRAM=10%, max_iops=5M)
reachable recall@k
1.0
0.8
0.6
0.4
0.2
0.0
200
300
400
500
p95 latency SLA (µs)
all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu



7.5 Uniform workload

Run directory: `results/sr_sift_exp_uniform_fast_20251215_135028`

The core of the uniform workload is: accesses are more even and hotspots are weak. Thus “hotspot-chasing” policies naturally have a harder time gaining advantage, because true hotspots are not concentrated.

7.5.1 Key numbers (max_iops=5M, nprobe=1)

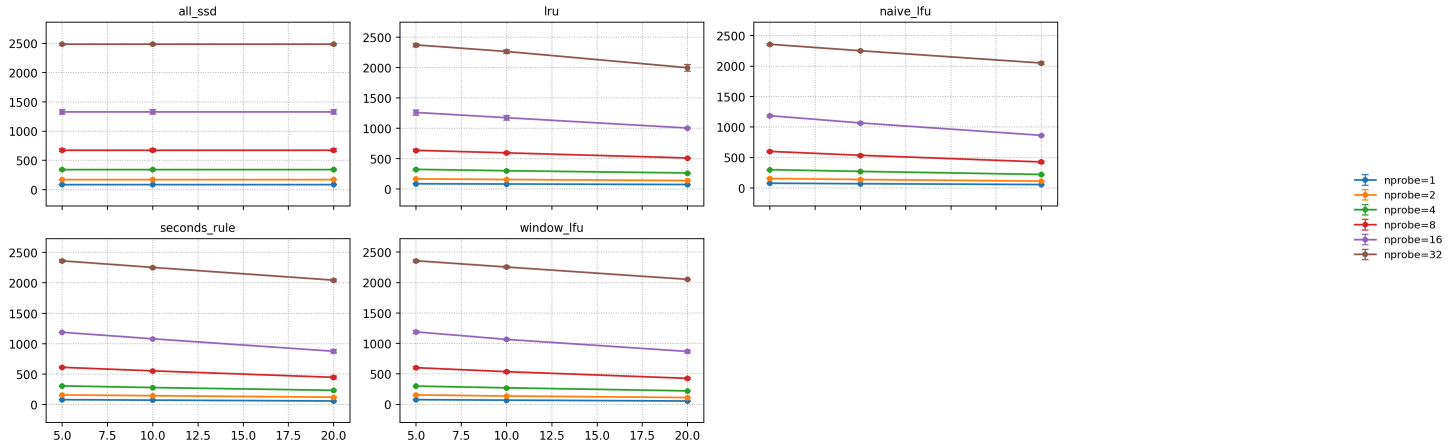
dram_fraction	policy	p95 (µs)	recall	IOAmp	total_migration_bytes
0.05	all_ssd	~3300	0.5293	~86.41	0
0.05	naive_lfu	~2910	0.5293	~74.18	~61.8 MB
0.05	window_lfu	~2910	0.5293	~74.09	~20.3 MB
0.05	seconds_rule	~3000	0.5293	~77.39	~224 MB
0.05	lru	~3280	0.5293	~81.23	~266 MB
0.10	naive_lfu	~2850	0.5293	~66.35	~81.8 MB

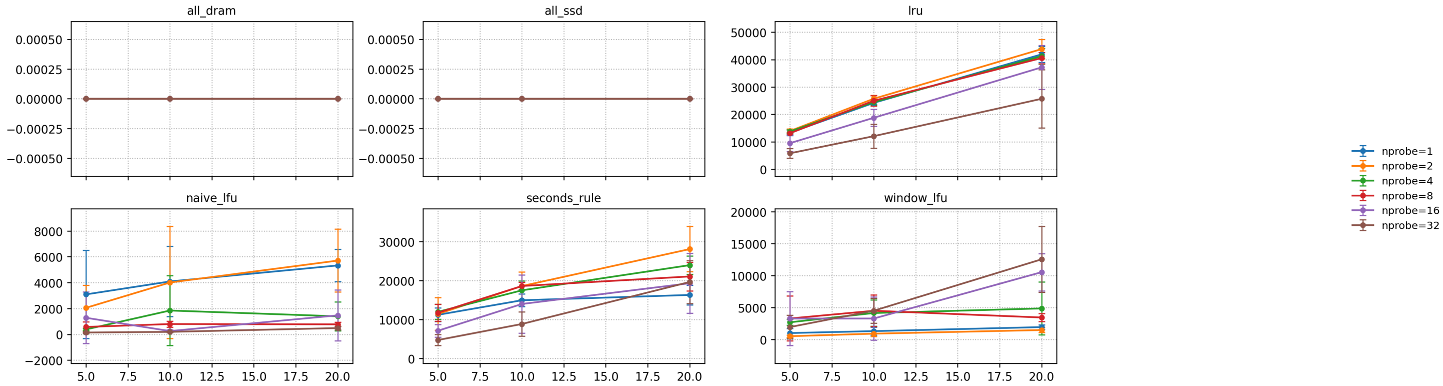
dram_fraction	policy	p95 (μs)	recall	IOAmp	total_migration_bytes
0.10	window_lfu	~2860	0.5293	~66.19	~26.4 MB
0.10	seconds_rule	~2930	0.5293	~69.73	~299 MB
0.10	lru	~3250	0.5293	~77.26	~485 MB
0.20	naive_lfu	~2790	0.5293	~52.70	~107 MB
0.20	window_lfu	~2760	0.5293	~52.32	~38.9 MB
0.20	seconds_rule	~2790	0.5293	~55.69	~327 MB
0.20	lru	~3030	0.5293	~68.75	~838 MB

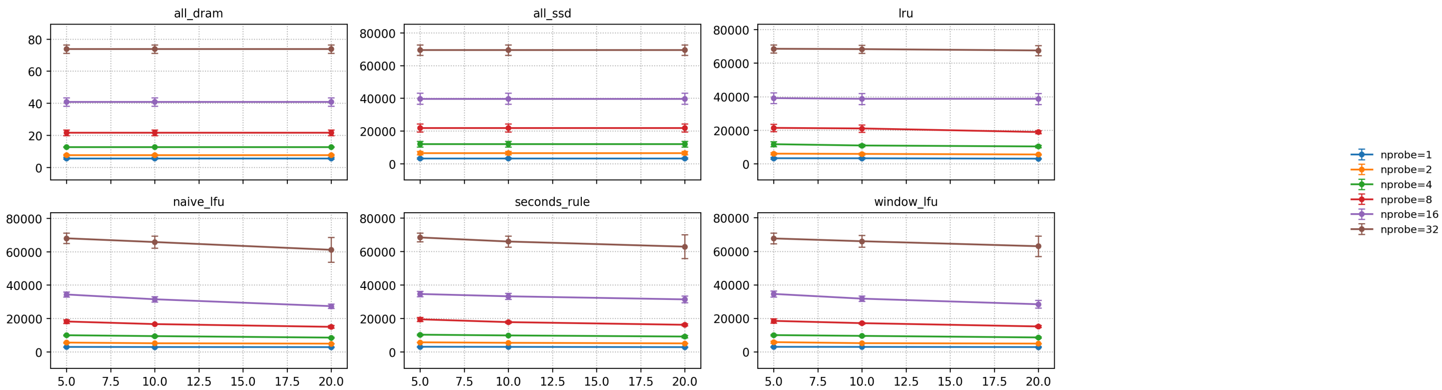
Observations and explanations

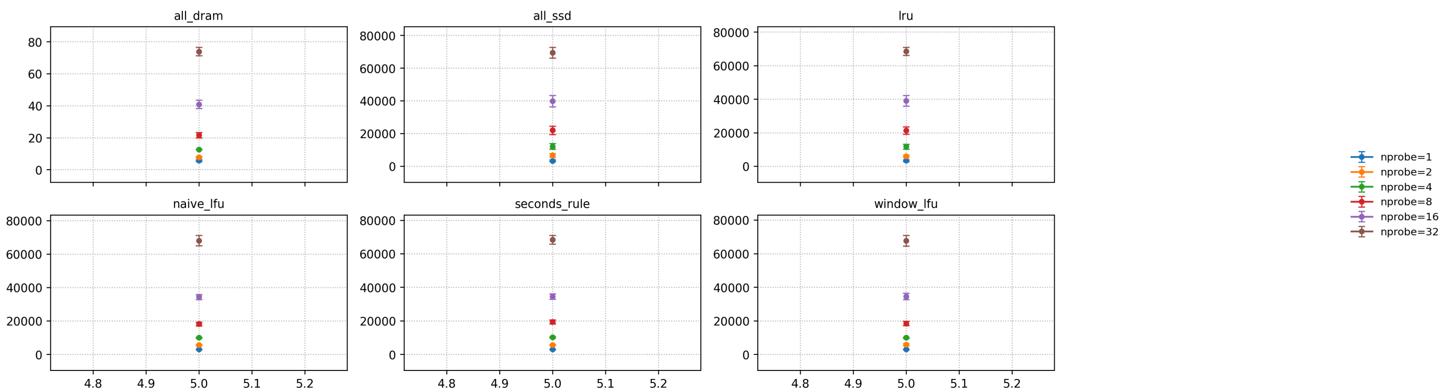
- `window_lfu` consistently shows the stable pattern of “low migration + low IOAmp + low p95” under all three DRAM configurations: in the uniform scenario, there is no intense hotspot drift, and sliding-window frequency is sufficient.
- `seconds_rule/lru` have significantly larger migration volume but no correspondingly lower IOAmp: when hotspots are weak, “chasing hotspots” easily becomes “chasing noise”, and the net benefit of movement is limited.
- As DRAM grows, IOAmp clearly decreases (from 86 → about 52) , and p95 drops from ~3.30 ms to ~2.76 ms: this shows that increasing DRAM steadily yields benefits, but is still limited by the “portion that inevitably falls on SSD due to uniform access”.

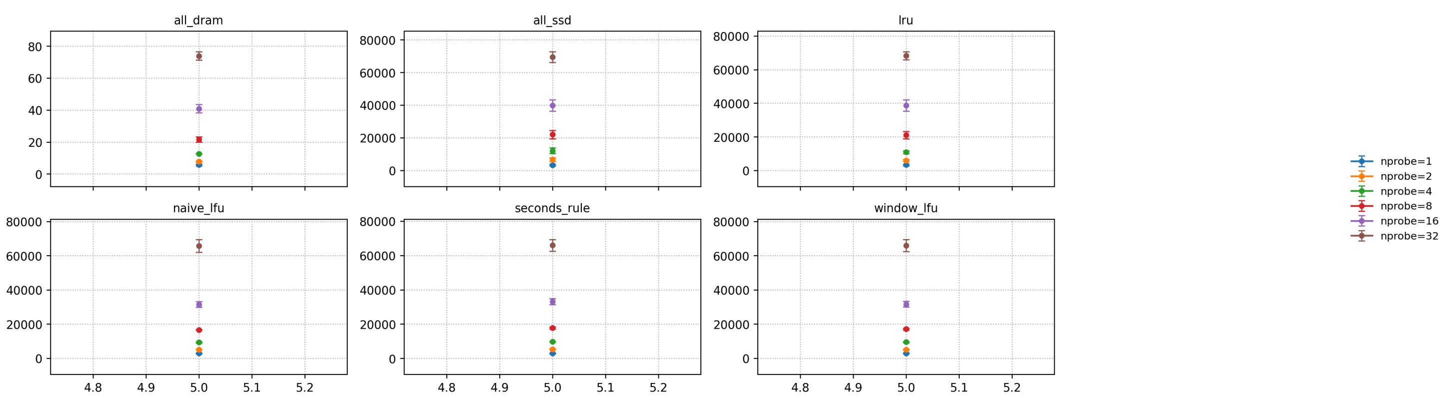
7.5.2 Figures

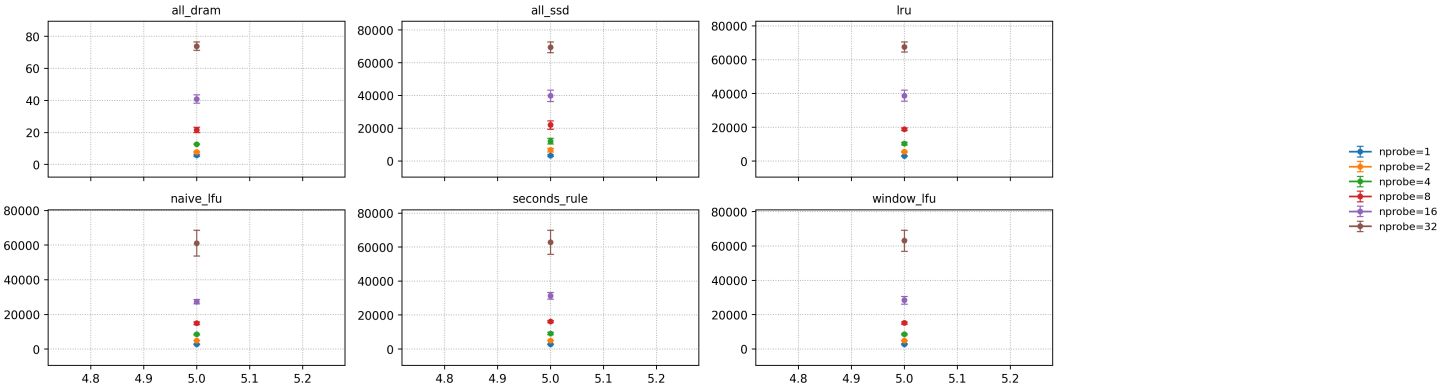


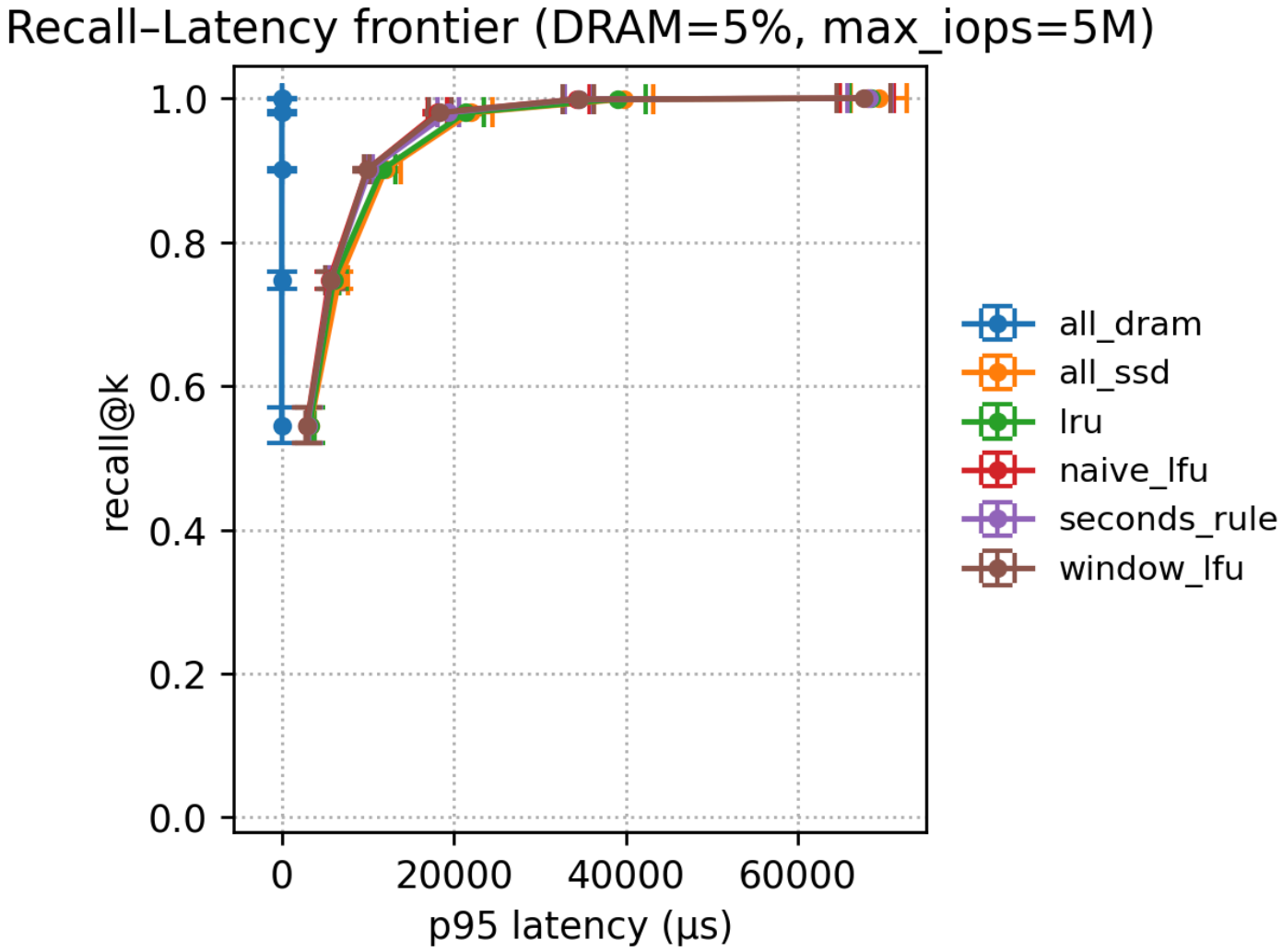

all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu
nprobe=1
nprobe=2
nprobe=4
nprobe=8
nprobe=16
nprobe=32

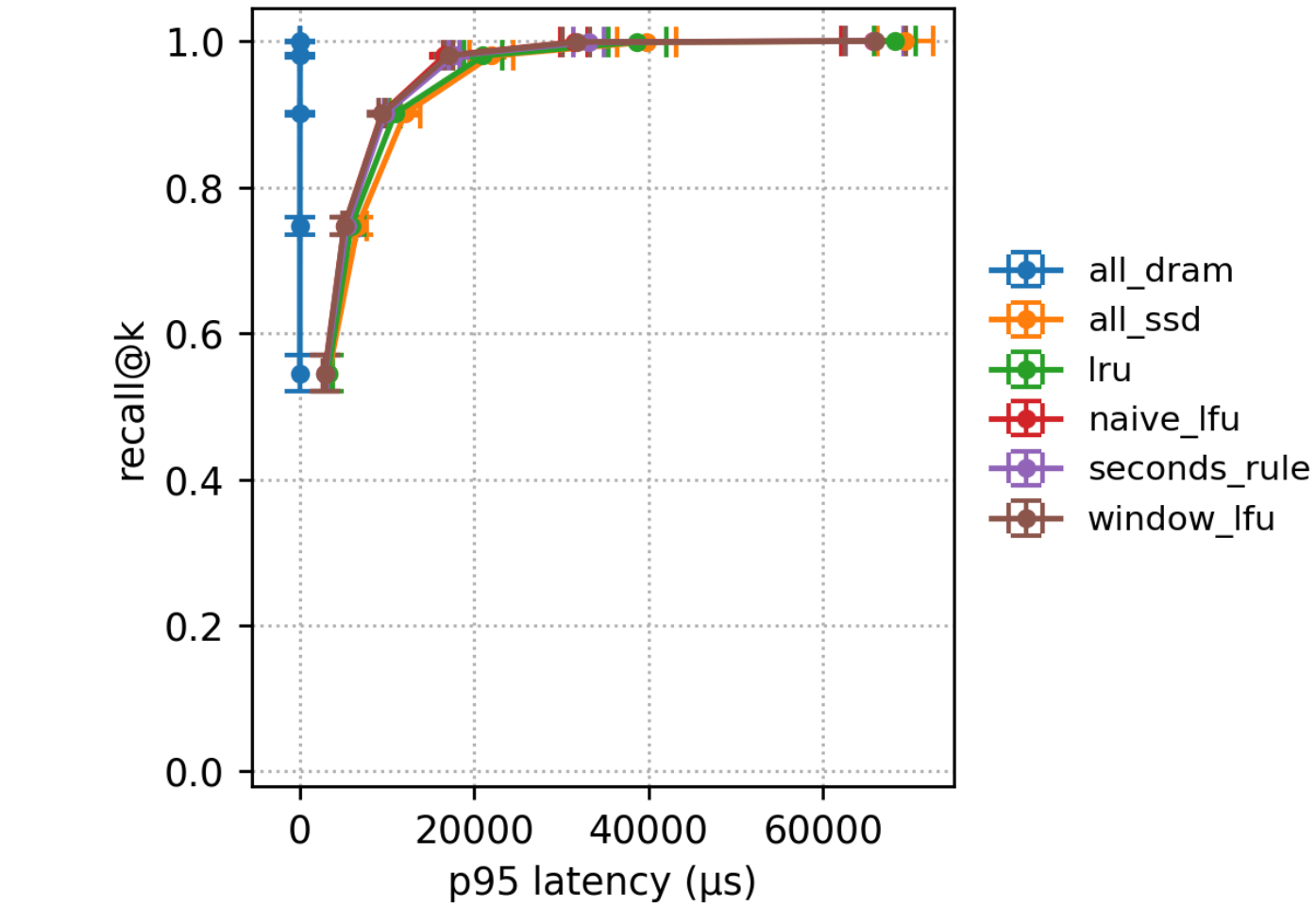

all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu
nprobe=1
nprobe=2
nprobe=4
nprobe=8
nprobe=16
nprobe=32

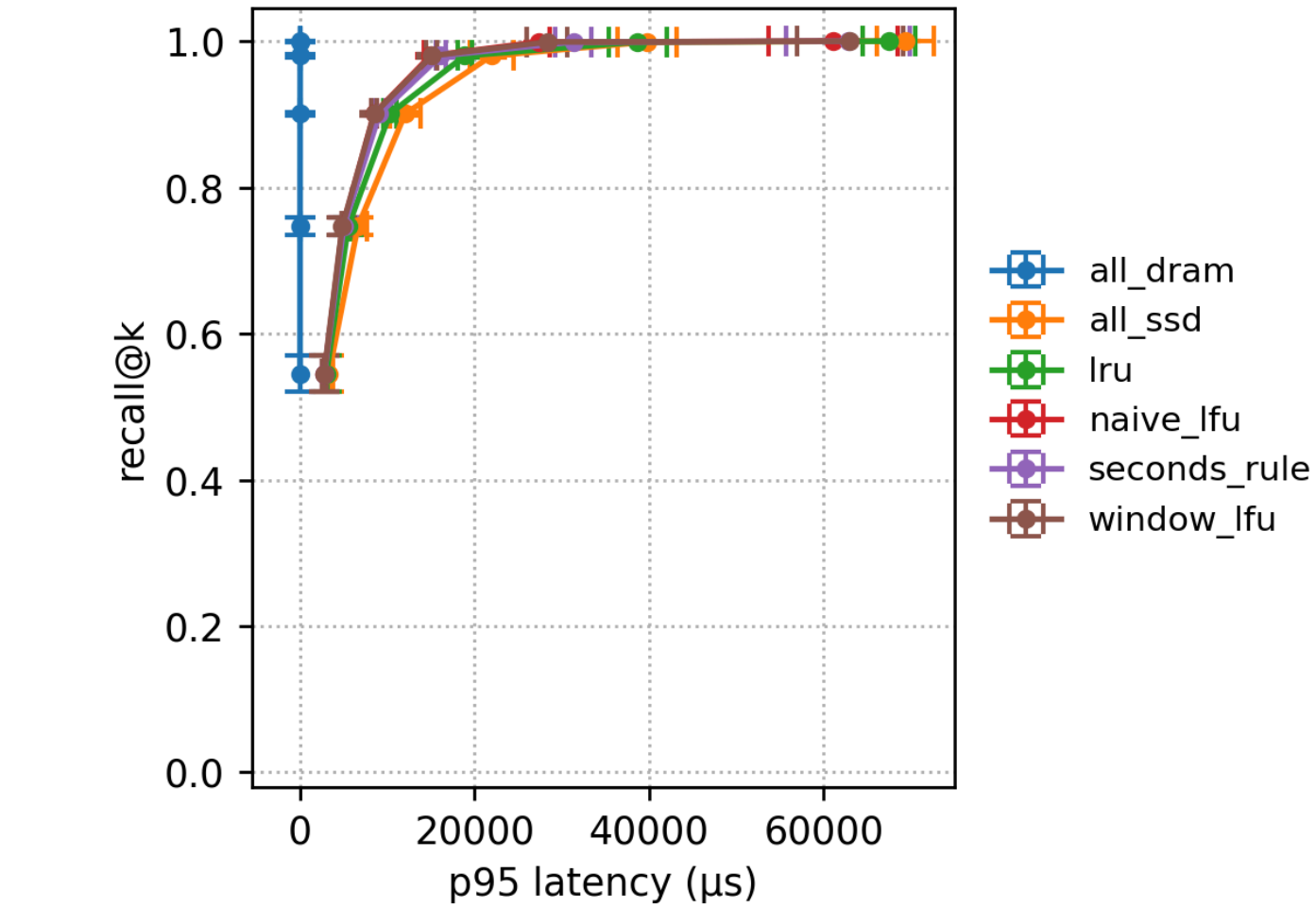

all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu
nprobe=1
nprobe=2
nprobe=4
nprobe=8
nprobe=16
nprobe=32

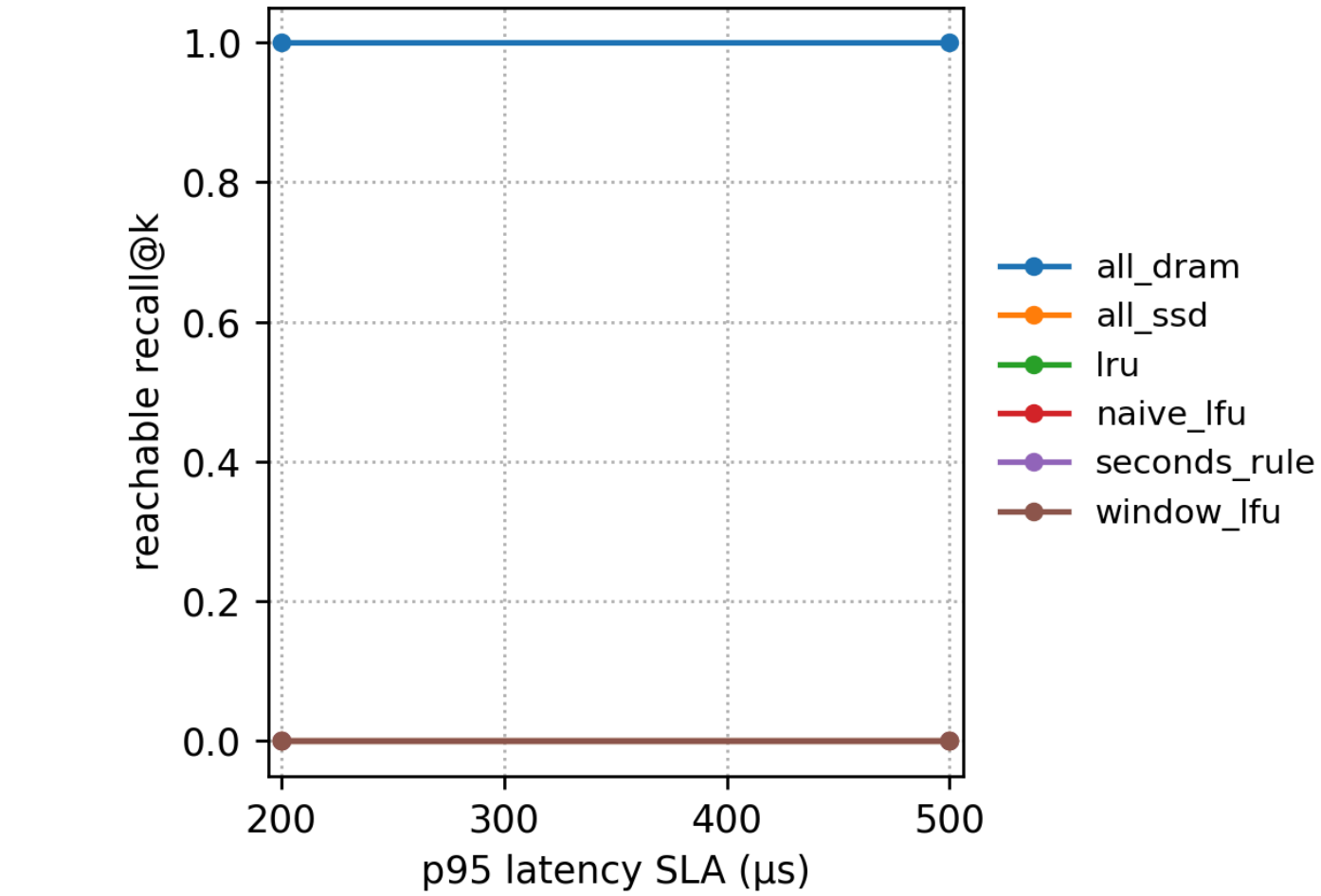

all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu
nprobe=1
nprobe=2
nprobe=4
nprobe=8
nprobe=16
nprobe=32

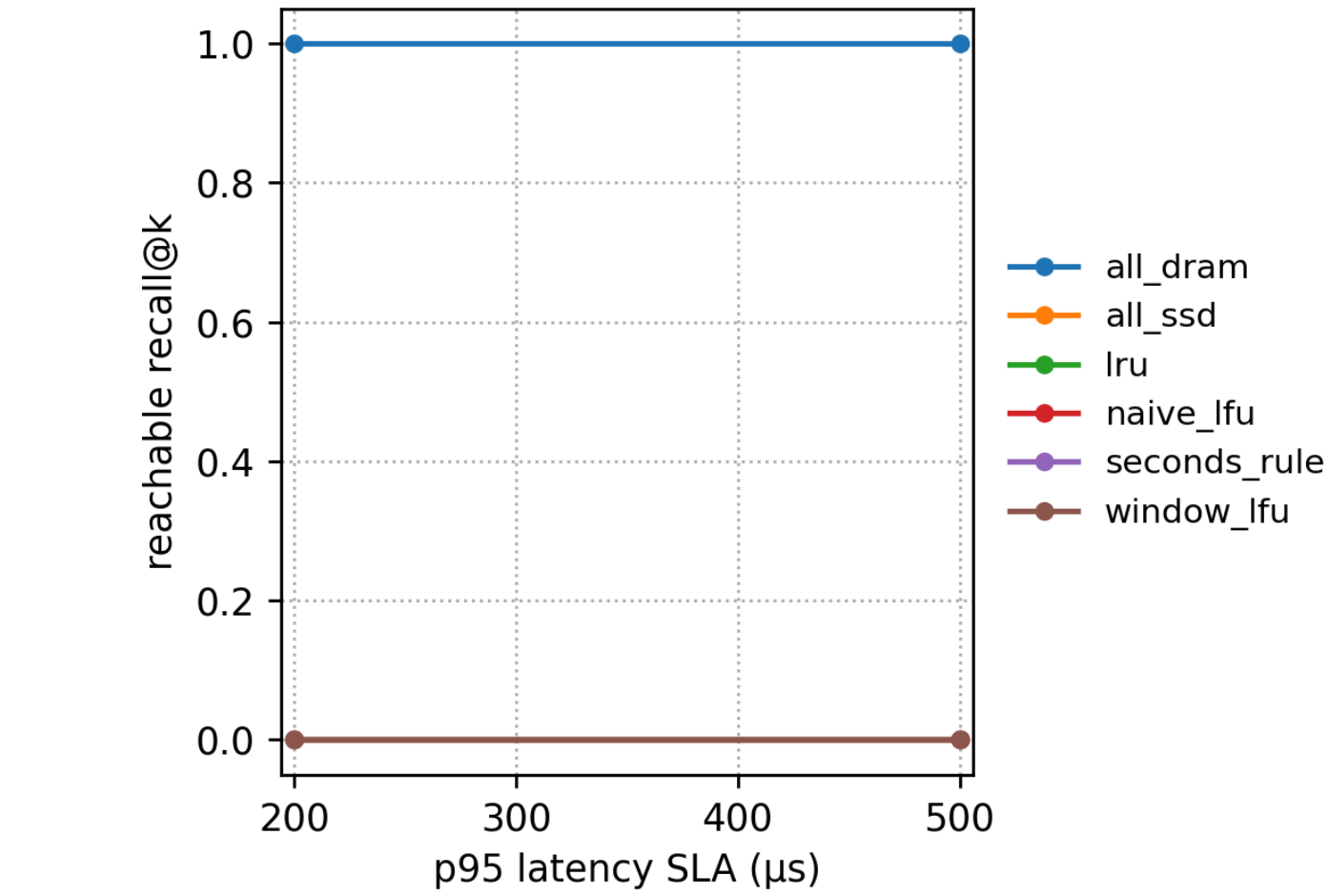

all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu
nprobe=1
nprobe=2
nprobe=4
nprobe=8
nprobe=16
nprobe=32

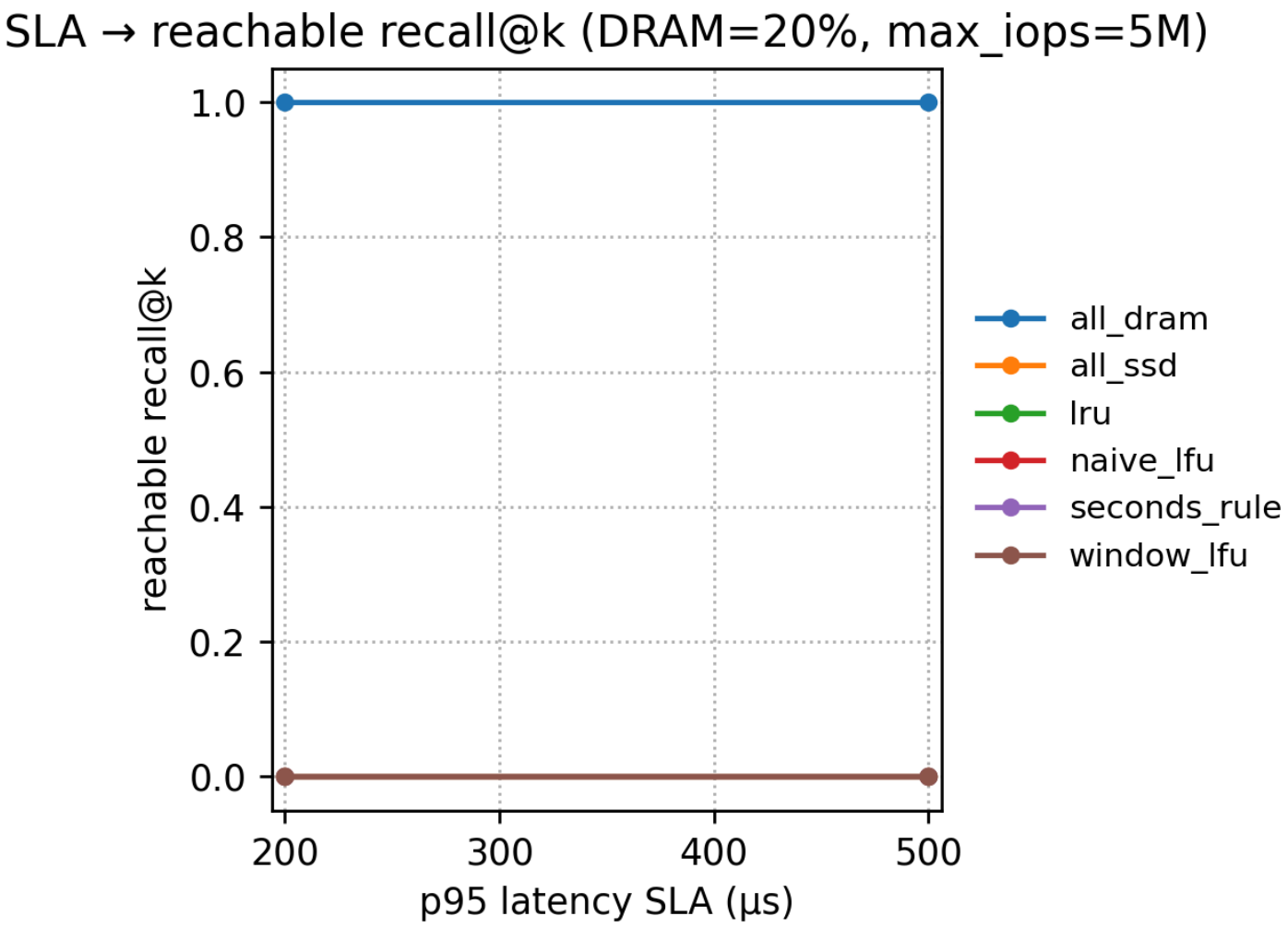

Recall–Latency frontier (DRAM=5%, max_iops=5M)
recall@k
p95 latency (µs)
all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu


Recall–Latency frontier (DRAM=10%, max_iops=5M)
recall@k
1.0
0.8
0.6
0.4
0.2
0.0
0
20000
40000
60000
p95 latency (µs)
all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu


Recall–Latency frontier (DRAM=20%, max_iops=5M)
recall@k
1.0
0.8
0.6
0.4
0.2
0.0
0
20000
40000
60000
p95 latency (µs)
all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu


SLA → reachable recall@k (DRAM=5%, max_iops=5M)
reachable recall@k
1.0
0.8
0.6
0.4
0.2
0.0
200
300
400
500
p95 latency SLA (µs)
all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu


SLA → reachable recall@k (DRAM=10%, max_iops=5M)
reachable recall@k
1.0
0.8
0.6
0.4
0.2
0.0
200
300
400
500
p95 latency SLA (µs)
all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu



7.6 Zipf workload (s = 1.2)

Run directory: `results/sr_sift_exp_zipf_s1.2_fast_20251215_135028`

The core of the Zipf workload is a strong long tail: a small number of clusters are very hot, and most clusters are very cold. This usually makes “frequency-based policies” easily obtain high hit benefits with small cost.

7.6.1 Key numbers (max_iops=5M, nprobe=1)

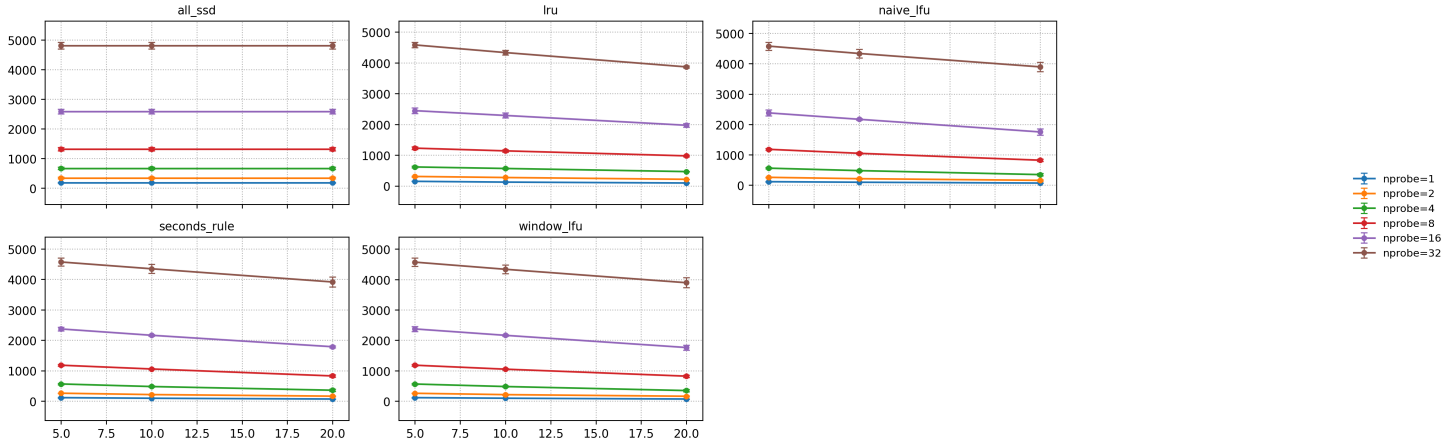
dram_fraction	policy	p95 (µs)	recall	IOAmp	total_migration_bytes
0.05	all_ssd	~7650	0.5293	~177.20	0
0.05	naive_lfu	~6730	0.5293	~112.74	~9.1 MB
0.05	window_lfu	~6730	0.5293	~112.74	~9.1 MB
0.05	seconds_rule	~6830	0.5293	~113.77	~73.9 MB
0.05	lru	~7650	0.5293	~147.86	~476 MB
0.10	naive_lfu	~6310	0.5293	~94.49	~19.6 MB

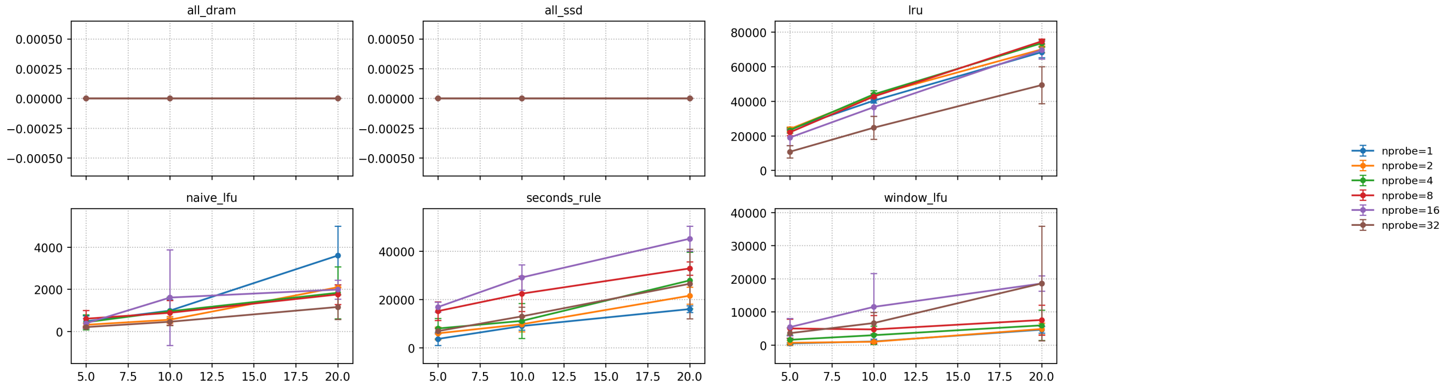
dram_fraction	policy	p95 (μs)	recall	IOAmp	total_migration_bytes
0.10	window_lfu	~6070	0.5293	~93.52	~21.5 MB
0.10	seconds_rule	~6340	0.5293	~94.07	~181 MB
0.10	lru	~6990	0.5293	~122.87	~807 MB
0.20	naive_lfu	~5590	0.5293	~67.47	~71.9 MB
0.20	window_lfu	~5590	0.5293	~67.25	~92.6 MB
0.20	seconds_rule	~5670	0.5293	~68.52	~321 MB
0.20	lru	~6560	0.5293	~93.72	~1.37 GB

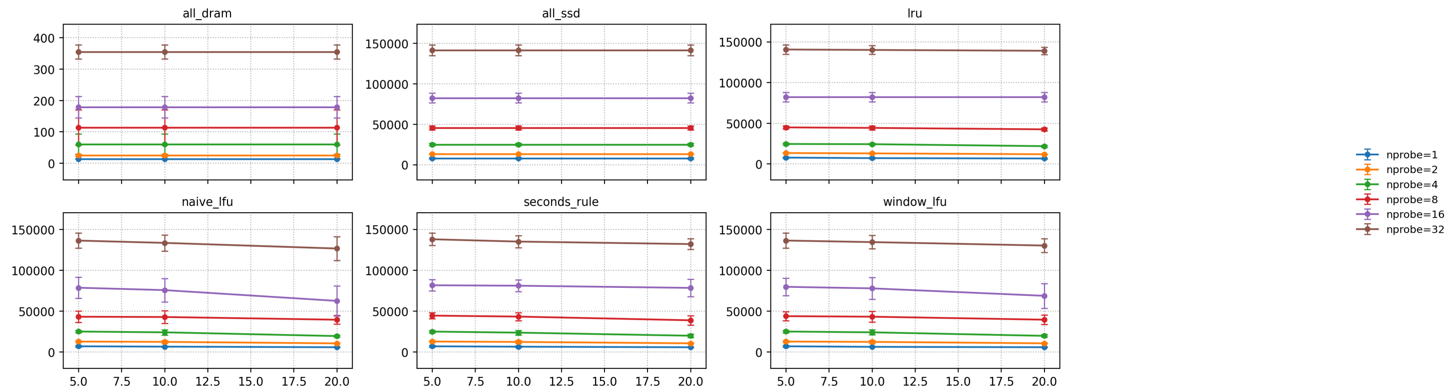
Observations and explanations

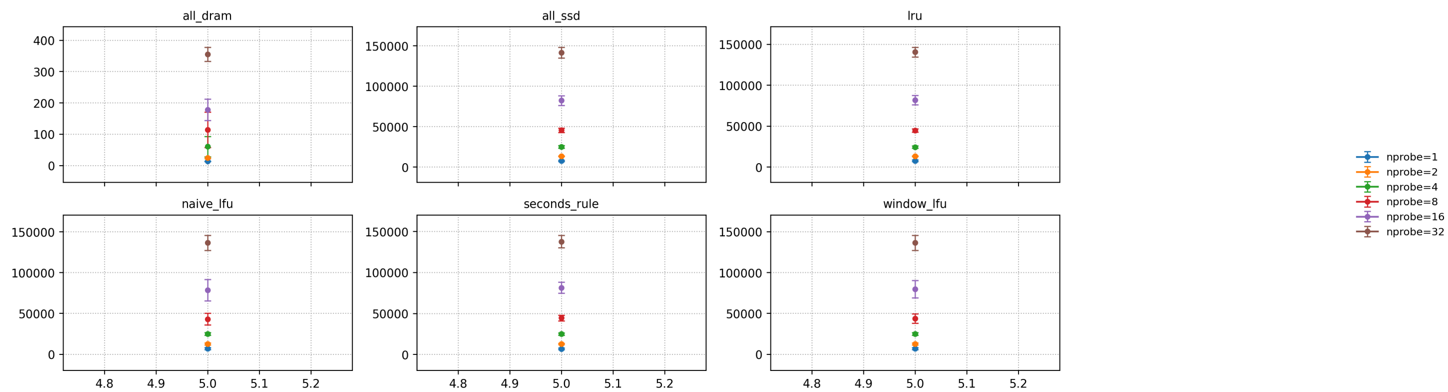
- Under dram=0.05, `naive_lfu/window_lfu` can already push IOAmp from ~177 down to ~113, while migration is only ~9 MB: Zipf's hotspots are extremely stable and concentrated, so frequency statistics easily "hit the correct objects long-term" and almost no frequent movement is needed.
- `seconds_rule` has IOAmp close to window_lfu but larger migration: this shows that in the Zipf scenario, "chasing hotspots in time" does not bring extra hit advantage, but instead introduces more movement.
- `lru` has large migration but does not lead in IOAmp: in a strong long-tail with stable hotspots, LRU is more likely to continually swap in short-term noisy accesses, causing unnecessary movement.

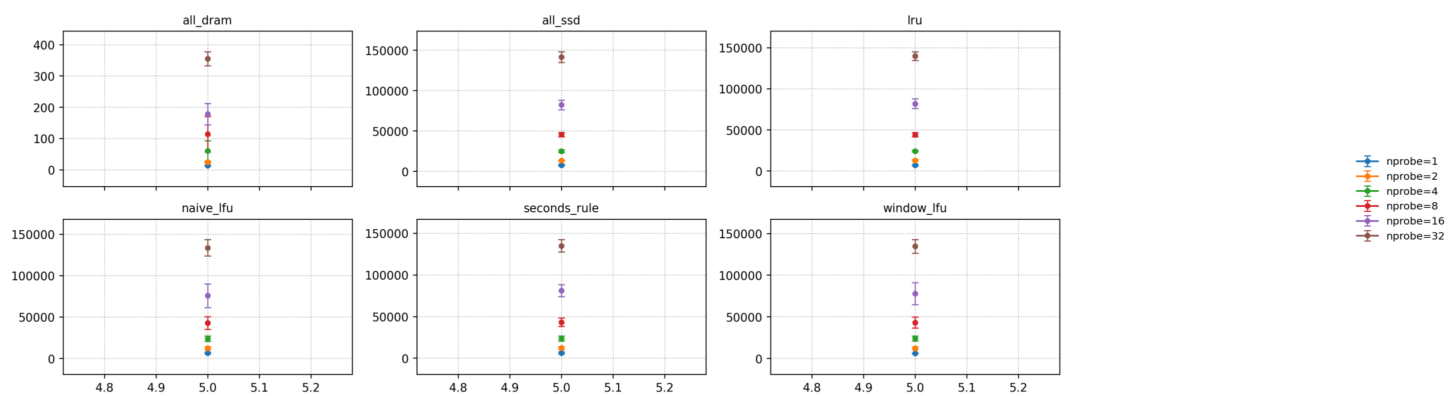
7.6.2 Figures

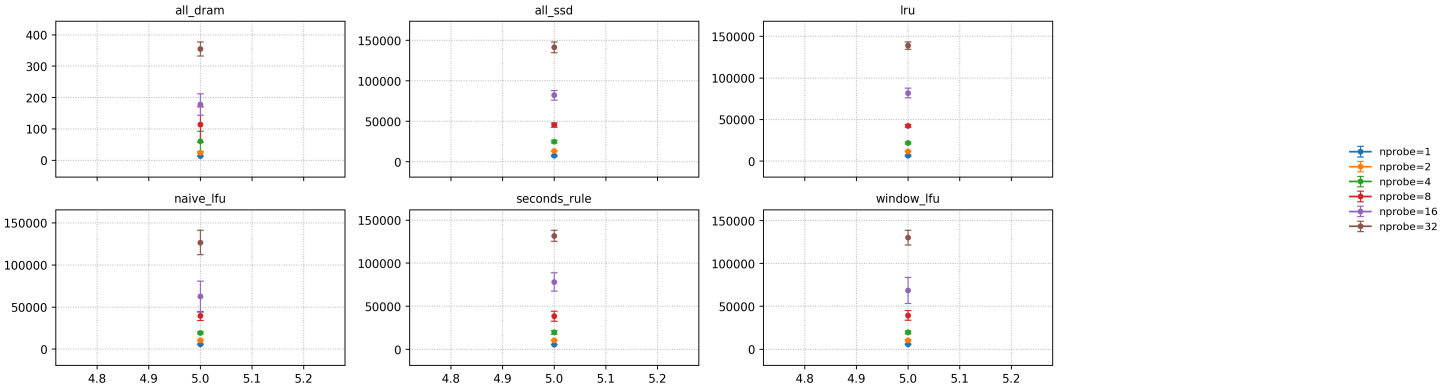


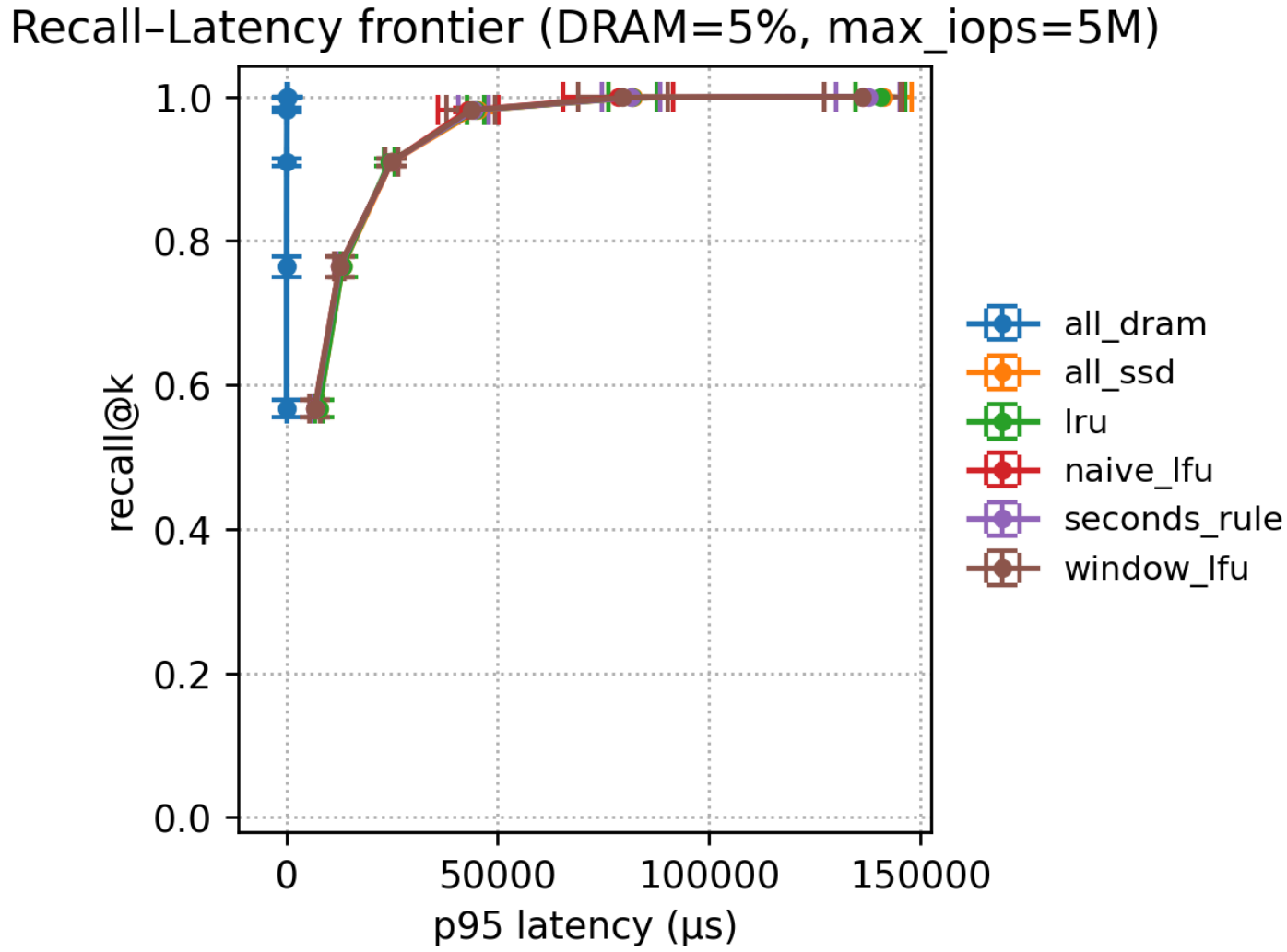

all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu
nprobe=1
nprobe=2
nprobe=4
nprobe=8
nprobe=16
nprobe=32

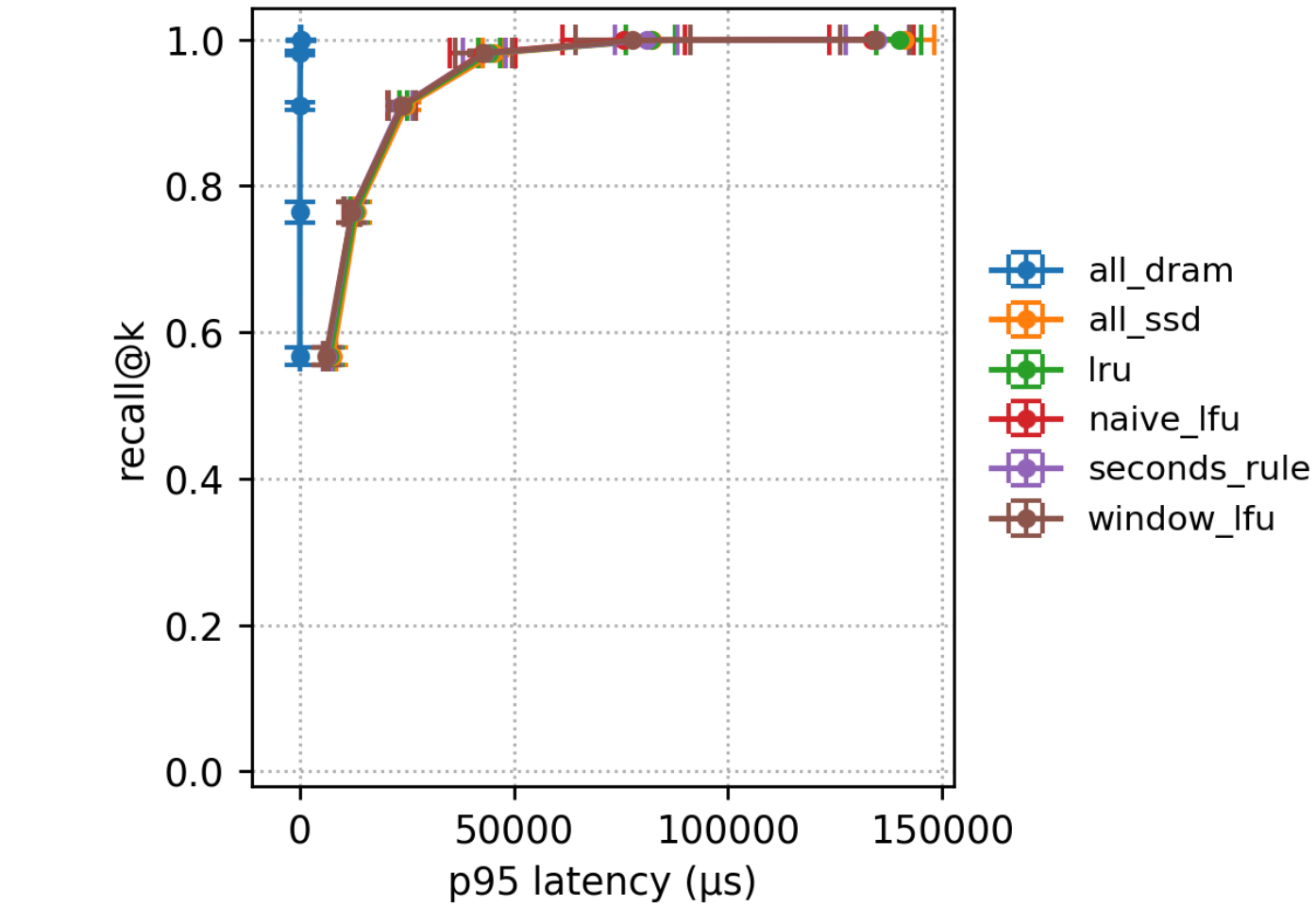

all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu
nprobe=1
nprobe=2
nprobe=4
nprobe=8
nprobe=16
nprobe=32

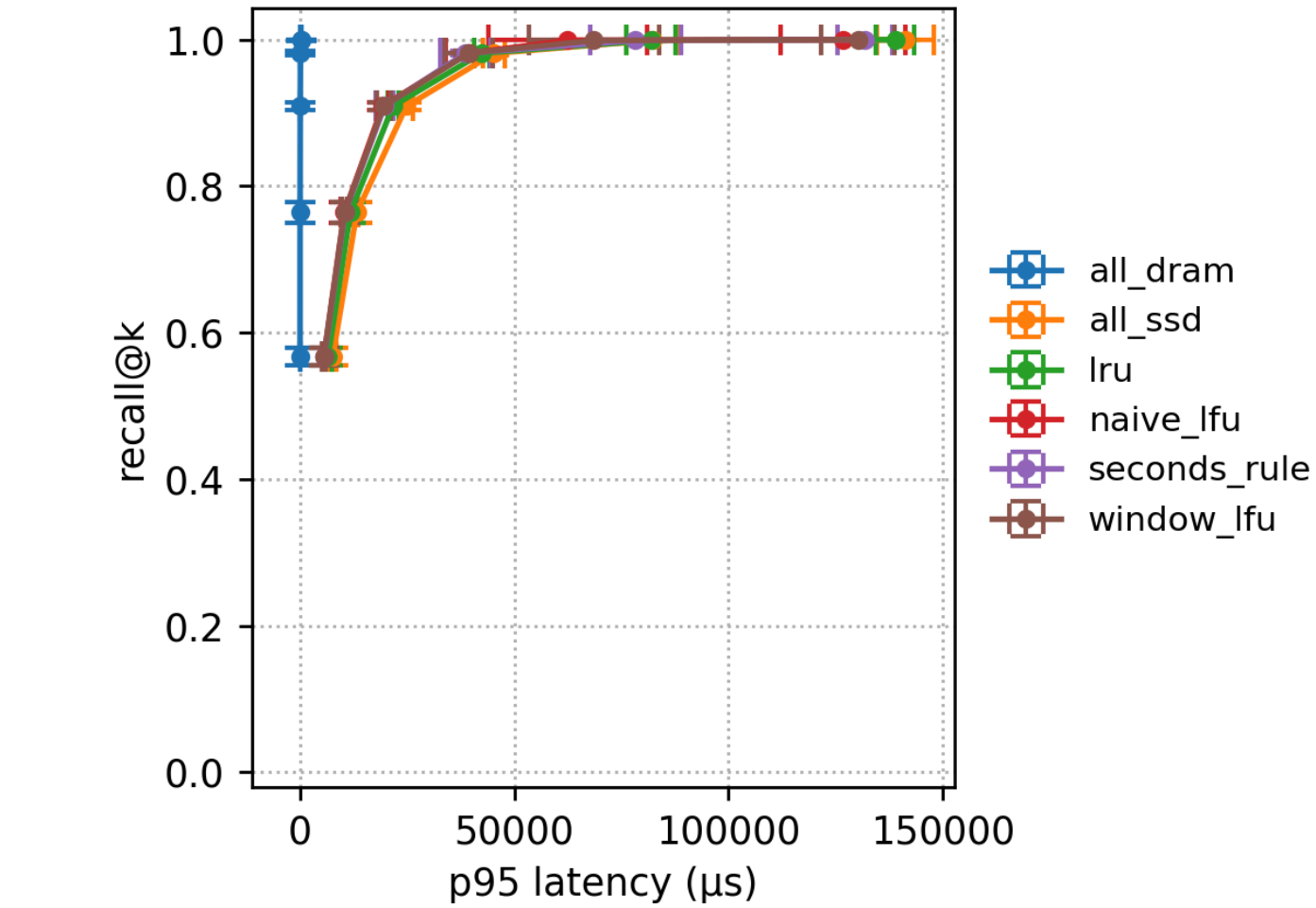

all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu
nprobe=1
nprobe=2
nprobe=4
nprobe=8
nprobe=16
nprobe=32

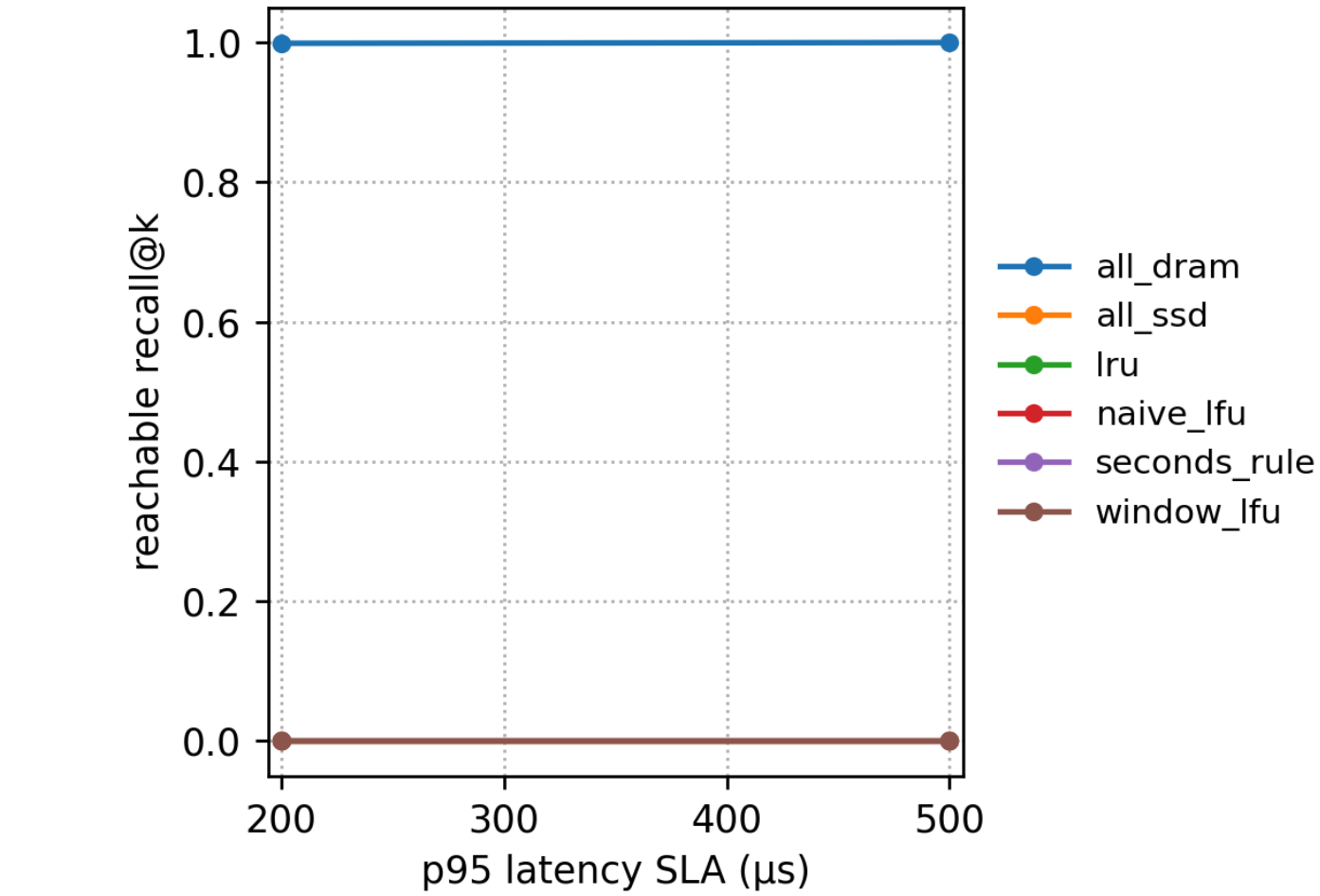

all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu
nprobe=1
nprobe=2
nprobe=4
nprobe=8
nprobe=16
nprobe=32

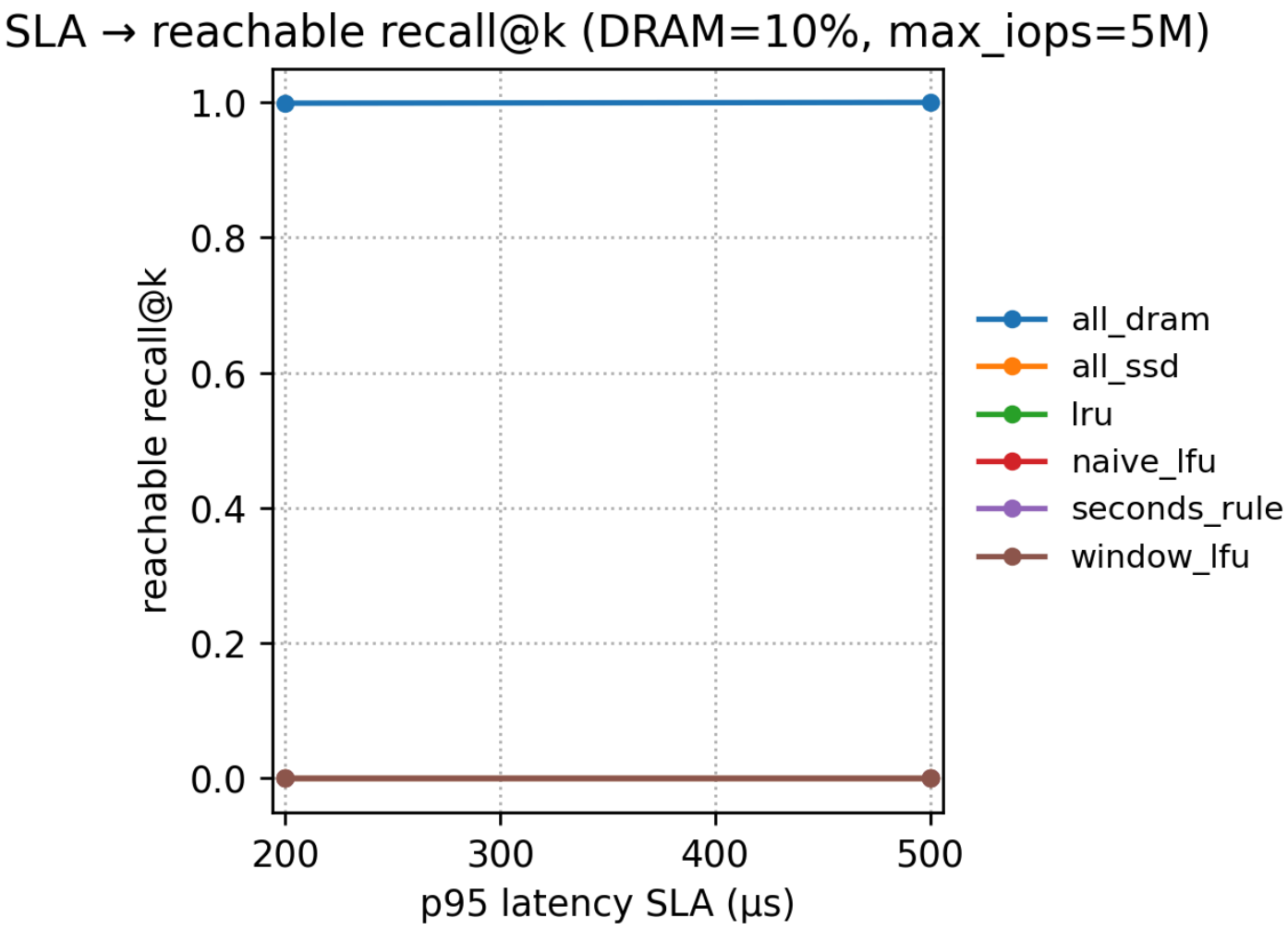

all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu
nprobe=1
nprobe=2
nprobe=4
nprobe=8
nprobe=16
nprobe=32

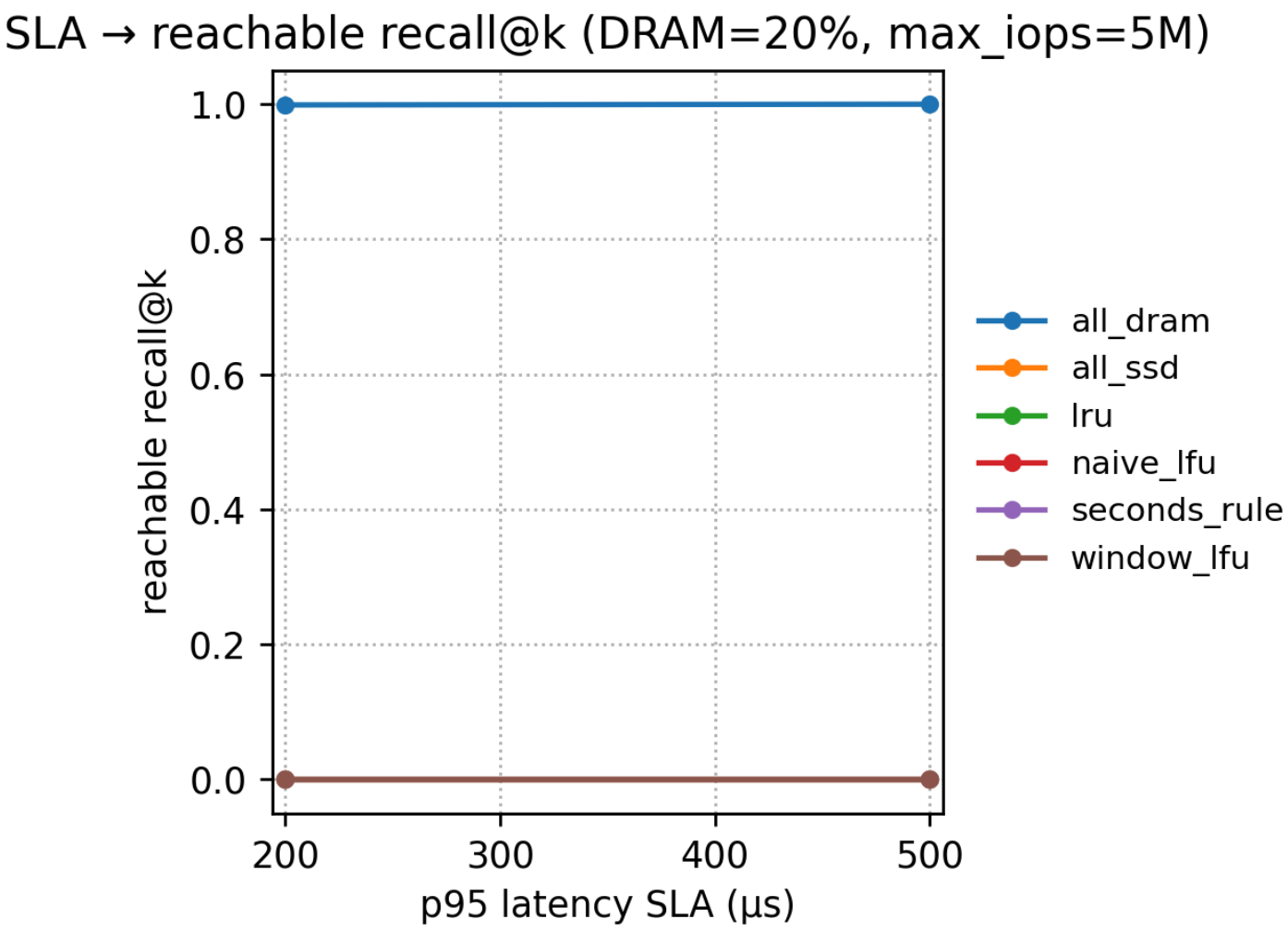

Recall–Latency frontier (DRAM=5%, max_iops=5M)
recall@k
p95 latency (µs)
all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu


Recall–Latency frontier (DRAM=10%, max_iops=5M)
recall@k
1.0
0.8
0.6
0.4
0.2
0.0
0
50000
100000
150000
p95 latency (µs)
all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu


Recall–Latency frontier (DRAM=20%, max_iops=5M)
recall@k
1.0
0.8
0.6
0.4
0.2
0.0
0
50000
100000
150000
p95 latency (µs)
all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu


SLA → reachable recall@k (DRAM=5%, max_iops=5M)
reachable recall@k
1.0
0.8
0.6
0.4
0.2
0.0
200
300
400
500
p95 latency SLA (µs)
all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu


SLA → reachable recall@k (DRAM=10%, max_iops=5M)
reachable recall@k
1.0
0.8
0.6
0.4
0.2
0.0
200
300
400
500
p95 latency SLA (µs)
all_dram
all_ssd
lru
naive_lfu
seconds_rule
window_lfu



7.7 Summary of findings across workloads

Looking at the results of each workload together, we can obtain three stable conclusions (without any "historical comparison", only summarizing this set of results itself) :

- The path of moving nprobe from 1→32 is clear: higher recall requires scanning more lists, so latency must increase**. The frontier plots clearly show this.
- SSD tail latency is mainly driven by IOAmp/page count: policies (or larger DRAM fractions) that significantly reduce IOAmp will significantly reduce p95; otherwise p95 stays at the ms level.
- Policy choice depends on hotspot stability:
  - Fast-changing hotspots: `seconds_rule/lru` can push IOAmp lower, but migration volume reaches the GB level;
  - Slow-changing or stable long-tail hotspots: `window_lfu/naive_lfu` often achieve comparable or even better IOAmp and p95 with smaller migration;
  - Nearly uniform access: hotspot-chasing policies tend to "chase noise", with large migration but limited hit benefits.

8. Discussion

8.1 Why SSD dominates p95

In the current page-level I/O model, a single SSD page access costs about 20.2 microseconds (when max IOPS = 5M) . If we ignore CPU computation, we can approximate p95 as

$$\mathrm{p95} \approx t_{\mathrm{page}} \times \mathrm{pages} + \mathrm{CPU}_{\mathrm{overhead}}$$

where `pages` is proportional to `io_amp` (up to a constant scaling) , and `CPU_overhead` is on the order of tens of microseconds under all DRAM. From the SLA we can back out the number of allowable pages:

- If SLA = 500 μs, the allowed number of pages is approximately

$$\frac{500}{20.2} \approx 24.75$$

- If SLA = 200 μs, the allowed number of pages is approximately

$$\frac{200}{20.2} \approx 9.9$$

However, in the experiments, under the default workload, all SSD, and `nprobe=1` , the average already requires about 43.9 pages; under hotspot / Zipf and other workloads, all SSD with `nprobe=1` has `io_amp ≈ 180` , corresponding to more than 220 pages per query. Even when the DRAM fraction reaches 0.2 and we use relatively good policies (such as window LFU) , in many scenarios the average page count is still higher than 25 pages.

This leads to two direct consequences. First, as long as there are SSD list scans in the tail, p95 will basically fall into the millisecond range. Second, under `SLA=200μs / 500μs` , all SSD-involving configurations have no feasible solution in `agg/sla_reachable_recall` , with `best_nprobe = -1` , and only all DRAM can satisfy the SLA.

This means that “giving a bit more DRAM + using a good caching policy” is not enough to pull the system into the sub-ms range, because the granularity of data on SSD is too large: once there is a miss, even if only one or two lists miss, you will still scan dozens or even hundreds of pages.

To really push p95 down, the key is to reduce how much SSD data a single query will touch, rather than just increasing DRAM. Several directions can be considered:

- Store lighter-weight representations on SSD. For example, store only PQ codes or low-dimensional projections on SSD, use these low-cost codes to do a coarse pre-filter, and then pull back only a very small number of candidates into DRAM for reranking, turning “one miss reads 0.4–0.9 MB” into “reading tens of KB”.
- Reduce the number of lists that need to be accessed. Use stronger routing or graph structures (similar to DiskANN, SPANN) to reduce the number of lists that must be probed; or transform “a few large lists” into “many small lists”, so that even if `nprobe` remains unchanged, the total bytes become smaller.

- Increase flexibility in caching granularity. In the current experiments the caching unit is an entire list; if we instead cache the head of a list, hot subsegments, or recently accessed blocks, limited DRAM could cover more of the tail, avoiding reading a full list from SSD on every miss.

In summary, under the current configuration, SSD I/O is the dominant term of tail latency: as long as SSD appears in the tail path, p95 naturally rises by two orders of magnitude; to achieve sub-ms SLA, either almost all queries must stay in DRAM, or accesses on SSD must be made sufficiently “few and light”.

8.2 Tradeoff between adaptability and migration

The fast–fast hotspot experiments nicely illustrate the tension between “adaptability” and “migration cost” in caching policies.

8.2.1 Strong adaptability: lower IOAmp, huge migration

Under hotspot fast–fast with DRAM at 5%:

- seconds rule / LRU can push `io_amp` down from ~180 for all SSD to around ~120;
- with DRAM at 20%, `io_amp` can even drop to ~30, and p95 is reduced from 8+ ms to ~5 ms;
- at the same time, migrated bytes soar to the hundreds-of-MB to GB range.

When hotspots move very quickly, such “high-adaptivity” policies do have value: those few large lists that just turned hot can be quickly pulled into DRAM, significantly reducing SSD scan counts and page counts.

Mechanistically, LRU / seconds rule are essentially “time-locality-first” policies: lists that have just been accessed frequently are quickly pulled into DRAM, and once hotspots shift, old hotspot lists are quickly kicked out. This matches the fast–fast hotspot workload very well, so the `io_amp` curve drops fastest.

The problem lies in migration cost: every time the hotspot shifts, a batch of very large lists must be moved, repeatedly migrating them between SSD and DRAM; from a total-bytes perspective, this may even be more expensive than simply scanning from SSD every time. In other words, I/O in the query path is reduced, but “migration I/O” is amplified.

8.2.2 Strong stability: low migration, slightly higher IOAmp

Compared with seconds rule / LRU, naive LFU and window LFU are clearly “more stable”:

- Under fast–fast, their `io_amp` and p95 are slightly higher than seconds rule / LRU (for example, p95 at 6.5–7 ms vs 5–6 ms) , but migration is controlled in the tens-to-hundreds-of-MB range, an order of magnitude lower.
- Under hotspot slow–fast and Zipf, window LFU / naive LFU can basically achieve tail latency close to seconds rule with much smaller migration overhead. In the slow–fast scenario, hotspots move more slowly, giving frequency statistics sufficient time to converge; in the Zipf scenario, hot clusters are stably hot for a long time, and long-term frequencies are essentially the true hotness.

Intuitively, strong adaptability means the residency set changes frequently, tracking hotspots in a timely manner, but with a lot of migration; strong stability means the residency set changes slowly, with low migration, but reacts relatively slowly when hotspots suddenly switch. window LFU sits between the two: compared with naive LFU it has “forgetting” capability and will not remember very old hotspots forever; compared with LRU / seconds rule it is much smoother and will not completely reshuffle the cache due to very short-term fluctuations.

8.2.3 Incorporating migration cost into the objective

If we want to use these policies in real systems in the future, looking only at `io_amp` / p95 is insufficient. Migration bytes themselves bring pressure on SSD bandwidth and lifetime, contention for DRAM bandwidth, and extra CPU overhead for participating in migration.

A more reasonable approach is to explicitly incorporate migration cost into the optimization objective. For example, for each list we could compute something like

$$\text{benefit density} = \frac{\text{expected reduction in SSD bytes}}{\text{migration bytes}}$$

and only bring into DRAM those lists with the highest “benefit density”, rather than simply sorting by access count. This can limit migration oscillations caused by very large lists, and avoid repeatedly moving in and out a batch of huge lists with only moderate access frequency in the name of chasing hotspots. If we include I/O caused by migration into total I/O and directly compare policies using “query I/O + migration I/O”, we will get closer to the true cost.

8.3 nprobe choice is still fundamental

Whether or not SSD is involved, `nprobe` is still the main knob controlling the recall–latency tradeoff. The all DRAM results in Section 7 show a stable pattern:

- `nprobe = 1` : recall ~ 0.5–0.6, extremely low latency;
- `nprobe = 2` : recall ~ 0.7–0.8;
- `nprobe = 4` : recall ~ 0.85–0.9, already close to a clear knee point;
- `nprobe = 8` : recall ~ 0.95 or so;
- `nprobe = 16` : recall ~ 0.998;
- `nprobe = 32` : recall approaches 1, with strongly diminishing returns.

Under all DRAM, latency mainly comes from operator cost, and this “multiplication” is still relatively mild. Once SSD is added, the amplification effect of `nprobe` basically becomes “page count grows by a multiplicative factor”: under the default configuration, all SSD with `nprobe=1` already uses dozens of pages; doubling `nprobe` from 1 pushes `io_amp` and page count to grow roughly by a factor, naturally multiplying p95.

Therefore, in systems involving SSD, a more reasonable idea is not “first crank `nprobe` up to reach 0.999 recall and then rely on caching to bail us out”, but to jointly tune around a “page budget”.

We can first back out the allowable I/O budget from the SLA. For example, with SLA = 5 ms and page cost ≈ 20 µs, the allowed total page count is about 250 pages; if we reserve 0.5–1 ms for CPU, the remaining I/O budget roughly corresponds to 200 pages. Under a given policy and DRAM fraction, we can estimate the relationship `pages(nprobe, policy, dram)` via offline experiments or online statistics; for example, under the default + 10% DRAM + window LFU scenario, `nprobe=4` might correspond to about 60 pages, `nprobe=8` about 110 pages, and `nprobe=16` about 200 pages.

Based on such a relationship, we can choose the largest `nprobe` within the budget, i.e., choose the configuration that yields the highest recall while satisfying `pages(nprobe, policy, dram) ≤ pages_budget` . For an online system, we can also dynamically adjust `nprobe` based on current load and observed p95: when IOPS approaches its bottleneck or p95 exceeds the budget, automatically tighten `nprobe` from 8 down to 4, sacrificing some recall in exchange for more stable tail latency.

In other words, the choice of `nprobe` should not be discussed separately from caching policy and DRAM fraction, but instead jointly optimized within a unified “page budget / SLA budget” framework.

8.4 Practical configuration hints

Combining the results from Section 7, we can give several engineering-oriented configuration suggestions (not as rigorous proofs, but as tuning experience summaries) .

Strict sub-ms SLA (200–500 µs)
Under the current model and data scale, only all DRAM can achieve such SLA. If SSD must be involved, we can only consider a very small amount of fine-grained SSD I/O, for example SSD storing only PQ codes or block-level reads/writes, which already belongs to architectural redesign at the system level, rather than being a simple `nprobe` / caching-policy tuning problem.

Millisecond-level SLA (e.g., 5–20 ms) with SSD
Under fast–fast hotspot scenarios, if higher migration overhead is acceptable, we can use seconds rule / LRU with a relatively high DRAM fraction (10–20%) , using strong adaptability to aggressively chase hotspots and push `io_amp` very low; if system-wide I/O and stability are more important, window LFU should be preferred, with slightly higher tail latency but far lower migration overhead. Under hotspot slow–fast and Zipf, window LFU / naive LFU are basically good enough, and the migration premium of seconds rule / LRU is usually not worth it; raising DRAM fraction from 5% to 10% yields significant benefits, and from 10% to 20% gives more incremental improvement. For the default / uniform scenarios, hotspots are not concentrated enough and the improvement space from DRAM is limited; it is better to rely on `nprobe` to control recall than to expect caching to “turn the tables”; a unified cache policy of window LFU is sufficient, while LRU / seconds rule often only increases migration for little gain.

Recommended nprobe range
If only moderate recall is needed and we want latency as low as possible, we can choose `nprobe ∈ {1, 2}` . If we want recall around 0.9, we can choose `nprobe ∈ {4, 8}` as a tradeoff. If recall must be extremely high (>0.98) , we can consider `nprobe ∈ {16, 32}` , but must accept that tail latency and I/O will grow by multiples.

In scenarios involving SSD, `nprobe > 16` is more suitable for offline batch processing or weaker-SLA requests, rather than online queries with the strictest tail-latency requirements.

Writing policy parameters into a “budget sheet”
From an engineering perspective, tuning can be organized into a “budget sheet”: given SLA, page cost, traffic, and DRAM fraction, solve for appropriate `policy` (such as window LFU / seconds rule) , `nprobe` , and permitted migration rate. Online, we can add a simple adaptive control loop: once we observe p95 exceeding the budget or migration bytes surpassing the target, automatically tighten `nprobe` or switch to a more stable policy (for example, from seconds rule back to window LFU) .

Overall, this set of experiments demonstrates three things. First, SSD I/O is the absolute dominant factor in tail latency: either we reduce the number of misses, or we reduce the cost of each miss; otherwise, sub-ms SLA is essentially unattainable. Second, cache policy choice is essentially about picking a point in the “adaptability vs migration cost” space, and window LFU gives a fairly robust tradeoff under most realistic workloads. Third, `nprobe` is still the most fundamental and powerful tuning knob when designing ANN services; it must be considered jointly with the I/O budget and caching policy, rather than tuned in isolation.

9. Lessons Learned

Combining this round of experiments and analysis, we can compress the more “hard” takeaways into two categories: one is experience conclusions that are already relatively clear, the other is directions that would be worth prioritizing for improvement if more time were invested.

9.1 Experience conclusions

p95 reflects SSD risk better than average latency

Average latency looks “okay” under many configurations, but once we look at p95, we see that as long as there is any SSD list scan mixed into the tail, latency immediately jumps to the millisecond range. In other words, average latency systematically underestimates the impact of SSD on online experience, and what truly exposes the bottleneck is p95/p99.

IOAmp is the key intermediate metric for IVF+SSD

Under the current model, p95 is almost linearly related to “how many SSD pages are read”, and `io_amp` is exactly a proxy for page count after aggregating workload, caching policy, and nprobe. As long as we keep an eye on IOAmp, it is basically equivalent to keeping an eye on latency in the SSD scenario; this is more intuitive than repeatedly inspecting p95 across different workloads, and is more convenient for policy optimization.

Caching at cluster granularity is simple but migration behavior can easily “run wild”

The current implementation uses list/cluster as the minimum caching unit: the implementation is simple and statistics are also convenient, but once list sizes differ significantly, policies can exhibit very extreme behavior

—for example, frequently migrating huge lists to chase hotspots, causing migration bytes to soar to the GB level, with maintenance cost far exceeding the SSD I/O of queries themselves.

LRU / seconds rule: high adaptability must be used together with migration constraints

LRU and seconds rule are indeed very sensitive to fast-changing hotspots (hotspot fast–fast), and can significantly push IOAmp down; but if no constraints are placed on migration, we end up in a situation where "latency goes down, but data is being shuttled between SSD↔DRAM every second". They are better suited as "high-adaptability options" combined with explicit migration budgets, rather than as default policies.

window LFU is a safer default policy

Across multiple workloads—default / hotspot slow–fast / Zipf / uniform—window LFU offers the most stable tradeoff between p95 and migration: latency is close to the best scheme, while migration is clearly an order of magnitude lower. In engineering practice, if only one default policy can be chosen, window LFU is more flexible than naive LFU and much more stable than LRU/seconds rule, making it a "low-tuning" safe choice.

The current results are better suited for "mechanism analysis", not strict statistical conclusions

In this round of experiments, `seeds = 1` for all configurations, so confidence intervals are formally 0 and we cannot see cross-run variance. The existing results are very suitable for observing mechanisms and comparing policy trends (which is more aggressive, which has larger migration, which is more affected by workload), but if we want to write stricter statistical conclusions, we must introduce multiple seeds and provide error bars and significance analysis.

9.2 Directions worth pursuing further

Introduce PQ/compression on the SSD side to directly reduce SSD bytes

At the moment, a single miss may scan 0.4–0.9 MB of raw list; if we can store only PQ codes / low-dimensional vectors on SSD, first use lightweight representations for coarse ranking, and then pull back only a tiny set of candidates into DRAM for reranking, we can reduce IOAmp and the cost of a single miss at the root, and get closer to the hybrid designs of real systems.

Introduce rate limiting or budgeting for migration

Currently, policies are completely "laissez-faire" with respect to migration, which is why seconds rule / LRU migrate up to the GB level in some scenarios. A more realistic approach is to set a migration budget for each time window (for example, at most how many bytes can be migrated per second / how many lists per minute), and automatically downgrade to more conservative policies or pause migration when the budget is exhausted, thereby keeping total I/O within an acceptable range.

Explore finer-grained caching units

Caching entire lists wastes DRAM’s ability to serve “long-tail hotspots”. We can consider splitting lists into segments (block-level caching), or directly extracting high-frequency vectors/high-frequency segments to cache separately, so that limited DRAM focuses more on “high-value bytes”, alleviating migration oscillations caused by large lists.

Introduce a more realistic I/O behavior model

The current I/O model is fixed page latency + IOPS cap, lacking common optimizations from real systems such as sequential reads, prefetching, and merged access. Follow-up work can add mechanisms such as sequential-page discounts, read-ahead, and batch fetch in the simulation, or be calibrated against traces from real devices/kernels; this would make the conclusions more transferable in engineering practice, rather than just an analysis under an “idealized page model”.

10. Reproducibility

10.1 Project Structure Overview

Project Structure Tree

```
.
.
├── FINAL_REPORT.md                # Manually curated final report (this file can be merged here)
├── README.md                      # Project overview and brief description
├── requirements.txt               # Python dependency list
├── configs/                       # Experiment configs for each workload
│   ├── default.yaml               # Default workload
│   ├── exp_hotspot_fast.yaml      # hotspot fast-fast
│   ├── exp_hotspot_slow.yaml      # hotspot slow-fast
│   ├── exp_uniform.yaml           # uniform
│   └── exp_zipf_s1.2.yaml         # Zipf s=1.2
├── data/
│   ├── get_sift1m.sh              #  (Downloaded via scripts/get_sift1m.sh) ※ actual script is in scri
│   ├── sift1m/                    # SIFT1M data (base / learn / query / groundtruth)
│   └── siftsmall/                 # Small-scale test data
├── scripts/
│   ├── get_sift1m.sh              # Download and extract SIFT data
│   ├── run_all.sh                 # Run all workloads in one shot
│   ├── one_click.sh               # One-click: run experiments + export results
│   ├── one_click_report.sh        # One-click: generate master report from results
│   ├── dump_results.py            # Export COMPACT text summary from agg
│   ├── export_report_assets.py    # Generate report_assets (tables + snippets) for each run
│   └── validate_run.py            # Perform integrity checks for a single run
├── src/
│   ├── ann_engine.py              # Core execution logic for IVF + caching policies
│   ├── policies.py                # Implementations of LRU / LFU / seconds rule and other policies
│   ├── latency_model.py           # I/O latency model
│   ├── experiment_runner.py       # Load configs and orchestrate experiments
│   ├── plotting.py                # Plotting (generate figures/*.png)
│   ├── run_all.py                 # Backend entry point (invoked by run_all.sh/one_click.sh)
│   └── workload.py                # Access pattern definitions for different workloads
├── results/
│   ├── sr_sift_default_fast_*/            # Results for default workload (raw/agg/figures/report_assets
│   ├── sr_sift_exp_hotspot_fast_fast_*/   # Results for hotspot fast-fast
│   ├── sr_sift_exp_hotspot_slow_fast_*/   # Results for hotspot slow-fast
│   ├── sr_sift_exp_uniform_fast_*/        # Results for uniform
│   ├── sr_sift_exp_zipf_s1.2_fast_*/      # Results for Zipf
│   ├── report_master_*.md                 # Automatically stitched master reports
│   ├── log_*.txt                          # Run logs for each run
│   ├── export_*.txt                       # Key summary for each run
│   └── validate_*.txt                     # Validation results for each run
└── tests/
    ├── test_ivf_alignment.py              # IVF alignment and construction checks
    ├── test_latency_model.py              # Unit tests for latency model
    └── test_policies.py                   # Policy behavior tests
```

11. Conclusion

To sum up, this project, on a fixed IVF ANN engine, changes only the dimension of which IVF lists are placed in DRAM and which are placed in SSD, and systematically reveals the limits of what tiered caching can achieve: once a query falls onto SSD, p95 is basically determined by how many pages are read, IOAmp is the key intermediate metric for understanding latency, and nprobe is still the most crucial recall–latency knob;

Under different workloads, no policy is absolutely better or worse——when hotspots change quickly, seconds rule / LRU have the strongest adaptability and can significantly push down IOAmp, but at the cost of GB-level migration overhead, whereas in more “mild” scenarios such as default, slow hotspot, Zipf, and uniform, window LFU achieves almost the same p95 with a clearly smaller migration cost, and is therefore a more reliable default choice in engineering practice;

The experiments also show that under the current page-level I/O model and parameters, it is difficult to achieve an SLA in the 200–500μs range in a hybrid DRAM+SSD setting by merely adding DRAM and tuning caching policies, and to truly push tail latency back into the sub-millisecond range, we must also start from 「reducing the SSD bytes that must be touched per miss」, for example by introducing PQ compression, finer-grained caching units, or stronger routing structures, all of which provide a clear and reproducible experimental baseline for subsequent system design